



in28minutes

Google Cloud

Cloud Digital Leader



What This Course Is All About

Google Cloud: A broad ecosystem of services

- **Our Focus:** Key cloud concepts and Google Cloud services
- **Decision Making:** CDL tests your ability to match business needs with suitable Google Cloud services

Course Approach: Learn key concepts and services through practical examples and business scenarios

Our Goal: Help you discuss cloud solutions with confidence and prepare for the certification



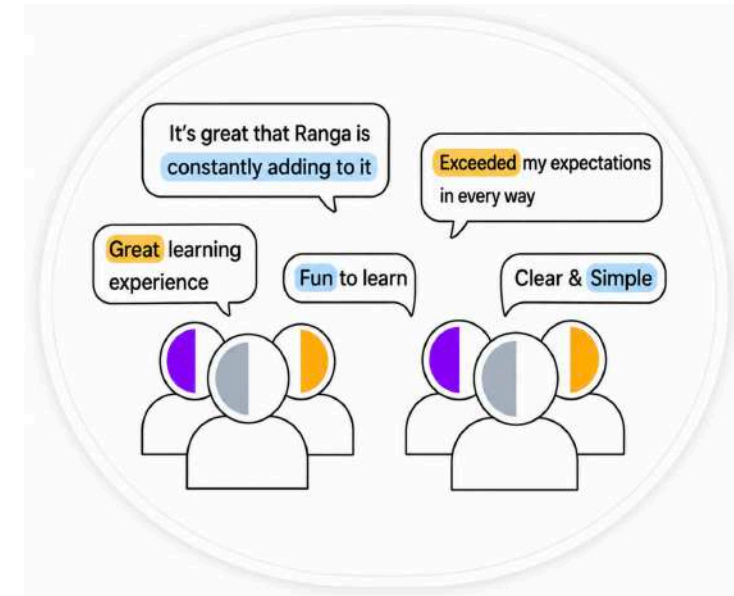


What Learners Are Saying

in28minutes: Trusted by millions of learners worldwide

What Learners Say:

- ★★★★★ - Wow, what can I say? I just passed my second exam this month (NO PRIOR EXPERIENCE). Now I have two Google Cloud Certs (CDL and ACE).
- ★★★★★ - The course was easy to follow and made learning fun.
- ★★★★★ - Very comprehensive course for the Google Cloud Digital Leader certification.





in28minutes

Getting Started with Cloud Fundamentals



The Journey: From Own Data Centers to Cloud

Goal: Understand the enterprise journey from managing data centers to adopting the cloud

The Starting Point: Challenges with managing data centers

- **Servers:** What a server really is
- **Data Centers:** Where servers live
- **The Pain:** Why managing data centers is difficult

The Solution – Cloud

- **The Benefits** – Why the world is moving to the Cloud





How does in28minutes.com work?

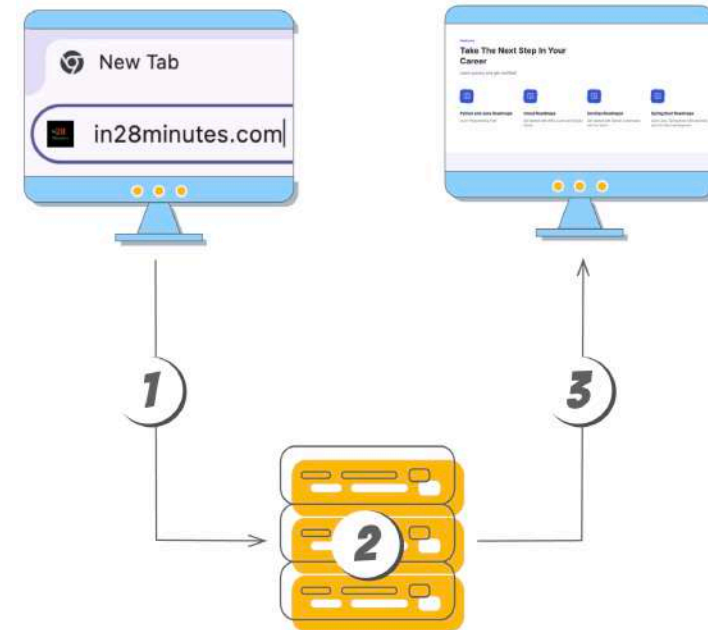
1: Browser - User types URL in the browser

- **Send Request** - Browser finds the server and sends a request to the server

2: Process Request - Server processes the request and prepares the response

3: Send Response - Server sends response back to the browser

- **User Display:** Browser shows the response to the user





What Is a Server?

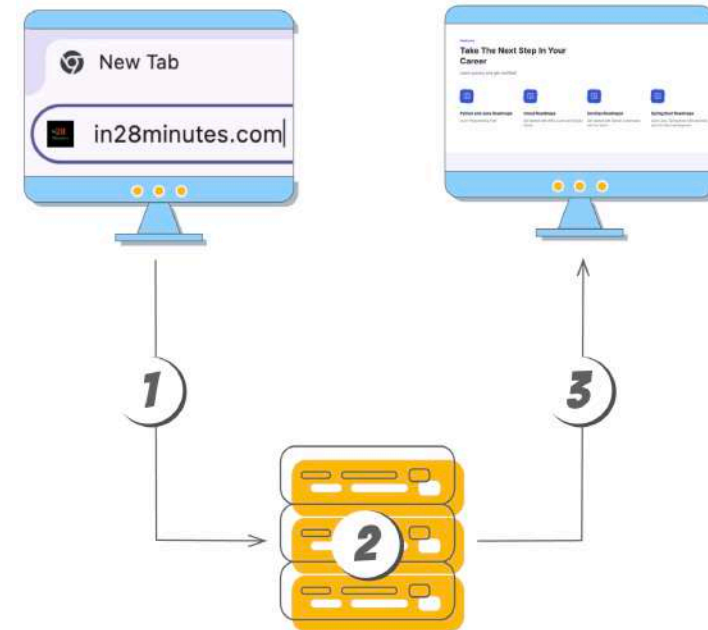
The Core Concept: A server is a powerful computer designed to provide services

- **Server:** The provider (e.g., in28minutes web server)
- **Client:** The requester (e.g., browser)

What Does a Server Serve?:

- **Websites:** HTML pages, images, videos
- **Files:** Documents, photos, backups
- **Emails:** Sending and receiving messages

Summary: Servers provide services, and clients consume those services





Can One Server Handle Everything?

Scenario: in28minutes.com goes viral

- **Traffic Spike:** Millions of users try to access it

Challenge 1: One server has limits

- **Capacity:** Can only handle a limited number of users

Challenge 2: What if the hardware fails?

- **Single Point of Failure !** : If the server fails, the application goes offline

Conclusion: Relying on a single server is risky for most applications





The Enterprise Reality – 1000s of Servers

Scenario: Consider a large bank or a financial institution

- **Lots of Apps:** Websites, mobile applications, internal applications and others
- **Thousands of Servers:** Needed to run all kinds of applications reliably

The Challenge: Not recommended to manage thousands of servers in an office

- **Requirements:** You need massive space and power
- **Question:** Where to house this infrastructure?





What is a Data Center?

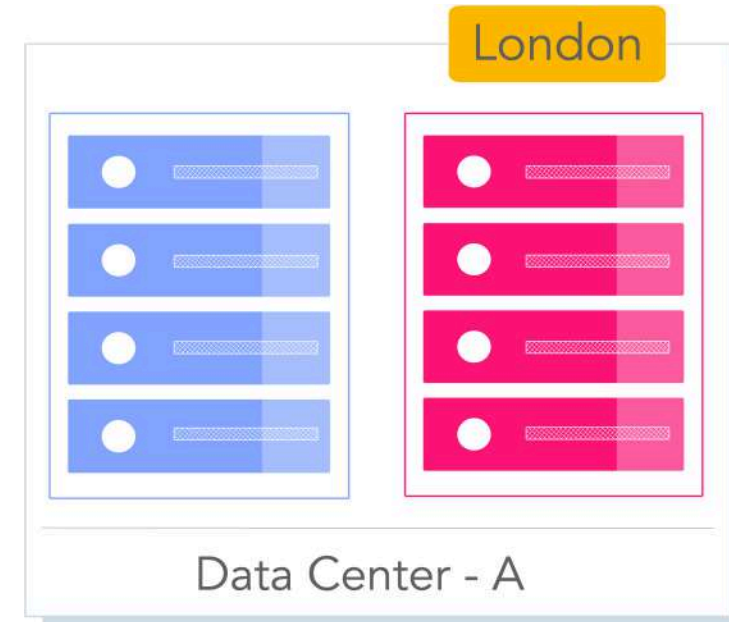
Basic Idea: A secure place for hosting servers

- **Massive Scale:** Houses hundreds, thousands, or even millions of servers

It is Not Just a Building: Needs specialized infrastructure

- **Power:** Uninterrupted power supply
- **Cooling:** Prevent servers from overheating
- **Networking:** High speed connectivity
- **Strong Security Needs:** Hosts key business systems

The Challenge: Building and managing data centers is expensive and complex





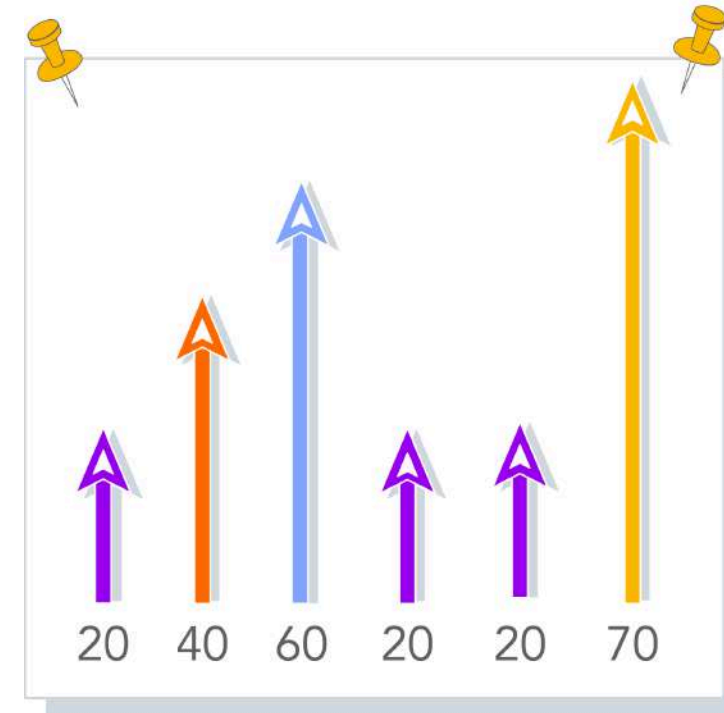
Example 1: Shopping Website

What Happens?: Online shopping demand is not constant

- **Peak Times:** Holidays, sales, weekends
- **Low demand:** During other times

Traditional Approach: Provision For Peak

- **Peak Load Provisioning:** Buy enough servers for peak (maximum) usage
- **Why?:** If you don't provision for peak, the site crashes when you need it most
- **Under Utilized Resources:** Most servers remain idle during periods of low demand





Example 2 : A Startup Succeeds

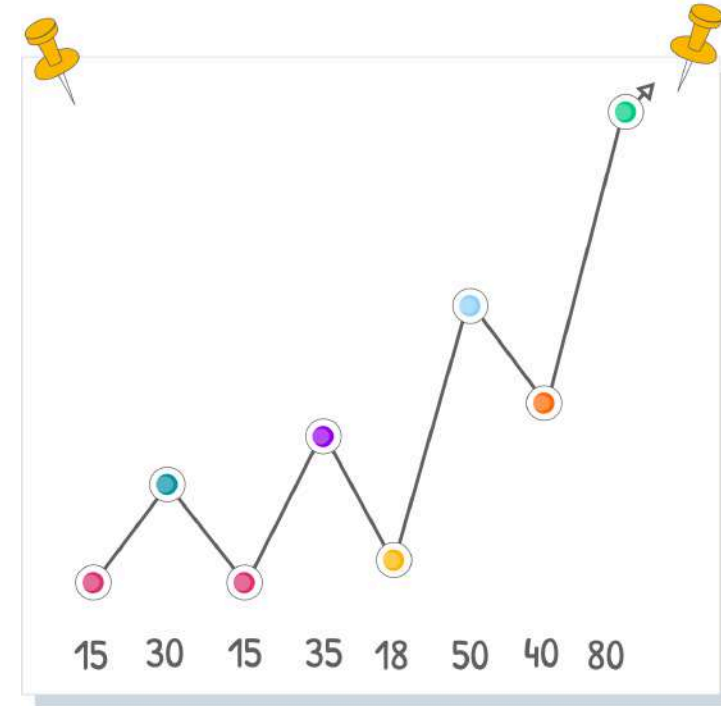
Scenario: Launches a new app that goes viral

The Problem: Unpredictable growth

- **Surge:** Users grow from 1000 to 1 million overnight
- **The Constraint:** Servers take time to buy and install

Traditional Approach: Buy servers in advance

- **Over-Provisioning:** Buy infrastructure in advance for future growth
- **Risk:** What if growth does not happen?
 - **Outcome:** Expensive hardware remains unused





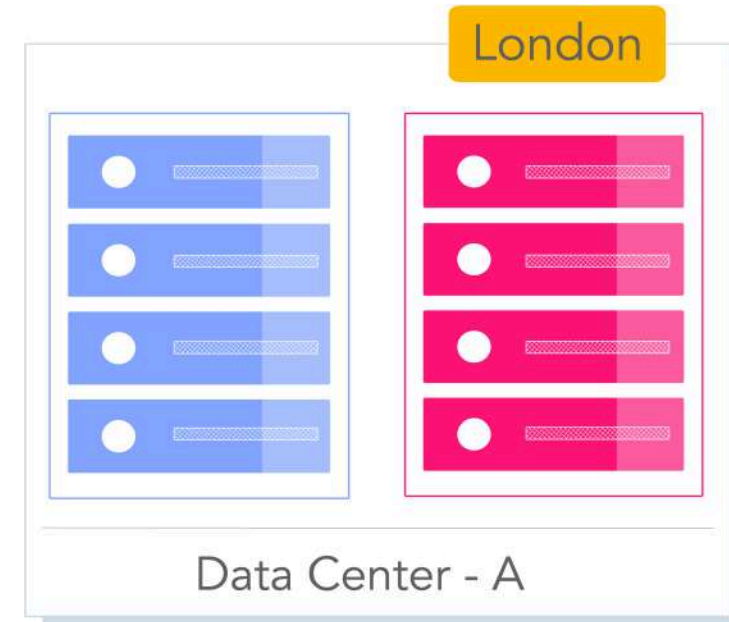
Example 3 : Enterprise Expansion

Scenario: A European company wants to serve Indian customers from a closer location

- **Current Infrastructure:** A Data Center in London
- **Goal:** Enable faster access for Indian users
 - Decrease Latency !

Traditional Approach: Set up a New Data Center in India

- **Time:** Takes months or years to set up
- **Cost:** Massive upfront investment





Challenges In Setting Up Own Data Centers

High Upfront Cost: To Buy Servers

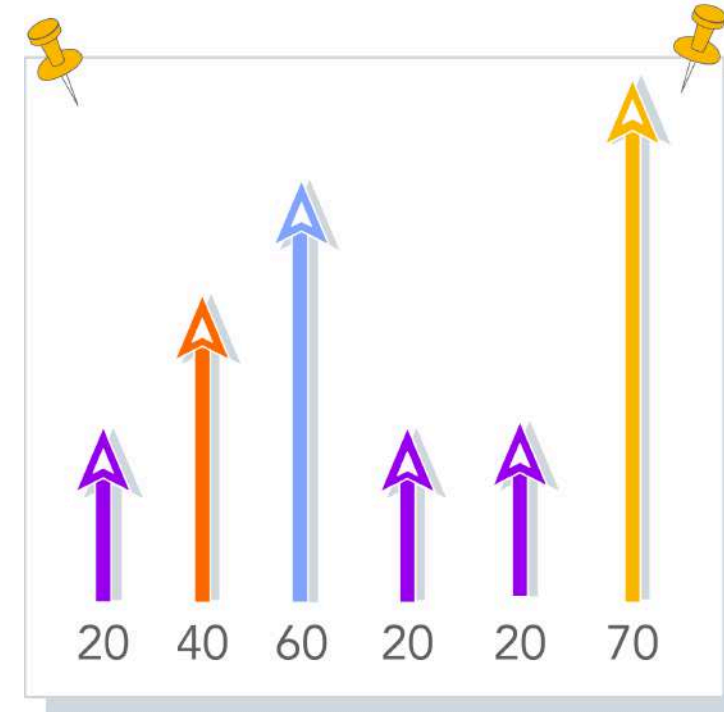
- **Slow Speed:** Takes weeks or months to set up

Needs a Dedicated Team: To maintain infrastructure

- **Think of a startup:** Can a startup afford it?

Guessing Game: You must predict traffic months in advance

Geographic Limitations: Difficult to expand into new locations quickly





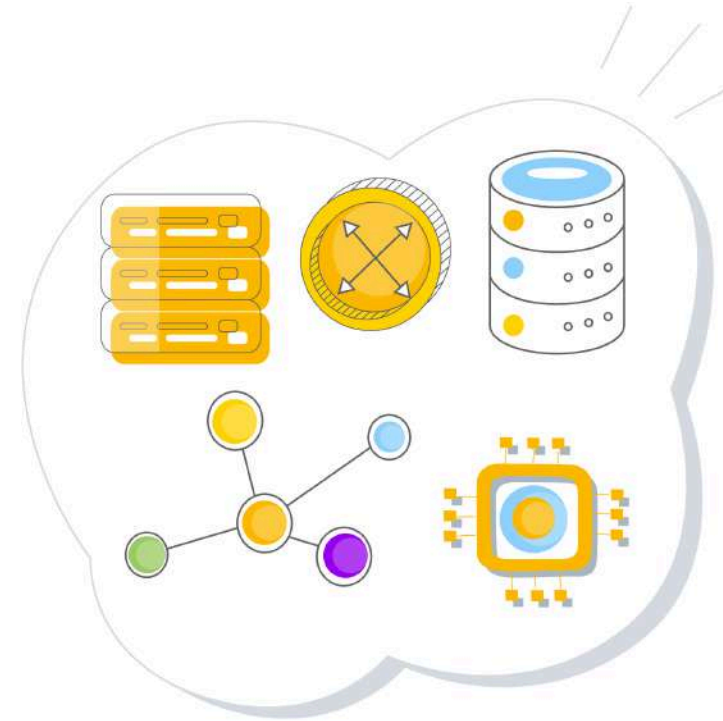
The Cloud: A Silver Lining

Key Question: Why buy infrastructure when you can rent it?

- **Cloud:** Rent servers when you need them!
- **Remember:** Cloud is NOT magic - it's just someone else's infrastructure (servers) that you rent

Core Concepts:

- **On-Demand Access:** Rent resources (servers, storage, databases) instantly
- **Elasticity !** : Increase or decrease the number of resources (servers) based on demand
- **Pay-As-You-Go !** : Pay only for what you use





Cloud Advantage – Pay for Use

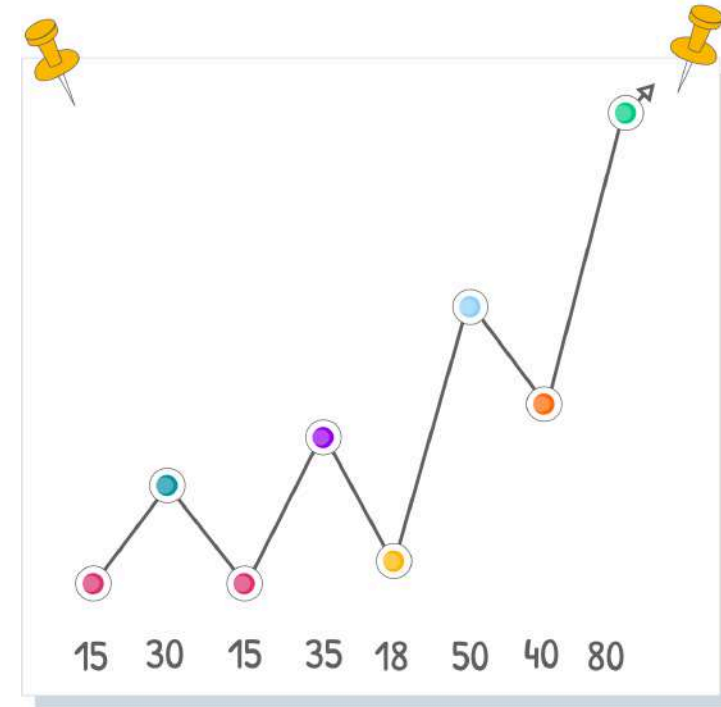
Scenario: A startup launching a new app

Traditional Way: Needs investment to set up a data center before launching

- **CapEx !** : Capital Expenditure or Capital Expense

The Cloud Solution: Rent and start small

- **Rent Servers:** From cloud provider
- **Pay for Use:** Costs only grow if the app grows
- **Lower Risk:** Fail without heavy investment
- **OpEx !** : Operational Expenditure or Variable Expense





Cloud Advantage – Economies of Scale

Scenario: Who gets a better deal?

- **You:** Buying 10 hard disks OR
- **Your Cloud Provider:** Buying 10 million hard disks

Economies of Scale ! : Buying infrastructure in large quantities typically lowers the cost per unit

The Cloud Reality: Cloud providers purchase infrastructure in massive volumes

- **Result:** Lower cost per unit
- **Customer Benefit:** Lower prices passed on to users





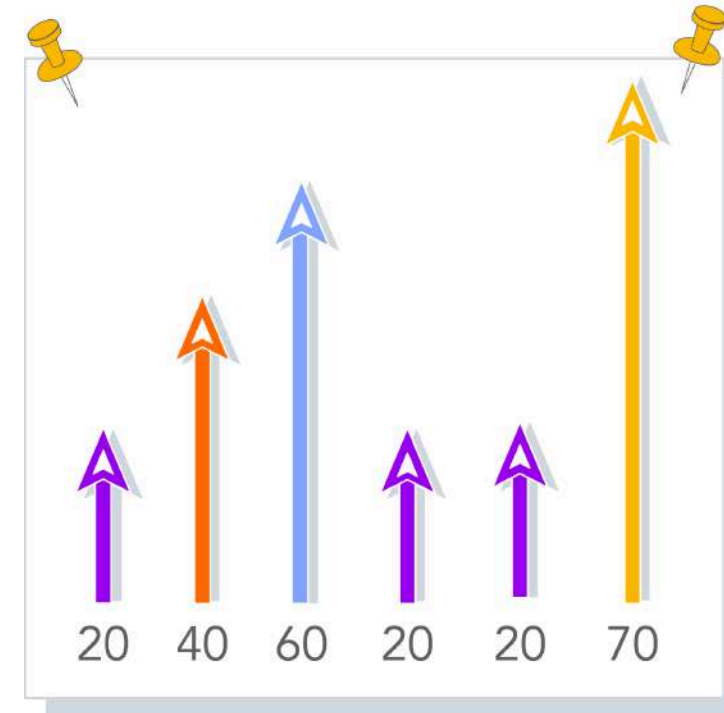
Cloud Advantage – Stop Guessing Capacity

Scenario: Applications and websites need to predict traffic a few weeks ahead of time

- **Not easy:** Difficult to predict accurately
- **Consequence:** Over-provisioning or Under-provisioning

The Cloud Solution: Elasticity !

- **Scale on Demand:** Add more servers instantly when traffic peaks
- **Release Servers:** Release servers when traffic drops
- **Impact:** You do NOT need to pay for idle resources





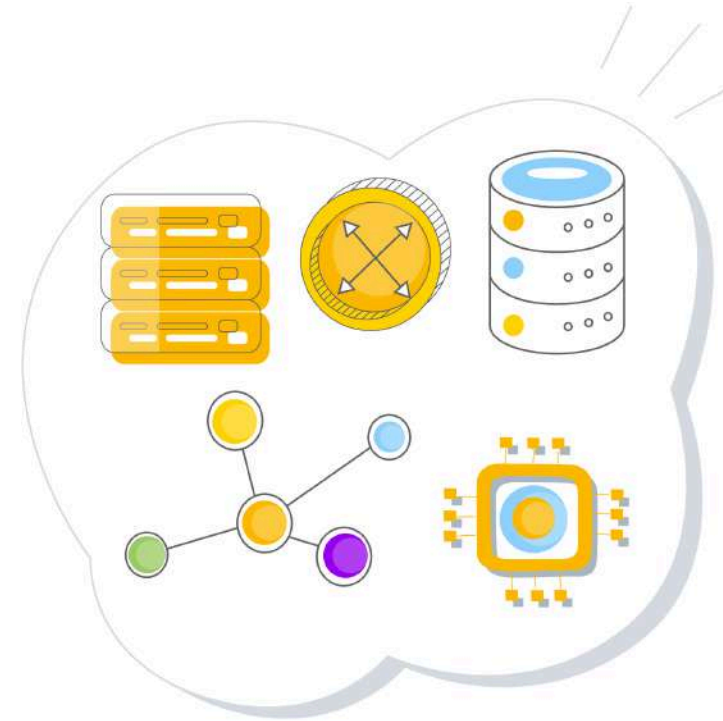
Cloud Advantage – Increased Speed and Agility

Scenario: Your team has a great idea for a new application

- **Traditional Approach:** "Wait 6 weeks for us to buy new servers"
- **Result:** Innovation slows down!

The Cloud Solution: "Click a button, get a server in minutes"

- **Agility !** : The ability to move fast, and respond to changes quickly
- **Faster Time to Market:** Launch features faster





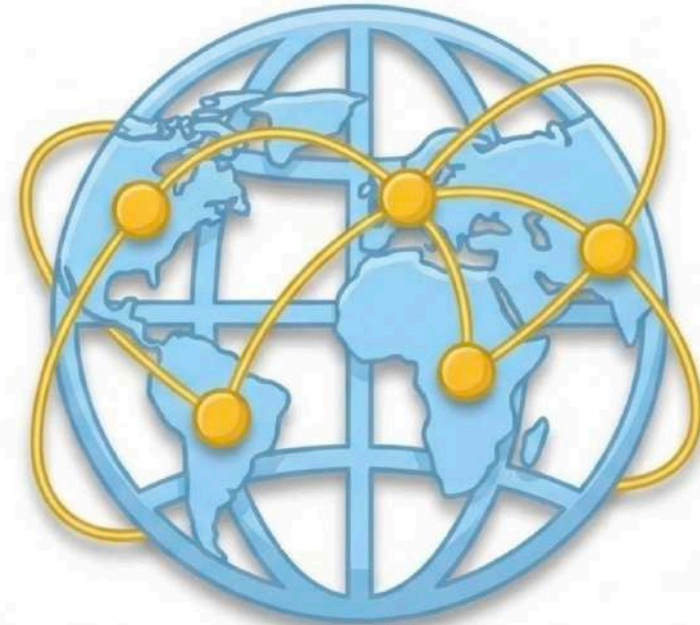
Cloud Advantage – Go Global

Scenario: You have users around the world!

The Challenge: High latency for global users if your servers are only in one location

Cloud Solution: Deploy to multiple regions in minutes

- **Global Reach !** : Expand to new markets around the world quickly
- **Better Experience:** Provide low latency (faster access) to users worldwide
- **Lower Cost:** No need to build data centers and manage infrastructure in multiple locations





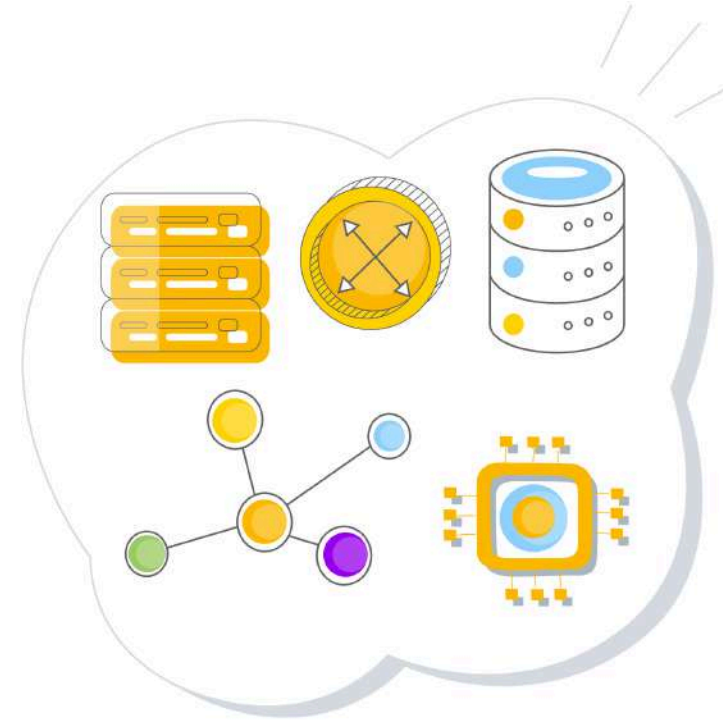
Cloud Advantage – Avoid Heavy Lifting

Scenario: Insurance Company

- 30% on infrastructure, 70% on core business
- **Undifferentiated Heavy Lifting !** : Tasks that are hard but don't make you unique

The Cloud Solution: Cloud reduces infrastructure burden

- **Avoid Undifferentiated Heavy Lifting:** Spend more time on what makes your business unique
- **Example – Insurance Company:** Get better at calculating premiums or detecting fraud (using AI)





Cloud – Key Terminology to Remember (1 of 2)

Term	Description
Elasticity	Ability to automatically scale resources up or down based on demand
Agility	Ability to move fast and respond quickly to business needs Quickly deliver software Quickly provision infrastructure
Global Reach and Geo-Distribution	The ability to expand to new markets and deploy applications to data centers around the world quickly
Availability	Are applications available when users need them
Latency	Time taken for a request to travel between user and server Reduced by serving users from nearby locations



Cloud – Key Terminology to Remember (2 of 2)

Term	Description
Capital Expenditure (CapEx)	Large upfront investment - For example, in infrastructure like servers and data centers
Operational Expenditure (OpEx)	Ongoing cost for renting resources (or services) as needed
Pay As You Go	Trade capital expense (CapEx) for variable expense(OpEx). Pay only for the resources you actually use.
Economies of Scale	Buying resources in large quantities lowers the cost per unit
Avoid Undifferentiated Heavy Lifting	Spend more time on what makes your business unique Example: Insurance company focusing on calculating better premiums.



Cloud Concepts – A Few Scenarios (1 of 2)

Scenario	Cloud Concept
Netflix adds thousands of servers instantly on Friday nights to stream movies to millions, then releases them on Monday morning.	Elasticity - increase or decrease no of servers based on demand
A startup launches a new app with minimal cost and increases spending only as user adoption grows	Pay As You Go or Operational Expenditure
A product team provisions infrastructure in minutes to test new features and releases updates faster	Agility
A healthcare application ensures patients can access services 24x7, even during maintenance or failures	High Availability



Cloud Concepts – A Few Scenarios (2 of 2)

Scenario	Cloud Concept
A SaaS company deploys its application in multiple regions to serve customers worldwide with low latency	Global Reach and Geo-Distribution
A media streaming platform serves videos from locations closest to users for faster playback	Low Latency
A cloud provider buys hardware in massive quantities and offers lower prices to customers	Economies of Scale
An insurance company focuses on fraud detection and customer experience while the cloud provider manages infrastructure	Avoid Undifferentiated Heavy Lifting



Google Cloud - Google's Cloud Platform

Built by Google: Runs powerful infrastructure driving products serving over billion users - Google Search, Gmail, YouTube among others

- **The Best Network:** Google owns one of the world's largest private fiber-optic networks
- **Speed:** Your data travels on Google's own private "internet" for faster, more secure connections
- **Data & AI DNA:** Widely recognized as the leader in Data Analytics and AI

Enterprise Ready: Used by startups and massive enterprises

- **Massive Service Catalog:** Hundreds of cloud services





We Worked Out the Best Path to Learn Cloud

So Many Services!: Compute, Databases, Storage, Networking, Analytics, and more

Overwhelming Choices – You could learn them one by one

- **Risk** – Easy to get lost in the details

The Big Question: *Where should you start?*

Our Solution – Structured, step-by-step learning path

Goal – Make learning cloud easy, practical, and fun





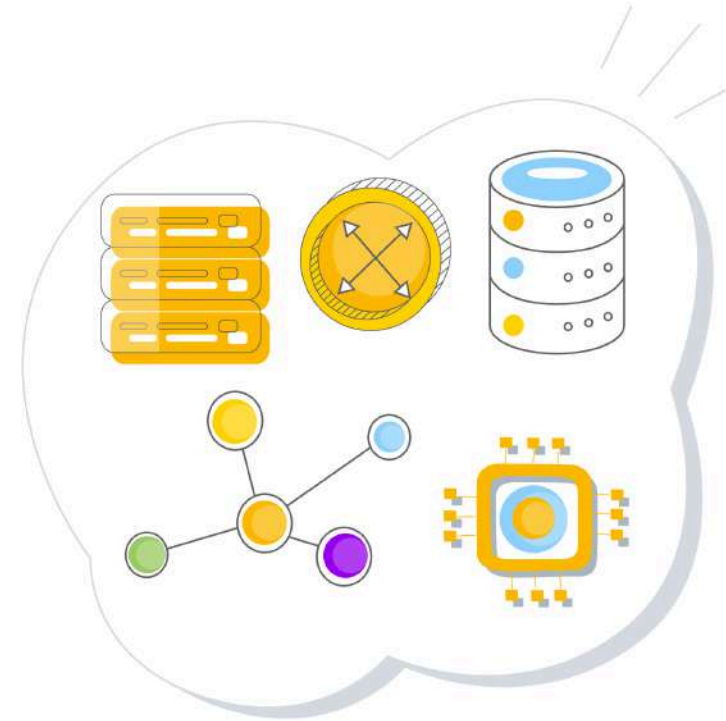
Setting Up Your Cloud Account

Gateway to Cloud – You need an account to access services

- **Global Access** – Deploy resources in regions around the world
- **Free Tier** – Most cloud providers offer free credits for beginners
- **Benefit** – Learn and experiment without immediate cost

First Step – Create your account to play with the cloud

- **I'm Excited!:** To Get You Started





in28minutes

Getting Started with Regions and Zones

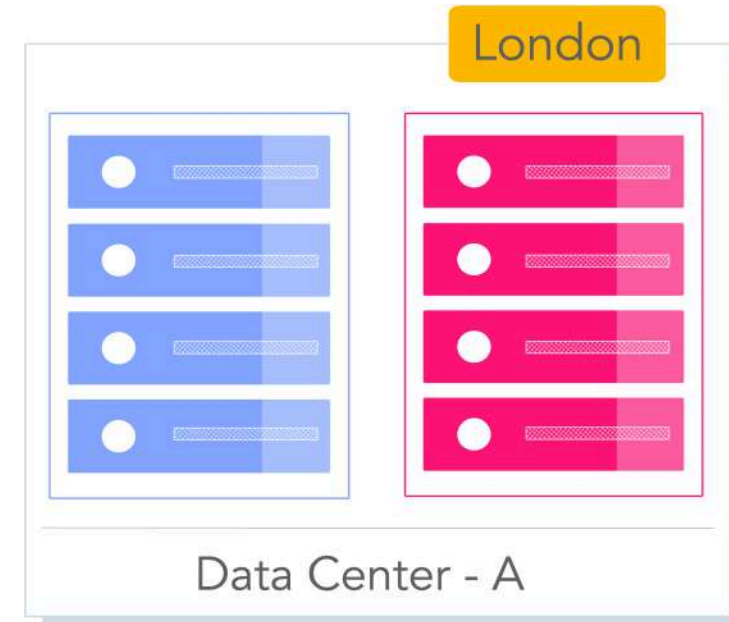


Scenario - Application in One Data Center

Scenario: Imagine that your application is deployed in a data center in London

Challenges:

- **Challenge 1 :** Slow access for users from other parts of the world (**High Latency !**)
- **Challenge 2 :** What if the data center crashes?
 - Your application goes down (**Low Availability !**)





Scenario - Application in Multiple data centers

Scenario: Let's add in one more data center in London

Challenges:

- **Challenge 1** : Slow access for users from other parts of the world
- **Challenge 2 (SOLVED)** : What if one data center crashes?
 - Your application is **still available** from the other data center
- **Challenge 3** : What if **entire region** of London is unavailable?
 - Your application goes down





Scenario - Add a New Region

Scenario: Let's add a new region - Mumbai

Challenges:

- **Challenge 1 (PARTLY SOLVED)** : Slow access for users from other parts of the world
 - You can solve this by adding deployments for your applications in other regions
- **Challenge 2 (SOLVED)** : What if one data center crashes?
 - Your application is still live from the other data centers
- **Challenge 3 (SOLVED)** : What if entire region of London is unavailable?
 - Your application is served from Mumbai





Regions

The Challenge: Imagine a startup in India setting up data centers in different regions around the world

- Would that be easy?

Cloud Makes It Easy: Cloud Providers provide Regions all around the world

Region : A specific geographical location to host your resources

- **Examples:** New York, London, Mumbai, Sydney





Choosing The Right Region - Factors

1 – Compliance ! : Be aware about regulations

- **Why:** Some countries require citizens data to stay within borders

2 – Latency: Be close to your users

- **Goal:** Choose a region closest to the majority of your users

3 - Services Available: Does the region support the services you need?

- **Note** – New services often launch in specific regions first

4 – Pricing: Check pricing in different regions

- **Why:** Costs can vary between regions for same resources





Why Use Multiple Regions?

Low Latency – Serve users faster by placing resources closer to them

Global Footprint – Expand services and reach customers worldwide

Data Residency – Store data in specific geographic locations to meet regulations

High Availability – Reduce downtime by running apps in multiple regions

Disaster Recovery – Protect against regional failures (e.g., natural disasters)





Zone(s) or AZ(s) - Availability Zones

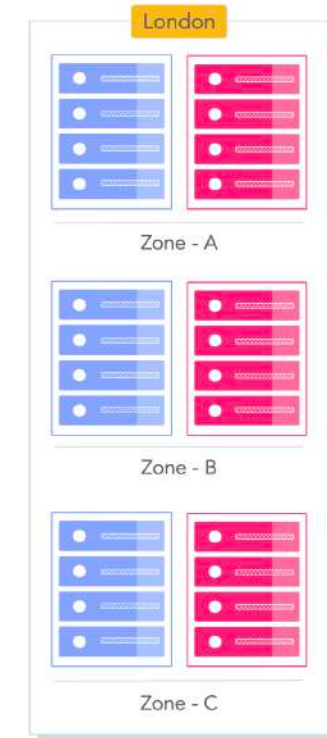
Scenario: How do you achieve high availability within a single Region?

Solution: Use Zones (Availability Zones)

- **Zones:** Isolated locations within a Region
- **Independent:** Each Zone has independent power, n/w & connectivity
- **High Speed Link:** Zones in a region are connected through low-latency links

Advantage: Increased availability and fault tolerance within same region

- Survive the failure of a complete data center





Regions vs. Zones - Quick Summary

Feature	Region	Zone/AZ
Definition	A separate geographic area (e.g., London, Mumbai)	An isolated location <i>within</i> a Region
Distance	Regions are separated by large geographic distances	Zones are connected by ultra-low latency links within the same region
Failure Scope	Deploying in multiple regions protects against massive disasters (earthquakes, floods) affecting a whole region	Deploying in multiple zones in a region protects against local failures (power outage, fire) in a specific building
Primary Goal	Disaster Recovery, Data Residency, Low Latency for global users	High Availability within the same region



Common Misconceptions – Regions and Zones

Misconception	Reality
Regions are only for large enterprises	Cloud makes regions accessible even to startups
All regions cost the same	Costs vary from region to region - based on local taxes and costs of running a data center in that specific region
Using one zone is enough for high availability	Zone failure can still bring apps down
All regions have same number of zones	Number of zones can vary by region
All zones have just one data center	Nope - Some zones can have more than one data centers



Regions and Zones – Scenarios

Scenario	Best Practice
Design highly available applications within a Region	Multi-Zone Deployment
Global SaaS application with users worldwide	Deploy across multiple regions
Survive data center failure	Distribute across Zones
Survive entire city or regional outage	Distribute across Regions
Meet data residency laws	Select specific compliant Region



Google Cloud - Regions and Zones examples

New Regions and Zones are constantly added

Region Code	Region	Zones	Zones List
us-west1	The Dalles, Oregon, North America	3	us-west1-a us-west1-b us-west1-c
eu-north1	Helsinki, Finland, Europe	3	eu-north1-a eu-north1-b eu-north1-c
asia-south1	Mumbai, India APAC	3	asia-south1-a asia-south1-b asia-south1-c



Getting Started with Virtual Machines



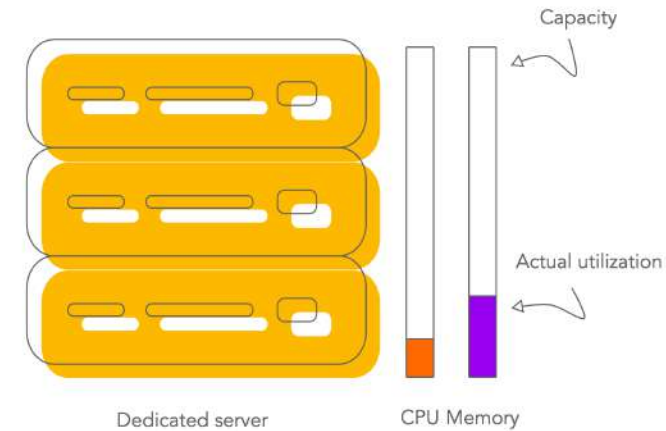
Getting Started – What is Virtualization

Goal: Take you on journey with virtual machines (VMs)

The Journey:

- **Problem:** Why dedicated servers are inefficient
- **Solution:** How virtualization solves the problem (enables creation of multiple VMs on a single physical server)
- **Evolution:** How VMs moved from data centers to cloud

Ready?: Let's get started on this fun journey





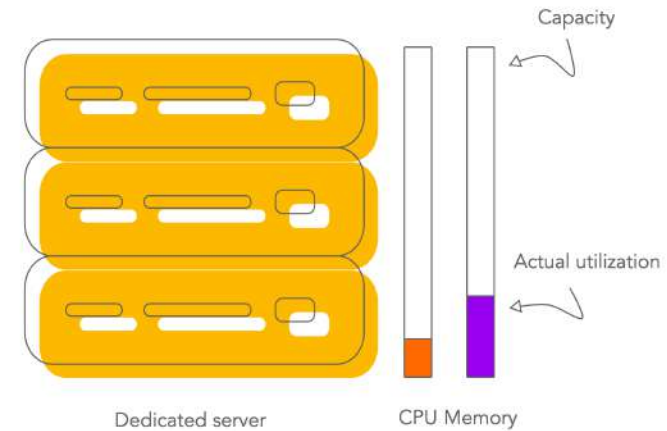
The Dedicated Server Dilemma

Scenario: You deploy an app on a dedicated physical server

Issues:

- **Resource Wastage:** Your app uses only 20% CPU (resources are under utilized)
- **Takes Time:** Setting up a new physical server for each app takes time
- **High Cost:** Buying a new physical server is expensive

Question: How can we do better?





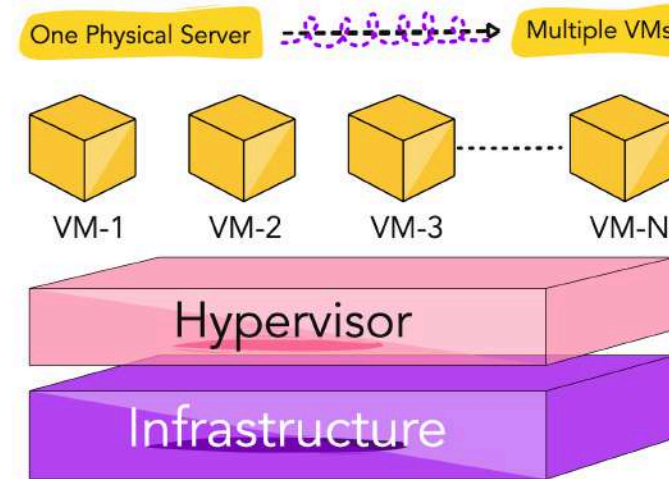
Enter Virtualization – The Solution

Goal: Run 10 apps on 1 server safely and efficiently

How?: Divide one physical server into multiple "virtual" servers

Requirement: A magic software layer

- **Role:** Sits on top of a physical server, creates & manages VMs
- **Resource sharing:** Resources (of the physical server) are shared between VMs
- **Isolation:** A VM should not impact other VMs





VMWare - Driving Virtualization in the Data Center

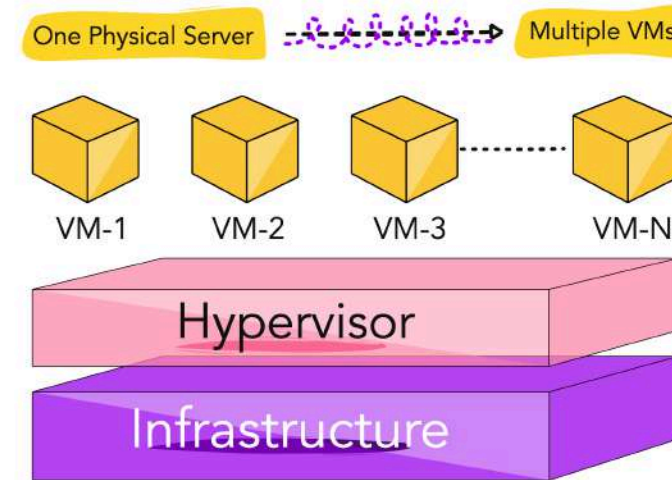
Question: How can multiple virtual machines run on a single physical server?

- **VMware:** Provides virtualization software (hypervisor)

Key Capabilities:

- **Flexibility:** Create, resize, and delete VMs via hypervisor
- **Isolation:** Failure in one VM does not impact others
- **Centralized Management:** Manage servers and VMs at one place

Challenge: Enterprises still need to maintain physical data center infrastructure



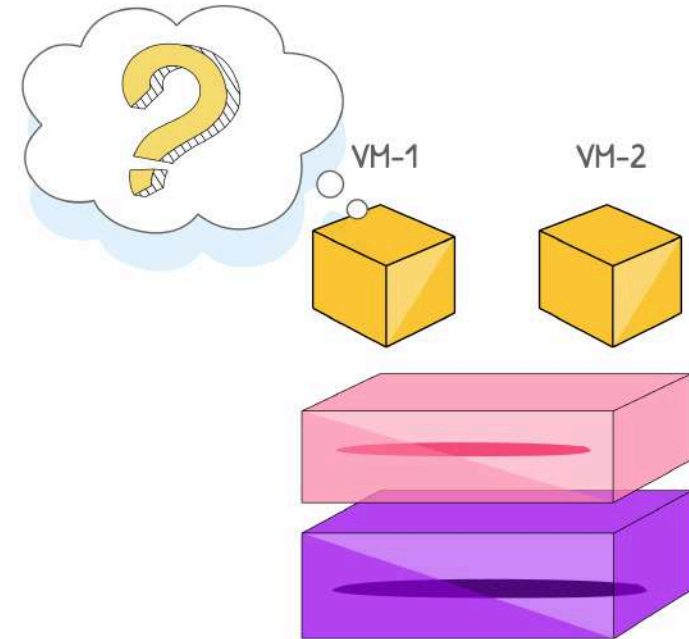


The Cloud Shift – Virtualization on Demand

The Next Step: Moving VMs from "Your Data Center" to "Cloud"

- **AWS:** Amazon EC2 (Elastic Compute Cloud)
- **Azure:** Azure Virtual Machines
- **Google Cloud:** Google Compute Engine
- **You Manage:** The VM configuration, OS, runtime and your app
- **Cloud Provider Manages:** Infrastructure (hardware, power, cooling, maintenance etc.) and Hypervisor

Freedom From Infrastructure: Focus on your VM and your app, not on infrastructure





How To Create Virtual Machines in the Cloud?

1: Choose the Right Service: EC2 or Azure VMs or GCE or ..

2: Choose Your VM Configuration:

- **Pick a Region:** Decide where to run your VM
- **Select an OS:** Choose from Linux, Windows, or ..
- **Pick the Hardware:** Pick CPU, memory, storage, ..

Launch in Minutes: Create a VM in minutes

- **Pay as you go:** Pay only for usage
- **Complete flexibility:** For example, create VMs in London from Hyderabad (India)





in28minutes

Getting Started with Compute Engine



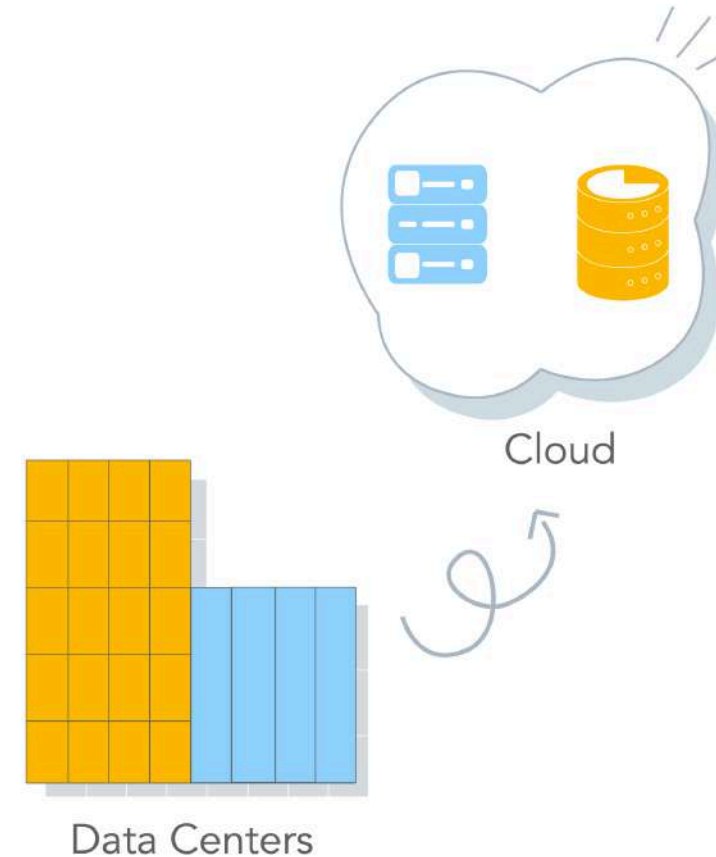
From Data Centers to the Cloud

VMware: Revolutionized data center virtualization

- Run multiple isolated VMs on a single physical server
- **Limitation:** Enterprises still responsible for infrastructure (hardware, networking, power, cooling, security)

Question: What if you could provision VMs without managing physical infrastructure?

- **Compute Engine:** Google Cloud Infrastructure-as-a-Service for creating and managing VMs

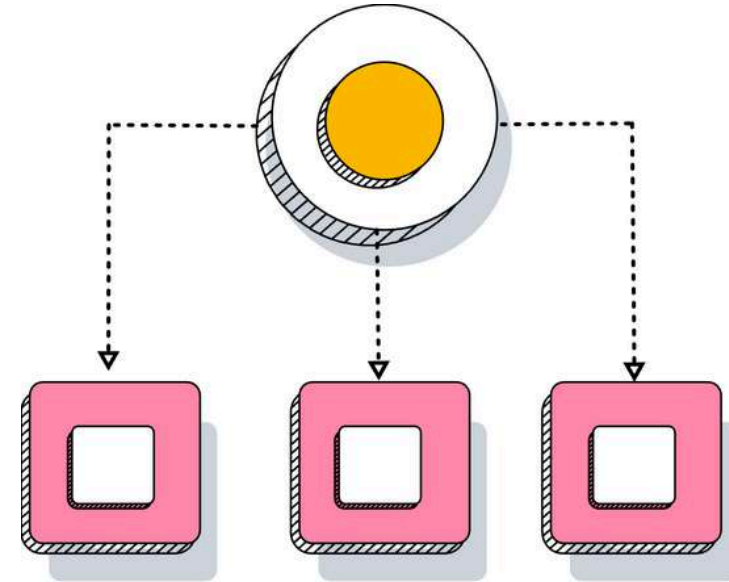




Compute Engine - Create VMs in Google Cloud

Compute Engine: Google Cloud managed service to create & manage VMs

- **Flexible Configuration:** Choose CPU, memory, disk, and OS to match your needs
- **VM Lifecycle Management:** Create, start, stop, resize, or delete virtual machines
- **Global Reach:** Deploy VMs close to your users
- **Pay-As-You-Go:** Pay only for the resources you use
- **Easily Scalable:** Add or remove VMs as needed
- **Group of VMs:** Create multiple VMs
 - **Load Balancing & Auto Scaling:** Handle more traffic automatically





Compute Engine - Key Concepts

Feature	Explanation
Compute Engine instance (VM)	A virtual machine running in Cloud
Image	Choose Operating System and Software
Machine Family	Choose the right family of hardware (General purpose or Compute/Storage/Memory optimized or GPU)
Machine Type (e2-standard-4, e2-standard-8, etc.)	Defines CPU and memory configuration (e.g., 4 vCPUs, 8 vCPUs)
Hyperdisk & Persistent Disk	Attach Virtual Disks to VMs (Block Storage)



Compute Engine Hands-on : Setting up a HTTP server

```
#!/bin/bash
sudo su
apt update
apt -y install apache2
sudo service apache2 start
sudo update-rc.d apache2 enable
echo "Hello World" > /var/www/html/index.html
echo "Hello world from $(hostname) $(hostname -I)" > /var/www/html/index.html
```

Commands Explained:

- `sudo su`: Switch to the root user to run administrative commands
- `apt update`: Refresh package index from APT repositories
- `apt -y install apache2`: Install Apache2 web server
- `sudo service apache2 start`: Start the Apache2 web server
- `sudo update-rc.d apache2 enable`: Enable Apache2 to start automatically on reboot
- `echo "Hello world from $(hostname) $(hostname -I)" > /var/www/html/index.html`: Write a test message with hostname and internal IP to the web page



Google Compute Engine - Scenarios

Scenario	Solution
You want to select hardware for your VM based on workload	Machine Family and Machine Type
What does 8 in e2-standard-8 indicate?	No of vCPUs
You want to choose the operating system and pre-configured software for your VM	Image
You want to attach storage volumes to your VM for data persistence	Persistent Disk / Hyperdisk
You need to ensure that your VM runs a specific operating system and software stack for your application	Custom Image



IP Addresses - Virtual Machines

IP Address	Description
Internal IP Address	Permanent Internal IP Address (restricted to the specific Google Cloud private VPC network) that does not change during the lifetime of an instance [no charge]
External Ephemeral IP Address	Temporary External IP Address (accessible from internet) that might change when an instance is stopped or restarted
External Static IP Address	Permanent External IP Address that can be attached to a VM ["If you reserve a static external IP address and do not assign it to a resource such as a VM instance, you are charged at a higher rate than for static and ephemeral external IP addresses that are in use"]



Compute Engine IP Addresses - Scenarios

Scenario	Solution
You want your VM External IP address to remain the same across stops, restarts, and/or reassignments	Static External IP Address
You do not want your VM to be directly reachable from the internet	Assign Only Internal Ip Address (No External IP Address)
You want to talk to a VM from another resource inside the same private network. Which IP address should you use?	Use Internal IP Address of the VM (Traffic will stay within the network - it will not go over internet)
True or False: If you reserve a static external IP address and do not assign it to a resource such as a VM instance, you are charged at a higher rate than when it is assigned to a VM instance	True



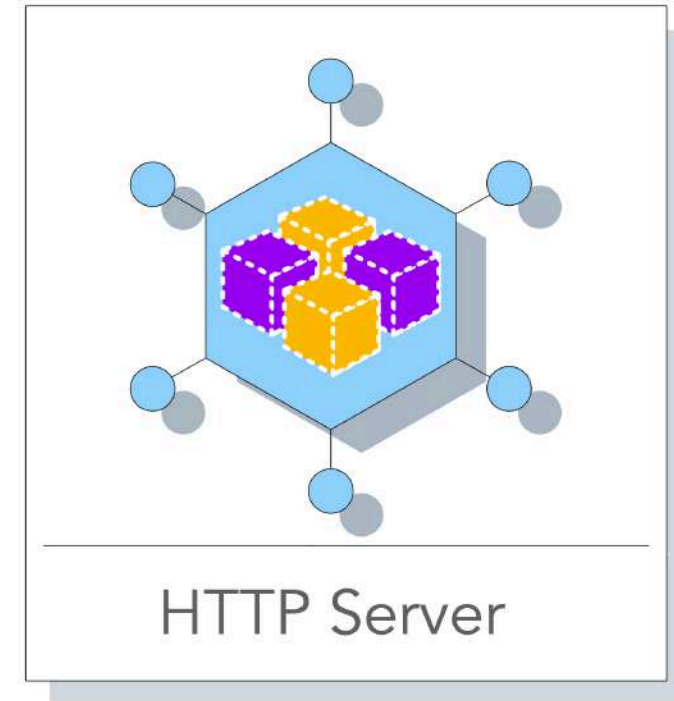
Simplify VM HTTP Server Setup

Goal: Reduce the number of steps in setting up a VM and HTTP server

Startup Script: Run setup automatically when a VM starts

Custom Image: Create ready-to-launch VM Disk Image with everything pre-installed

Instance Template: Reuse predefined VM configuration





Bootstrapping with Startup Script

Requirement: Install software on 10 VMs

- **Problem:** Each time you launch a VM, you need to connect to VM and install software
- **Solution:** Bootstrapping

Bootstrapping: Automatically install software when VM first starts up

- Install web server, patches, .. automatically

Startup Script: VM configuration to perform automatic bootstrapping

- Your VM sets itself up automatically, saving time and effort

```
#!/bin/bash
apt update
apt -y install apache2
echo "Hello world from $(hostname)
$(hostname -i)" > /var/www/html/index.html
```



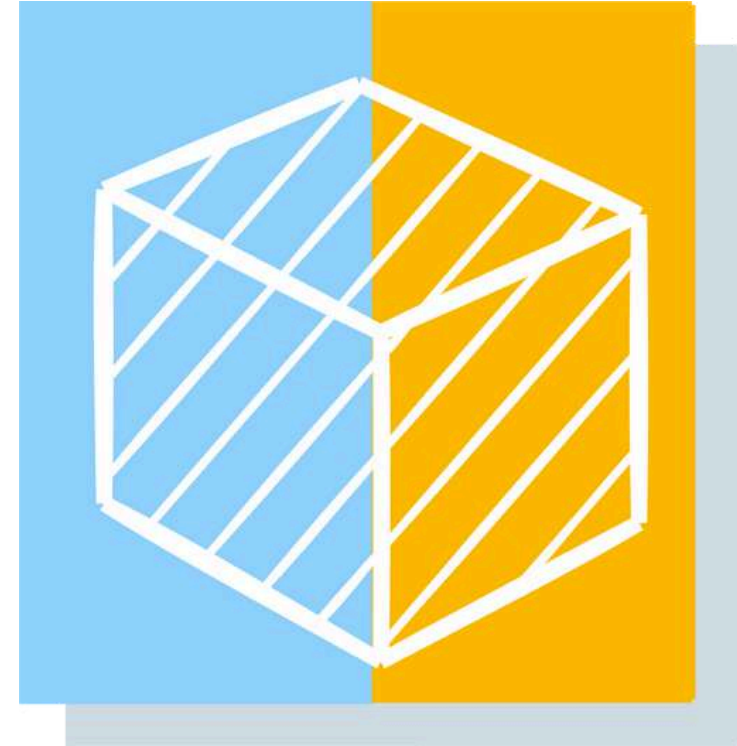
Reducing Launch Time with Custom Image [1/2]

Scenario: Every time you launch a new VM, it installs updates and software from scratch

Problem: Increases your VM startup time

Solution: Create a **Custom Image** with everything pre-installed

- Built from an existing VM, disk, snapshot, or Cloud Storage file
- Can be shared across multiple projects for consistency



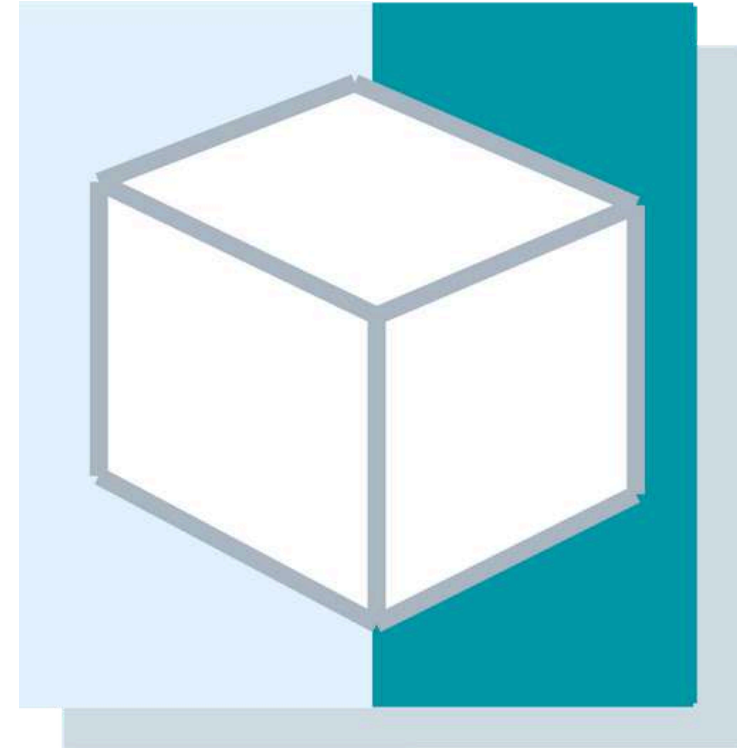


Reducing Launch Time with Custom Image [2/2]

Best Practices:

- Deprecate old images and specify replacements
- **Harden Images** to meet your organization's security standards
- **Prefer using Custom Image to Startup script**

Why It Matters: Custom Images make VM creation faster while being secure and reliable





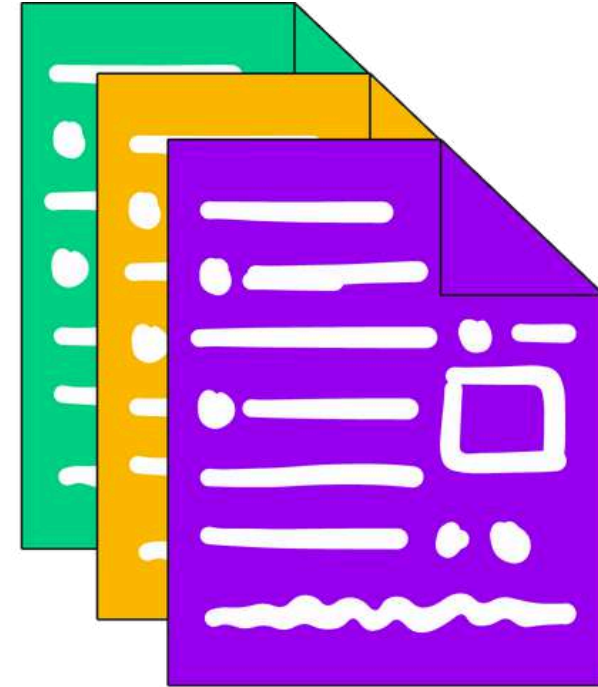
Instance Templates

Scenario: You've just set up (configured) a perfect VM - right image, right machine type, startup script, enabled http access ..

Problem: Each time you create a new VM, you need to configure it all over again

Solution: Use an **Instance Template**

- Configure machine type, image, labels, startup script, ..
- Quickly launch new VMs





Instance Templates - Remember

Multiple Use Cases: Create individual VMs or managed instance groups from same template

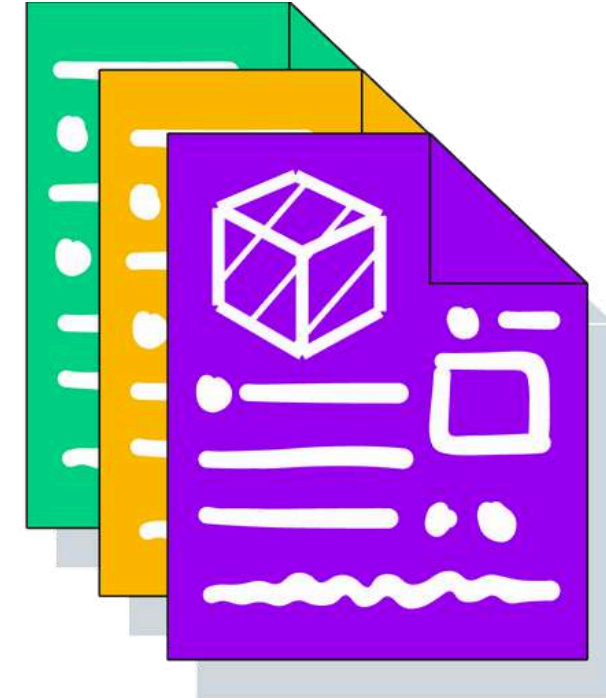
- **Very Convenient:** Launch similar instances quickly without configuring again

Immutable Templates: Cannot modify existing templates once created

- **Change Process:** Copy existing template and modify the copy to make changes

Optional: Specify Image Family(ex:*debian-13*)

- Automatically uses latest image from image family





Compute Engine - Launching VM - Scenarios

Scenario	Solution
You want to run software installation and setup every time a VM starts	Startup Script
You want to launch multiple VMs with the same machine type, image, startup script and other configuration	Instance Template
You want to create Managed Instance Groups using a reusable VM blueprint	Instance Template
You want to reduce VM launch time by pre-installing OS updates and software	Custom Image
You want to create secure, hardened VM images that follow organization standards	Custom Image



Reducing Costs - Compute Engine Virtual Machines

Feature	Description
Sustained use discounts	Automatic discounts for using resources for long periods of time Example: If you use certain machine types for more than 25% of a month, you get a discount on every incremental minute
Committed use discounts	Reserve compute instances ahead of time Commit for 1 year or 3 years Discount varies based on machine type
Spot VMs	Cheaper, temporary instances for non critical workloads (Cheapest but availability not guaranteed!)

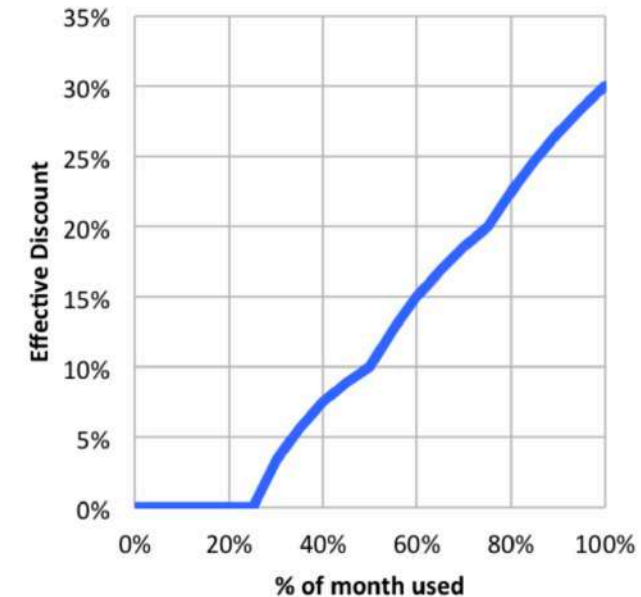


Sustained Use Discounts

Scenario: What if your VM runs for most of the month?

Concept: Google Cloud provides automatic discounts for sustained usage

- **Constraints:** 1) Used for more than 25% of a billing month 2) NOT receiving any other discounts
- Discounts increase gradually with usage over the billing month
- No upfront commitment required (Applied automatically to eligible resources)
- **Applicable To:** VMs of certain machine types in GCE and GKE





Committed Use Discounts

Scenario: What if you know your workload will run continuously for a long time?

Concept: Get a **discount** by committing resources for 1 or 3 years

- Google Cloud offers CUDs in return for purchasing committed use contracts (also known as commitments)
- Commit either to a minimum amount of resource usage or to a minimum spend amount for a duration of one or three years
- Can be a good option if your workloads have predictable resource needs



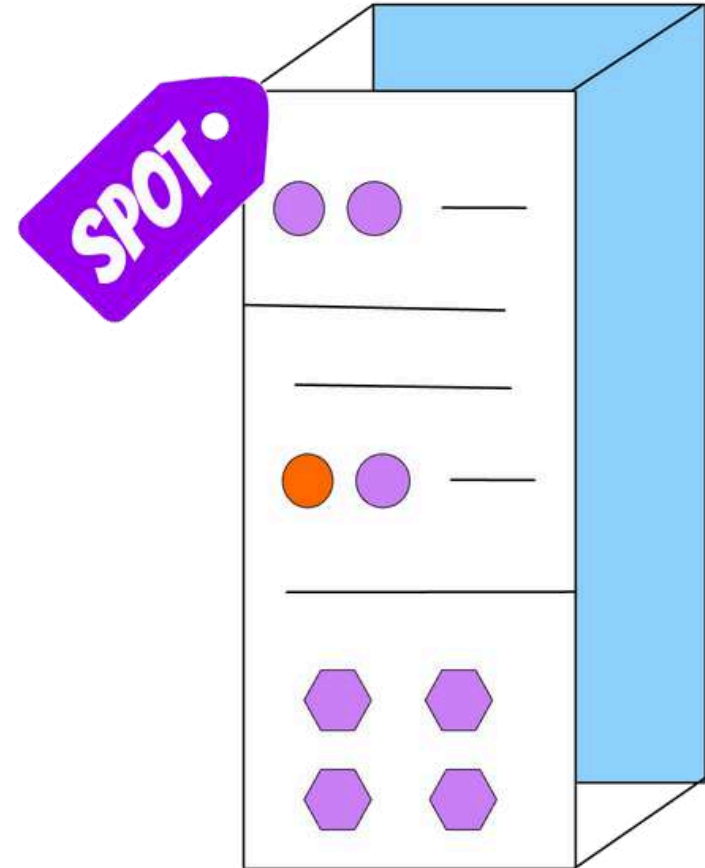


Spot VMs

Scenario: How can you run non-urgent workloads at the lowest possible cost?

Spot VMs: Use spare Google Cloud capacity at deep discounts (sometimes more than 80%)

- **Dynamic Pricing:** Changes based on available capacity
- **30-Second Warning:** You get a short warning to save work before shutdown
- **Use When:** Your apps are fault tolerant, cost-sensitive, and workload is not urgent (e.g., batch jobs)
- **Restrictions:** Not always available, no SLA, can't migrate to regular VMs, free tier credits do not apply





Spot VMs - Step by Step

Step 1: Create a Request – Launch a VM with Provisioning model - Spot

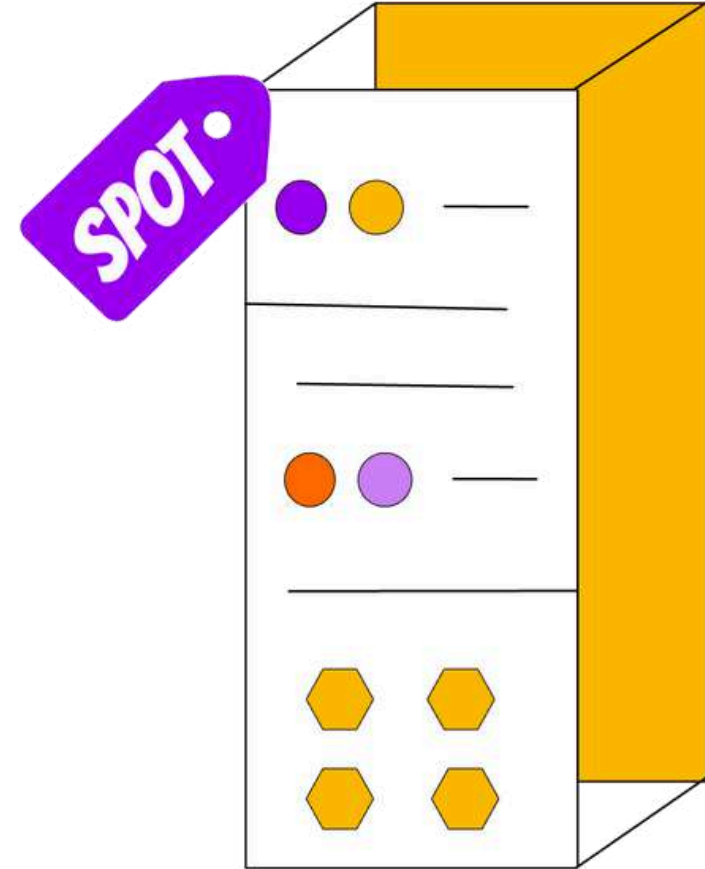
Step 2: Allocation – VM is provisioned if spare capacity is available

Step 3: Signal – Google Cloud sends a termination signal

- (or you can choose to stop the VM yourself)

Step 4: Termination – VM is stopped and resources are released

Note: You get a **30-second warning** before termination to save your work





Licensing Costs for OS

License: Usage agreement between you and a software vendor

- **Free licenses:** No charge for usage
- **On-demand/Pay-as-you-go (PAYG):** Google Cloud charges for usage. Cost varies depending on the OS
- **Bring your own license (BYOL)/Bring your own subscription (BYOS):**
 - You have a previous agreement with OS vendor
 - You can reuse it in Google Cloud (Remember: There are a lot of constraints)
 - **Recommendation:** Use your existing license while it is valid
 - After that shift to Pay-as-you-go model (PAYG)

Boot disk

Select an image or snapshot to create a boot disk; or attach an existing disk. Can't find what you're looking for? Explore hundreds of VM solutions in [Marketplace](#)

[Public images](#)

[Custom images](#)

[Snapshots](#)

[Archive Snapshots](#)

Operating system

SUSE Linux Enterprise Server

Version *

SUSE Linux Enterprise Server 12 SP5 x86_64

x86_64, built on 20260506

License type ⓘ *

Select the type of license to use with this OS image.



Pay-as-you-go (PAYG)

Google Cloud charges a license fee for usage of this image. [Pricing](#) varies based on the operating system.



Bring your own subscription or license (BYOS or BYOL)

Google Cloud doesn't charge you a license fee to use this OS image. With this option, you use the license subscription from your OS vendor.



Compute Engine Features - Custom Machine Types

Scenario: Predefined VM options are NOT appropriate for your workload

- **Custom Machine Type:** A machine type customized to your needs

Adjust: vCPUs, memory & GPUs

- **Limited Machine Families:** Among G, N and E machine series
- **Wide OS Support:** Use popular operating systems like Ubuntu, Debian, Red Hat, Windows, and more
- **Key Cost Factors:** Number of vCPUs and amount of memory provisioned

General purpose Compute optimized Memory optimized Storage optimized **GPUs**

Graphics processing units (GPUs) accelerate specific workloads on your instances such as machine learning and data processing. [Learn More](#)

GPU type: NVIDIA T4 Number of GPUs: 1

☐ Enable Virtual Workstation (NVIDIA GRID)

Series ?	Description	vCPUs ?	Memory ?	CPU Platform
☒ N1	Balanced price & performance	1 - 96	1.8 - 624 GB	Intel Haswell

Machine type

Choose a machine type with preset amounts of vCPUs and memory that suit most workloads. Or, you can create a custom machine for your workload's particular needs. [Learn more](#)

Preset Custom

Creating a custom machine incurs additional costs

Cores: 1 96 1 vCPU

Memory: 1 6.5 1 GB



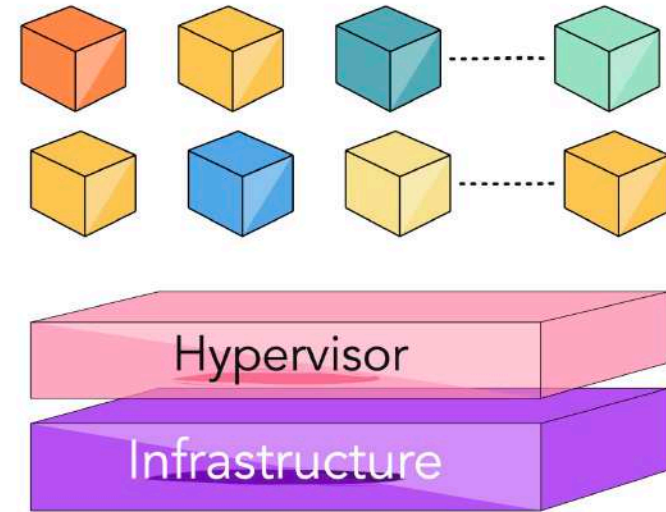
Need For Sole-Tenant Nodes

Do You Know?: Regular VMs share physical hosts with other customers

Sensitive Workloads: Some workloads require host level isolation for security, compliance, or licensing

- **Licensing Needs:** Some software (for example, Windows Server or SQL Server) is licensed based on the number of physical cores or processors on a host
- **Organizational Policies:** Internal policies may require dedicated hardware

Solution: Use **Sole-Tenant Nodes** to run VMs





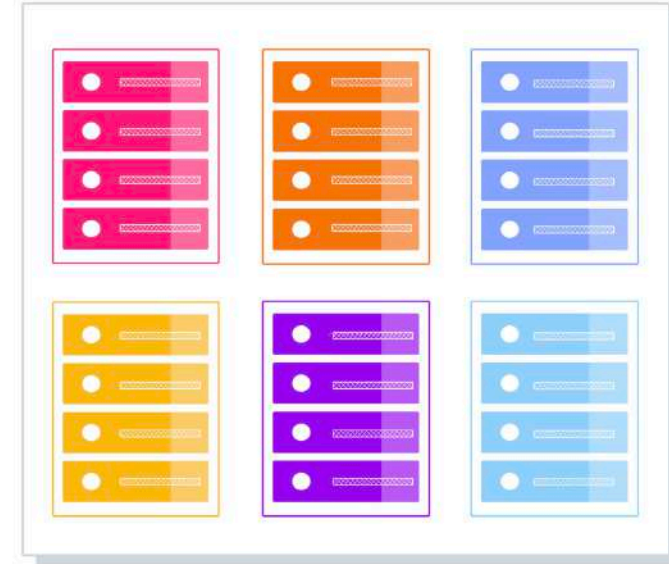
What Are Sole-tenant Nodes?

Dedicated Physical Servers: The entire physical host is dedicated to one customer

Fully Integrated with Compute Engine: Use the same machine types, images and other features

How?:

- **1: Create a Node Group:** Create one or more nodes with affinity labels (`group:a`, `group:b`, `group:c`)
- **2: Create VMs:** Use affinity rules (`group:IN:a`, `group:IN:a,b`, `group:NOT_IN:a`) to control the placement of VMs on nodes





When Should You Use Sole-tenant Nodes?

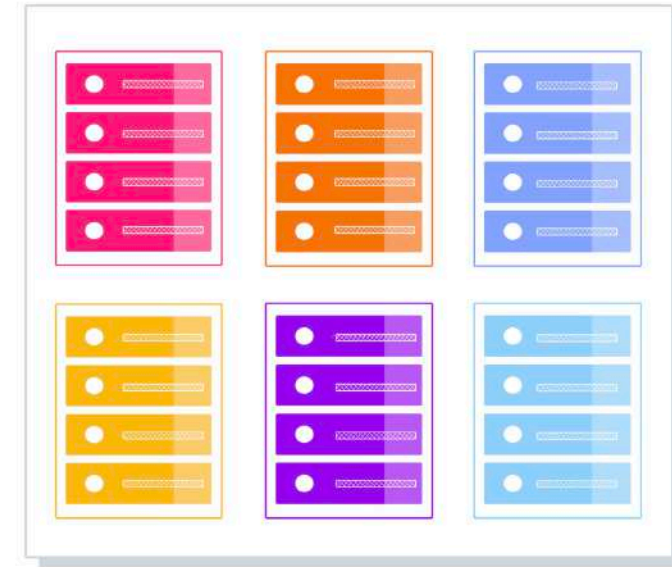
Strict Isolation Requirements: Compliance frameworks like PCI-DSS, HIPAA, FedRAMP

BYOL Licensing: Use an existing per-core or per-processor license

- *Examples:* Windows Server, SQL Server

Need Flexibility: With VM placement

- **1:** Place related workloads on the same host (All VMs on one Node Group)
- **2:** Spread workloads across hosts for High Availability (Some VMs on one node group and others on another node group)





Compute Engine - Quick Review

Topic	Details
Machine Types	Optimized combination of compute (vCPUs) and memory for your workloads. Create Custom Machine Types when needed.
Image	What OS and software to use? Reduce boot time and improve security by creating custom hardened images. Images can be shared with other projects.
Instance Templates	Pre-configured templates simplifying the creation of VM instances
Sustained Use Discounts	Automatic discounts for running VM instances for significant portion of the billing month
Committed use discounts	1 year or 3 year reservations: For workloads with predictable resource needs
Spot VMs	Short-lived cheaper compute instances for non-time-critical fault-tolerant workloads



Virtual Machine - Best Practices

Best Practice	Description
Choose Zone and Region Carefully	Consider cost, legal regulations, availability requirements, latency, and machine type needs
Distribute VM Instances	Spread your VMs across multiple zones and regions to ensure high availability and fault tolerance
Pick the Right Machine Type	Experiment with different machine types to find what fits your workload best
Consider GPUs	For math-heavy or graphics-intensive applications like AI/ML to boost performance
Consider Committed Use Discounts	For steady long running workloads
Consider Spot VMs	For fault-tolerant, non-urgent, cost-sensitive workloads



Compute Engine Scenarios

Scenario	Solution
What are the pre-requisites to be able to create a VM instance?	1. Project, 2. Billing Account, 3. Compute Engine APIs should be enabled
You want to login to your VM instance to install software	You can SSH into it
You want to allow HTTP or HTTPS traffic to your VM	Configure Firewall Rules
You have predictable resource needs that will remain constant for a few years. How can you reduce your VM costs?	Committed Use Discounts
You want to create an image with your preferred OS, a few pre-installed apps, custom configuration, and security settings	Use a Custom Image
You want to launch multiple VMs with the same configuration - machine type, image, startup script, etc.	Create an Instance Template



in28minutes

Getting Started with Instance Groups



Instance Groups

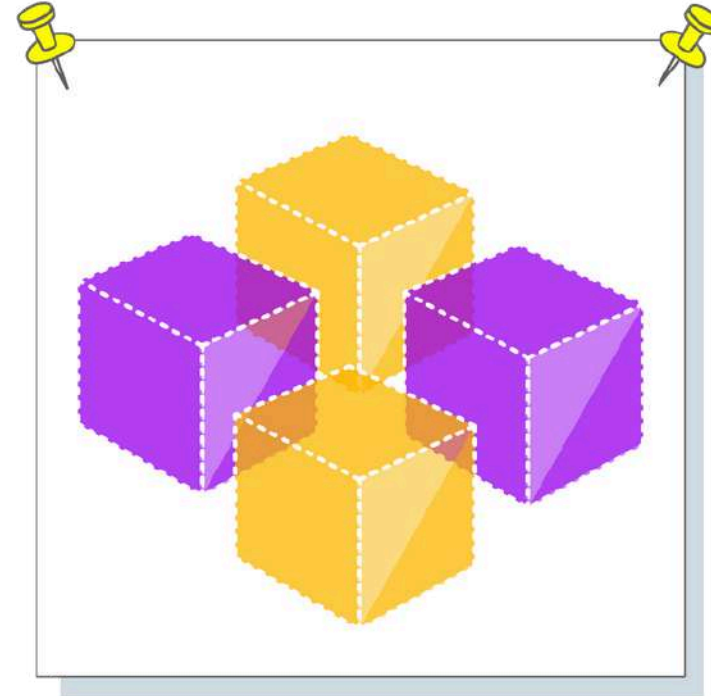
Scenario: What if you need to manage multiple VMs belonging to same app?

- **Instance groups:** Collection of VM instances that can be managed as a single entity

Two Kinds:

- **Managed** – Identical VMs created from an instance template (with auto-scaling, auto-healing, etc.)
- **Unmanaged** – VMs with different configurations

Zonal or Regional: **Zonal** (all VMs in one zone) or **Regional** (spread across zones in one region for higher availability)



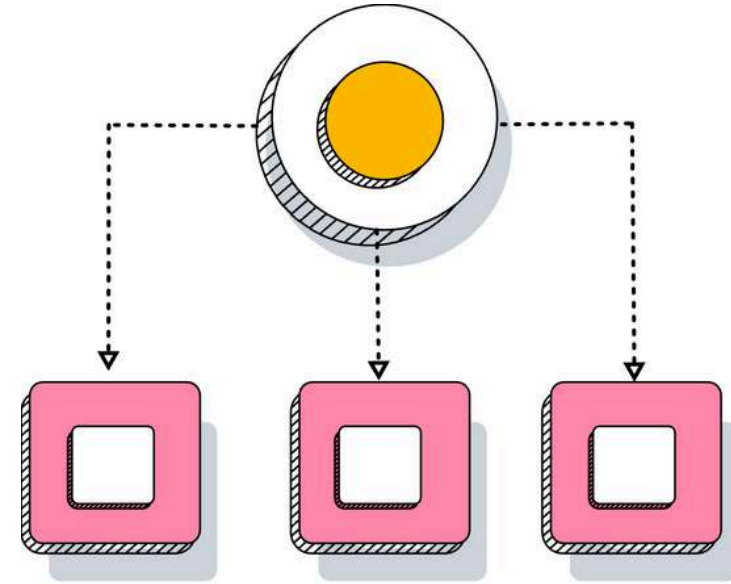


What is a Managed Instance Group (MIG)? [1/2]

Managed Instance Group (MIG): Group of VMs created using an instance template

Key Features:

- **Maintain Instance Count:** Keep a constant number of VMs running all the time (If a VM fails, MIG replaces it)
- **Self Healing:** Uses health checks to spot and fix app failures
- **Auto Scaling:** Automatically add or remove VMs depending on the load
- **Load Balancing:** Distribute traffic across healthy VM instances in MIG



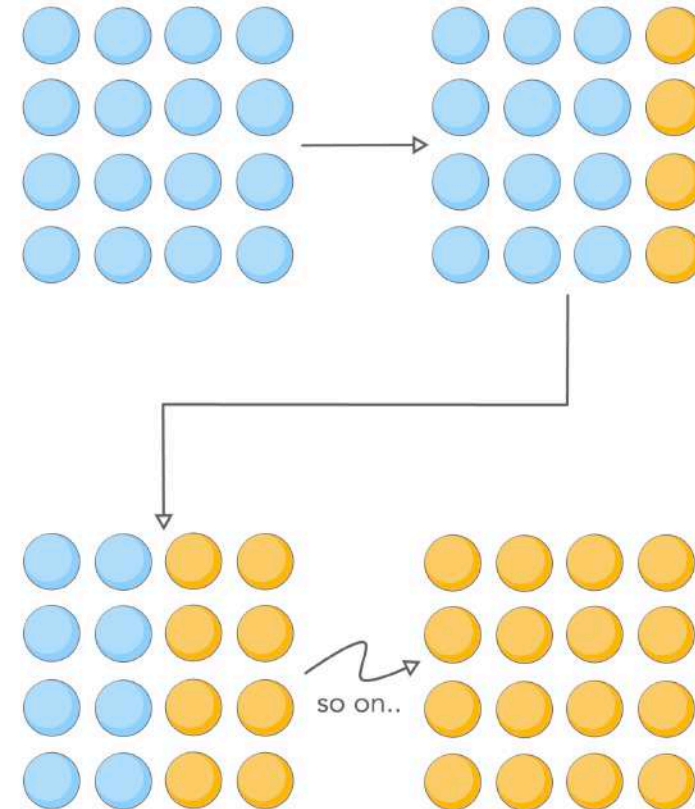


What is a Managed Instance Group (MIG)? [2/2]

Regional MIGs: Create instances in multiple zones in a single region (higher availability compared to zonal MIGs)

Release New Versions: Without downtime

- **Rolling updates:** Release new version step by step (gradually). Update a percentage of instances to the new version at a time. [Multiple Intermediate Steps]
- **Canary Deployment:** Test new version with a group of instances before releasing it across all instances. [Typically One Intermediate Step]





Instance Group Scenarios

Scenario	Solution
You want to create and manage a group of identical VMs in an easy way	Use a Managed Instance Group with an instance template
You need simple grouping of already existing VMs without autoscaling	Use an Unmanaged Instance Group
Your application must remain available even if an entire zone goes down. You want to create an Instance Group. Which type (regional or zonal) would you recommend?	Create a regional MIG (or multi zone MIG) to distribute instances across zones
You want failed or unhealthy VMs to be automatically detected and replaced	Configure health checks in MIG to enable autohealing



in28minutes

Getting Started with Load Balancing

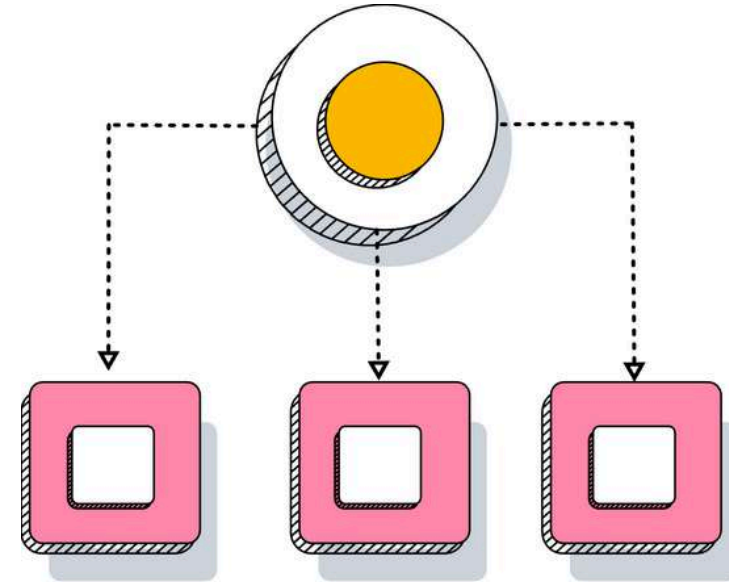


Cloud Load Balancing

Scenario: What if your application receives millions of user requests at the same time?

Cloud Load Balancing: Distribute traffic across multiple VMs

- Highly available (99.99% SLA)
- Automatically adjusts capacity with **auto scaling**
- **Global load balancing** with a single Anycast IP
- Supports **internal load balancing** for private traffic
- Routes traffic to healthy instances using **health checks**





Cloud Load Balancing - Terminology

Backend - Group of endpoints that receive traffic from LB

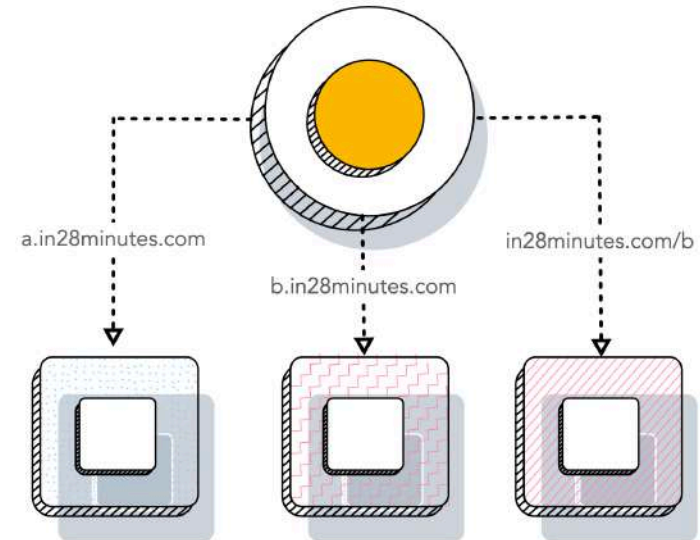
- **Examples:** MIG, Cloud Run instances, Kubernetes Pods, External backends

Frontend - Specify an IP addr, port & protocol

- **IP address:** Frontend IP for your clients requests
- **For SSL:** A certificate must also be assigned

For HTTP(S) LB : Define Routing rules using

- **path** - in28minutes.com/a vs in28minutes.com/b
- **Host** - a.in28minutes.com vs b.in28minutes.com
- **HTTP headers** and methods (POST, GET, etc), ..





in28minutes

Getting Started with

Managed Compute Services

Getting Started with Managed Services



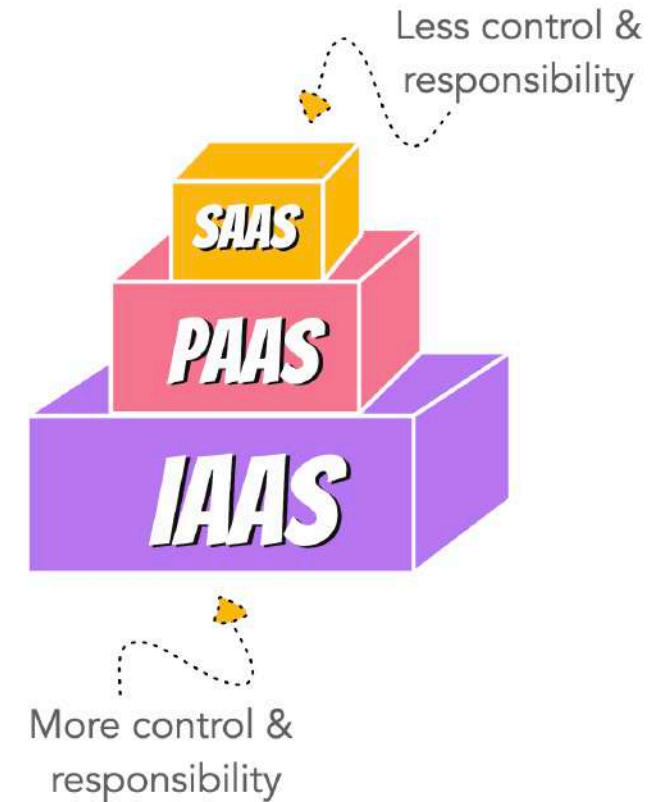
Question: Do you want to run apps in the cloud like you do in a data center?

- **Alternative:** Or are there better cloud-native approaches?

You should **understand some terminology:**

- **IaaS** (Infrastructure as a Service)
- **PaaS** (Platform as a Service)
- **SaaS** (Software as a Service)

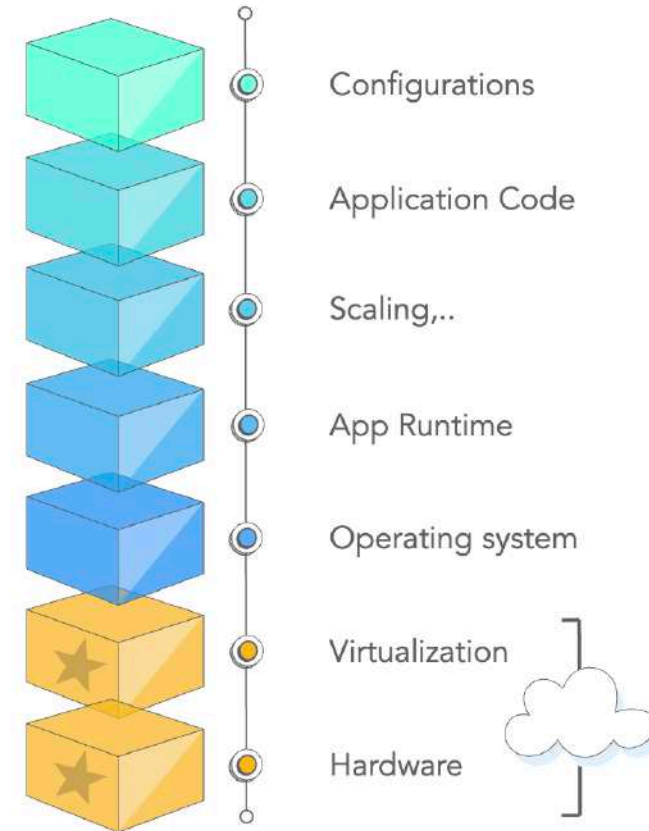
Ready for a journey to understand these?





Example 1A: Setting up an App - IaaS Approach

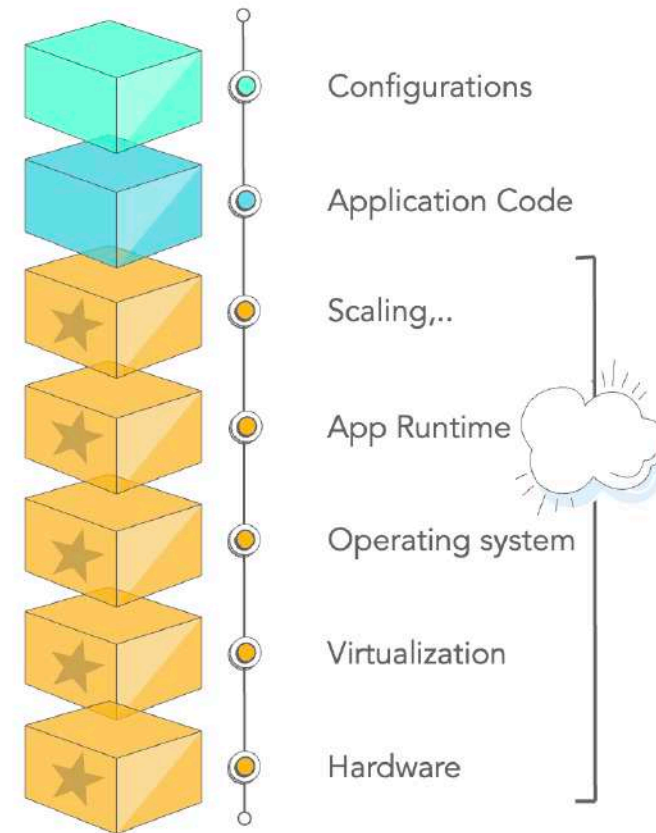
- 1: Provision VMs – Create VMs on cloud
- 2: Install Software – Install OS, runtimes etc
- 3: Deploy Code – Upload code and configure your application manually
- 4: Configure Ops – Set up load balancing and auto scaling yourself
- 5: Maintain – You are responsible for OS patching, runtime upgrades and server health





Example 1B: Setting up an App - PaaS Approach

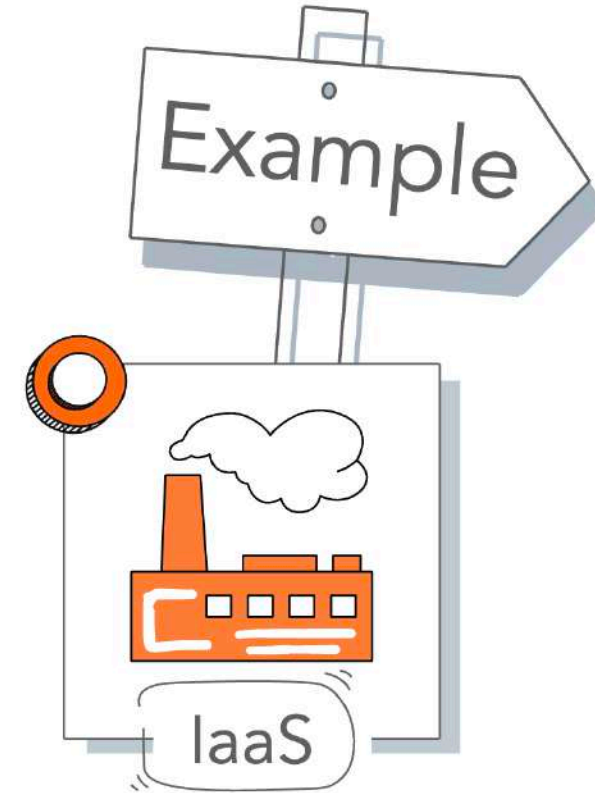
- 1: Pick a Platform** – AWS Elastic Beanstalk, Azure App Service, or Google App Engine
- 2: Upload Code** – Upload your code artifact
- 3: Automatic Setup** – Platform provisions VMs, installs OS, and starts app
- 4: Built-in Ops** – Scaling and Load Balancing are easily configurable
- 5: Zero Maintenance** – No Manual OS patching or server management required





Example 2A: Setting up a SQL Database - IaaS Approach

- 1: Provision VMs** – Create VMs to host your database
- 2: Install Database** – Set up MySQL, PostgreSQL, or any DB you need
- 3: Configure Storage** – Attach disks
- 4: Set Up Ops** – Manually configure backup jobs and replication
- 5: Maintenance** – Manage scaling, patches, and monitor performance

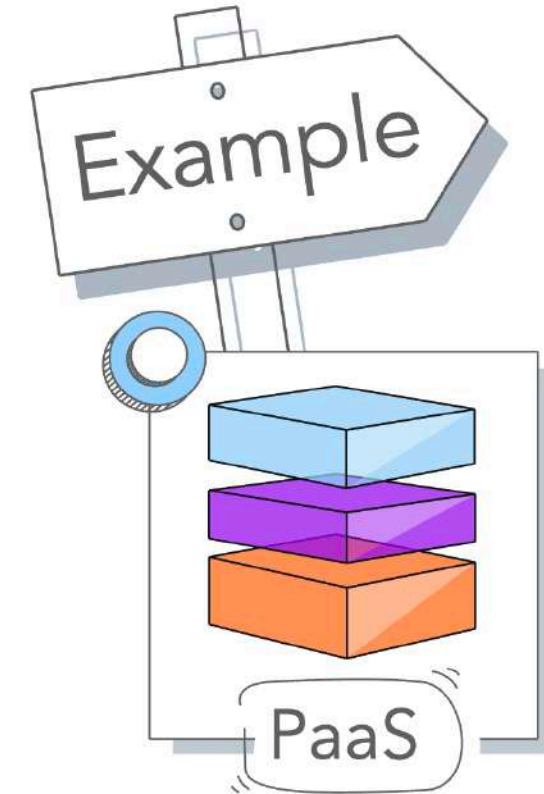




Example 2B: Setting up a SQL Database - PaaS Approach

- 1: **Pick a Service** – AWS RDS, Azure SQL Database, or Google Cloud SQL
- 2: **Choose Type** – MySQL, PostgreSQL, SQL Server, etc
- 3: **Configure** – Decide CPU, memory, and storage size
- 4: **Enable Ops** – Turn on built-in backups & high availability
 - **Ready** – Database is ready to use

Focus: Data and queries, NOT servers or maintenance





What is IaaS?

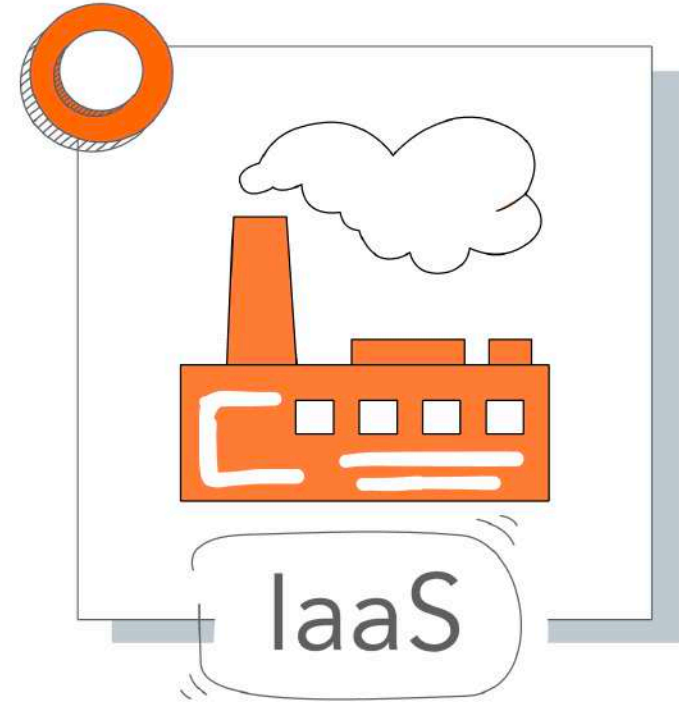
IaaS: Infrastructure as a Service

- **Concept:** Use only infrastructure from the cloud provider

Customer Responsibility: High

- **Components:** OS, runtime, and system software
- **Operations:** Availability, durability, and scaling

Summary: You manage almost everything except the hardware





What is PaaS?

PaaS (Platform as a Service): Use a complete platform provided by the cloud

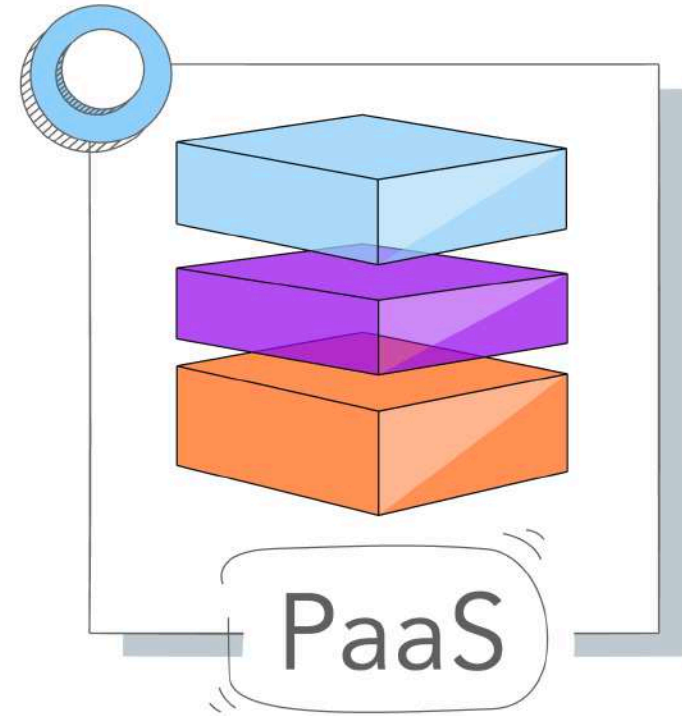
Cloud Provider Responsibility: High

- **Components:** OS, runtime, availability, durability, scaling

Customer Responsibility: Low

- **Focus:** Code, data, and configuration

Summary: Cloud provider manages the routine work





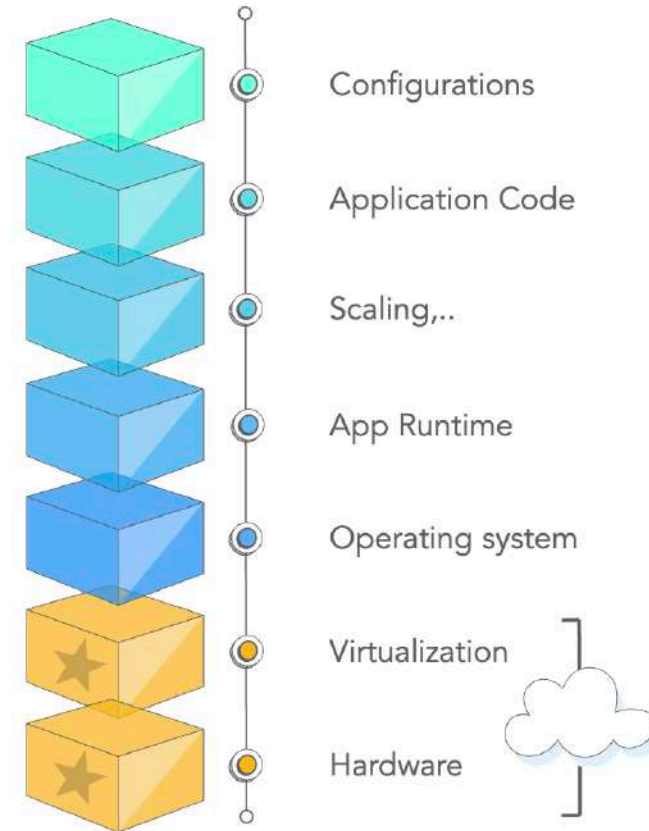
Who is Responsible for What? - IaaS

Cloud platform: The Basics

- Virtualization, Hardware and Networking

Customers of cloud: Everything Else

- **Application:** Code and Runtime
- **Operations:** Load balancing, Auto scaling, Availability
- **Database:** Software, upgrades, configuration (tables, views,..), and data
- **OS:** Upgrades and patches





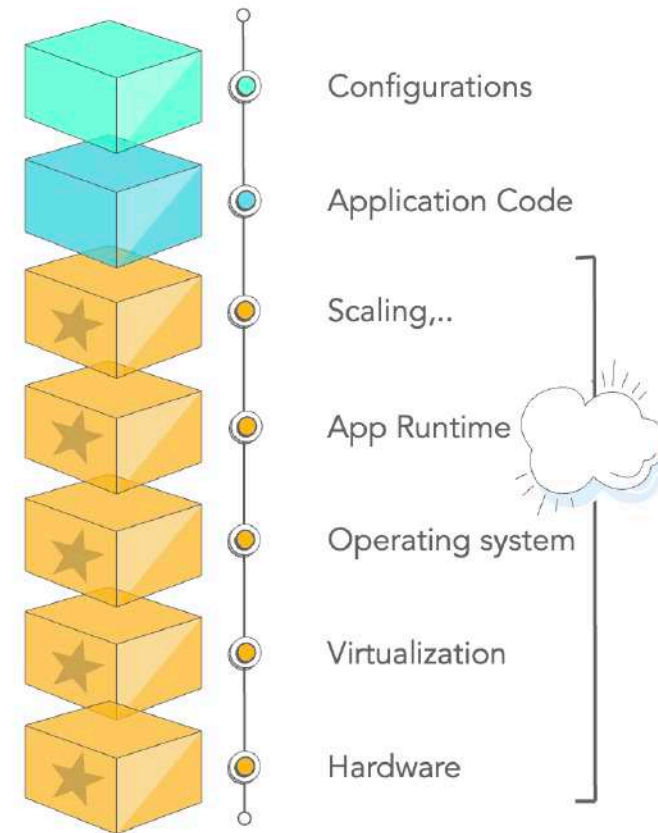
Who is Responsible for What? - PaaS

Cloud Platform: Heavy Lifting

- **Infrastructure:** Virtualization, Hardware, Networking, *OS: Upgrades and patches
- **Platform:** Runtime, Database software, Scaling, Availability, Durability, Load Balancing

Customer: Business Logic

- **Application:** Code & configuration
- **Database:** Schema, Tables, and Data
- **Restricted Access:** Usually no access to OS or VM instances





PaaS Services Can Do a Lot More

Scope: PaaS is NOT limited to running apps and databases

Managed Services: Includes common building blocks

- **Storage:** Amazon S3, Azure Blob Storage, Google Cloud Storage
- **Analytics:** BigQuery, Azure Synapse, AWS Redshift

Goal: Let developers focus on logic and innovation

- **Benefit:** Skip the setup and maintenance





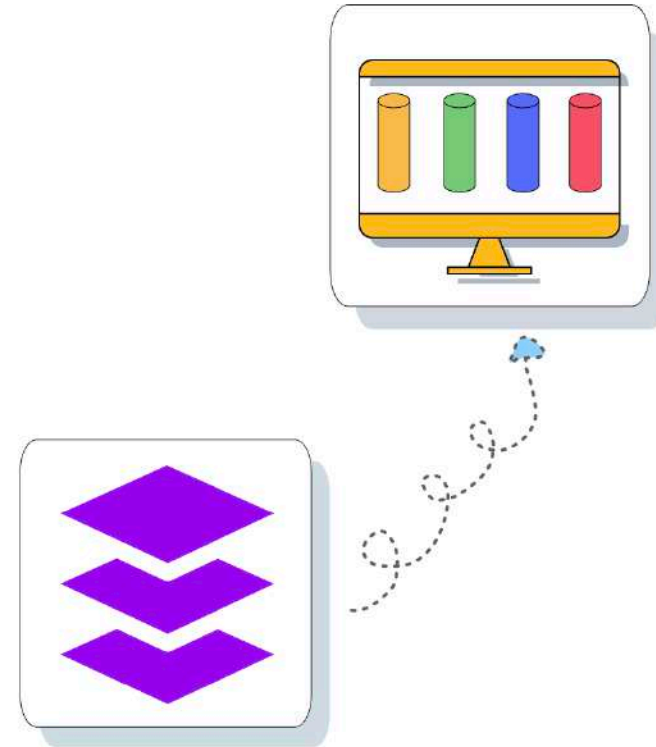
From Platforms to Complete Applications

IaaS: You manage most things yourself (infra, scaling, app, data)

PaaS: The platform helps you focus on your application and data

SaaS: You use complete software - no setup needed!

Next Step: Let's explore SaaS (Software as a Service)





What is SaaS (Software as a Service)?

Definition: Centrally hosted software

- **Hosting:** Mostly on the cloud

Billing: Offered on a subscription basis (Pay as you go)

Examples:

- **Email:** Gmail, Outlook 365
- **Office tools:** Microsoft Office 365, Google Docs
- **Calendar:** Google Calendar





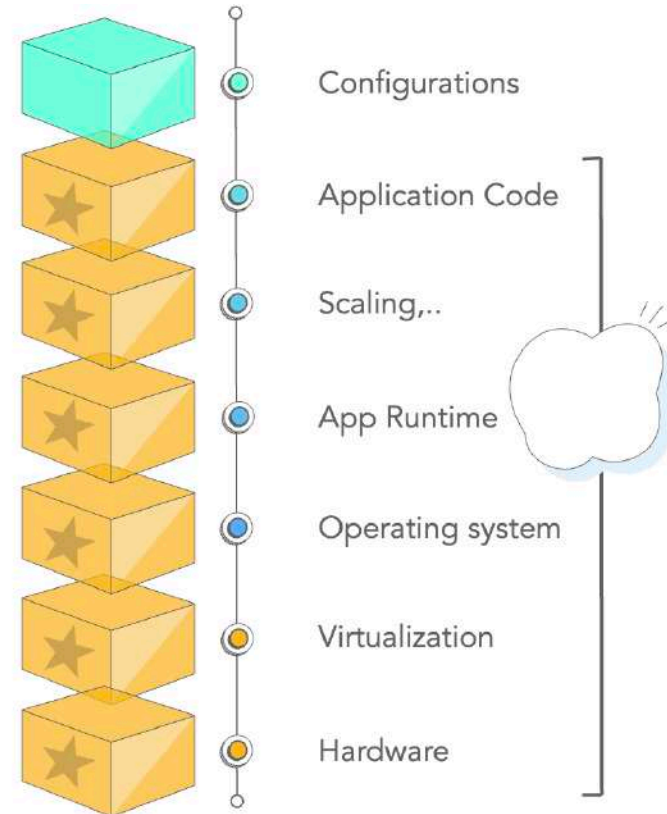
Who is Responsible for What? SAAS

Service Provider: Almost Everything

- **Infrastructure:** OS, Runtime, Networking
- **Application:** Code updates and maintenance
- **Operations:** Auto scaling, Availability, Load balancing

Customer: Usage

- **Content:** Documents, emails, sheets
- **Configuration:** Users, permissions etc





Quick Recap - IaaS vs PaaS vs SaaS

IaaS: Platform provides infra

- **You:** Manage all other layers

PaaS: Platform handles infra, setup, and scaling

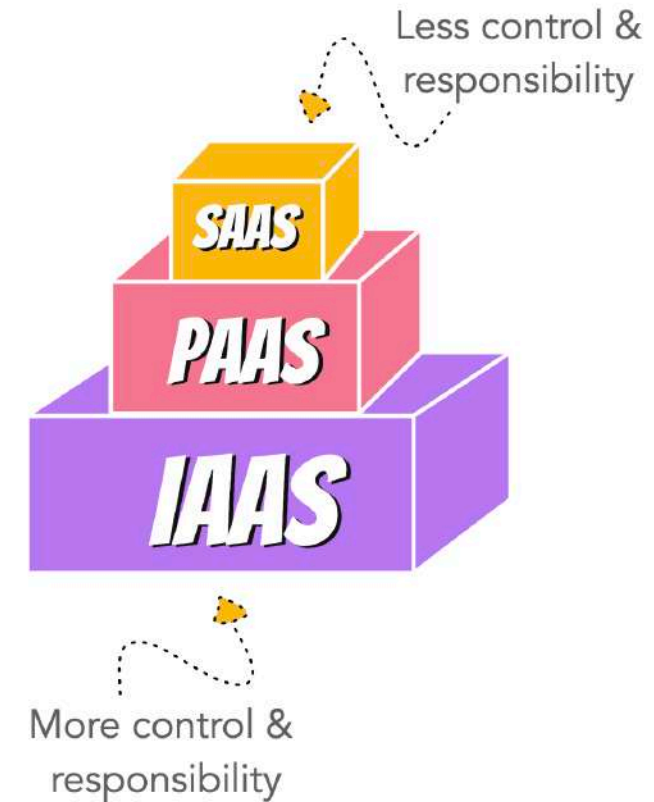
- **You:** Handle code, configuration, and data

SaaS: Platform provides everything

- **You:** Just use the app

Trend: Moving from IaaS → PaaS → SaaS

- **Control:** Decreases
- **Ease of Use:** Increases





Service Categories - Scenarios

Scenario	Solution
IaaS or PaaS or SaaS: Deploy Custom Application in Virtual Machines	IaaS
IaaS or PaaS or SaaS: Deploy a Database in Virtual Machines	IaaS
IaaS or PaaS or SaaS: Using Gmail	SaaS
IaaS or PaaS or SaaS: Using a Managed Service to setup a database	PaaS
True or False: Customer is responsible for OS updates when using PaaS	False
True or False: Customer is completely responsible for Availability when using PaaS	False
True or False: In PaaS, customer has access to VM instances	False
True or False: In PaaS, customer can customize OS and install custom software	False
True or False: PaaS services only offer Compute services	False
True or False: In PaaS, customer can configure hardware needs (memory, cpu etc)	True



Choosing Between IaaS, PaaS, and SaaS - Scenarios

Scenario	Solution
Legacy Migration: You must migrate a 10-year-old app that requires a specific, older version of the OS with customization	IaaS
Rapid Development: A startup wants to deploy a new NodeJS web app immediately. They have no Ops team and don't want to manage servers	PaaS
Productivity: You need email, calendar, and document collaboration tools for 500 employees starting tomorrow	SaaS
Custom Database: You need to run a specialized database engine that needs customization of Operating System	IaaS
CRM Needs: Your sales team needs a tool to track leads and customers. You want zero maintenance overhead	SaaS



in28minutes

Google Cloud

Managed Compute Services



Cloud Managed Services for Compute - Models

Model	Explanation
IaaS (Infrastructure as a Service)	Use only infrastructure from the cloud provider Customer manages almost everything (OS, runtime, availability, durability, scaling) except the infrastructure
PaaS (Platform as a Service)	Use a complete platform provided by the cloud Delegate a lot of responsibility (OS, runtime, availability, durability, scaling) to the cloud provider
SaaS (Software as a Service)	Centrally hosted software offered on a subscription basis Examples: Gmail, Outlook 365, Google Docs, Google Calendar Customer has minimal responsibility



Cloud Managed Services for Compute - Key Concepts

Concept	Explanation
Serverless	• When you build an app, you think about: "Where to deploy? What server? How to scale and keep it available?" What if you did not have to worry about any of these? • Concept: You focus on code - the cloud platform handles everything else • Zero Infrastructure Worries: Automatic Scaling + Pay Only for Usage (Ideally Zero requests → Zero cost)
Containers	• Package entire app and dependencies as one unit: code + runtime + dependencies • Run the same package, the same way everywhere (Laptop, test, and cloud) • Language or application type does NOT matter
Container Orchestration	• Microservices: Need a lot of features and automation • Container Orchestration platforms like Kubernetes: Enable running 100s of microservices (each with multiple instances) with minimum management headache • Key Features: Auto scaling, Load Balancing, Release Management, Automatically recover from failures, etc.



in28minutes

Getting Started with Serverless



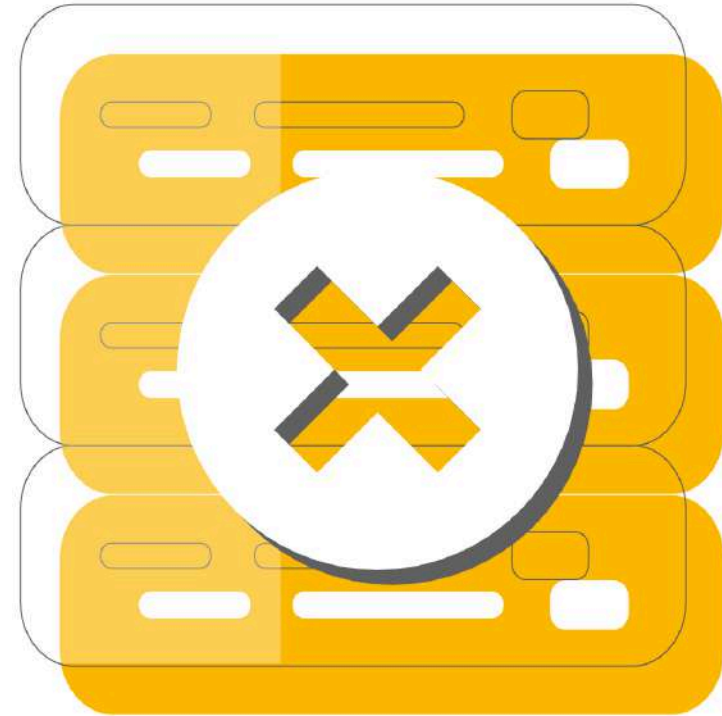
What if You Didn't Have to Manage Servers?

Scenario: When you build an app, you think about:

- Where to deploy? What kind of server?
- How to scale and keep it available?

Serverless: What if you did not have to worry about any of that?

- **Concept:** You focus on code - the cloud handles everything else
- **Remember:** Serverless $\neq$ No Servers
- **Servers still exist:** You just don't manage them





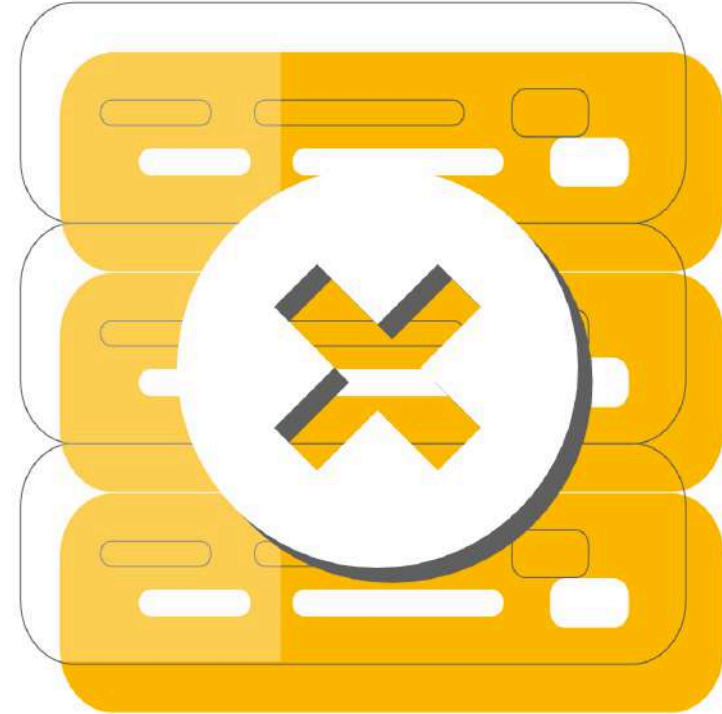
Serverless Explained – The Uber Analogy

Think of Uber:

- You don't own or maintain a car
- You just request a ride when you need it
- Uber handles cars, drivers, and maintenance
- Cars, drivers and maintenance exist, BUT you don't worry about them!

Serverless works the same way

- You focus **ONLY** on your app logic
- The cloud handles infrastructure, scaling, and uptime
- You **pay only for what you use**





What Does Serverless Mean for You?

Zero Infrastructure Worries

- You have **no visibility** into servers, VMs, or scaling

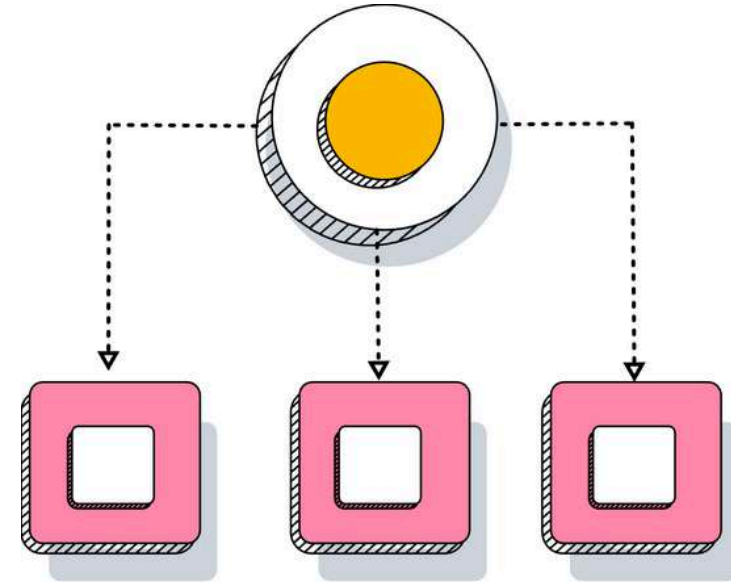
Automatic Scaling & High Availability

- The cloud handles it for you

Pay Only for Usage

- Zero requests → Zero cost
- You pay for requests, **not for servers**

Focus on Code – the rest is managed automatically





Misconception: Serverless ≠ Just Compute

Serverless is not limited to compute services

- It includes databases, APIs, messaging, and more

Examples:

- **Databases** – Amazon DynamoDB, Google Cloud Firestore, Azure Cosmos DB (Pay per operation)
- **Messaging** – Google Cloud Pub/Sub, AWS EventBridge, Azure Service Bus (Pay per message)
- And a lot of other types of managed services





Serverless - My Perspective

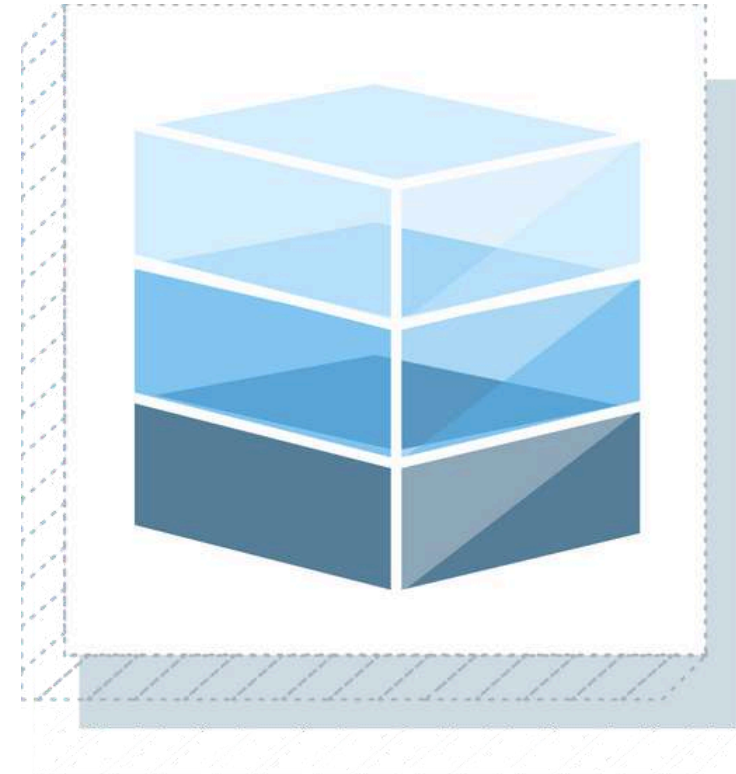
Level 1 (Managed): Handles scaling and availability BUT you pay for instances

- Can scale down to zero instances
- **Examples:** Google App Engine, AWS Fargate

Level 2 (True Serverless): Pay per invocation, not per instance

- **Examples:** Google Cloud Run Functions, AWS Lambda, Azure Functions

When I say “Serverless,” I mean Level 2: Zero infra management + Pay per request





in28minutes

Getting Started with Containers



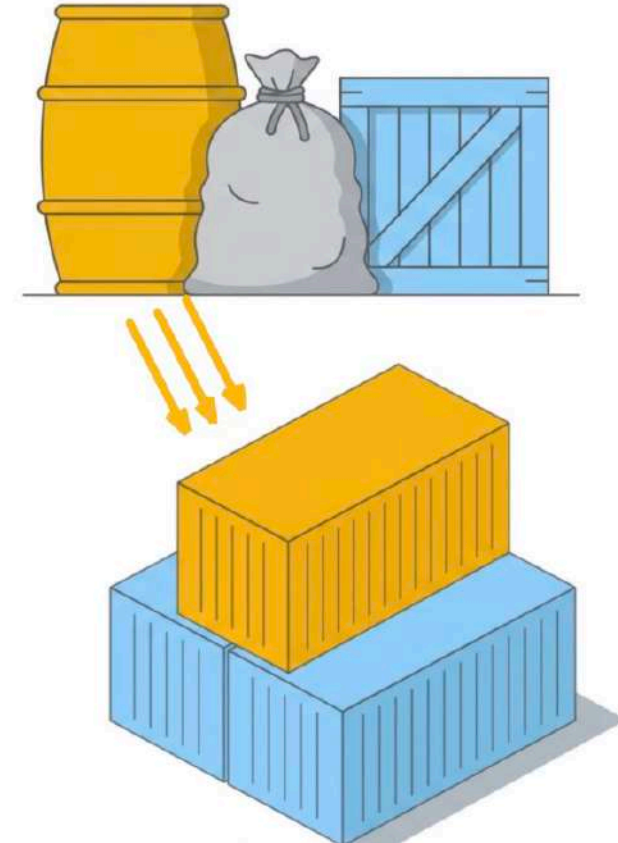
Evolution of Shipping Containers

Before: Goods shipped as loose items in different shapes and sizes

- **No Standard:** Each shipment needed custom handling
- **Unsafe:** Goods damaged or lost during transfers

Breakthrough: A single **standard box** called a shipping container

- **Universal:** Same box works on ships, trucks etc.
- **Fast:** Easy to load, stack, and move
- **Cheap:** Lower handling cost





Containers Bring Same Revolution for Software

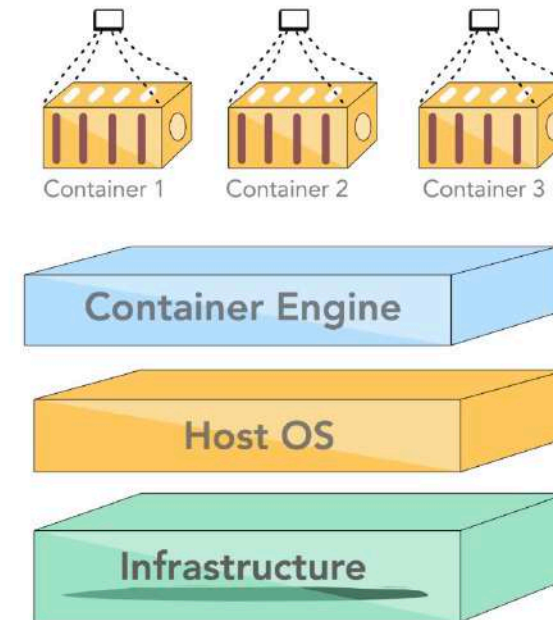
Before: Every application had a different deployment process

Solution: Package entire application and dependencies as one unit

- Code, runtime, and dependencies in one unit

Result: Run the same package, the same way everywhere

- Laptop, test, and production (cloud) environments
- Language or application type does NOT matter



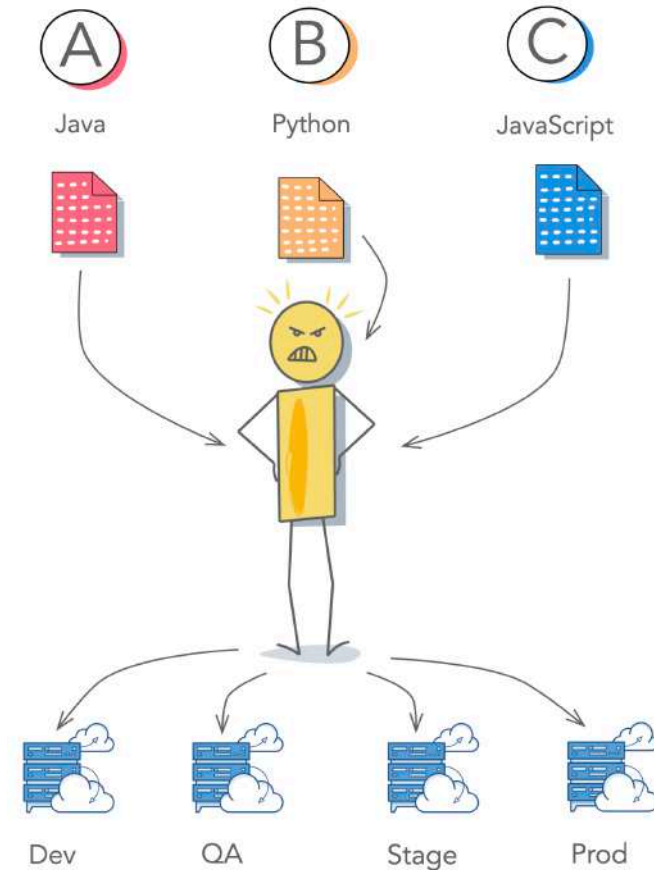


Traditional Deployment Before Containers

Documentation Driven: Deployment steps are described in a document

Manual Steps:

- **Hardware:** Setup Server
- **OS:** Install Linux/Windows/Mac/..
- **Runtime:** Install the right version of Java/Python/Node.js
- **Application:** Copy application code or binary files
- **Dependencies:** Setup dependencies





Challenges with Traditional Deployment

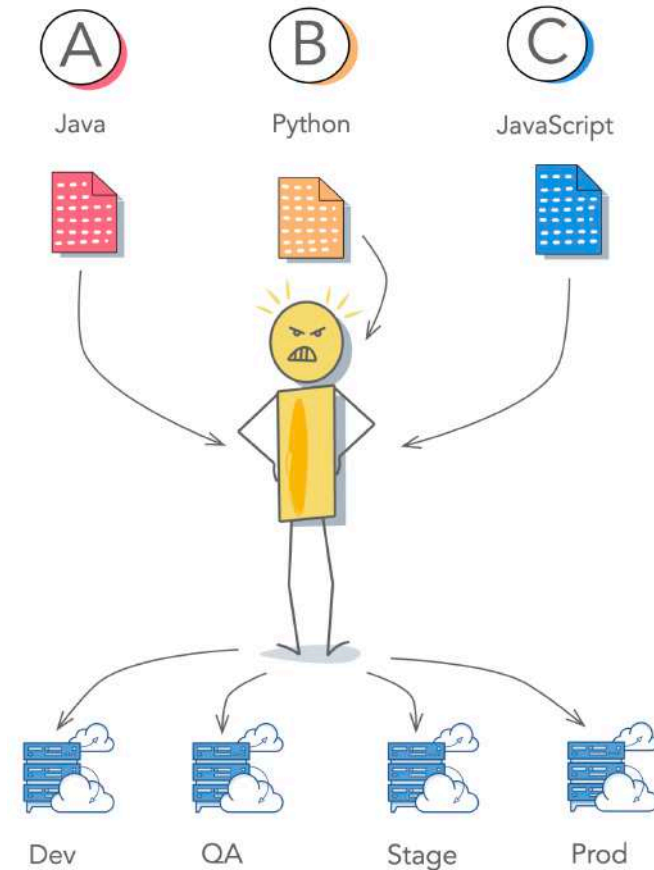
Manual Setup: Install OS > Setup software > Setup app > dependencies

Slow: Takes hours per environment

Error-Prone: Missed steps, mismatched versions

Different Docs: For different applications!

Hard to Debug: Works on my machine problems





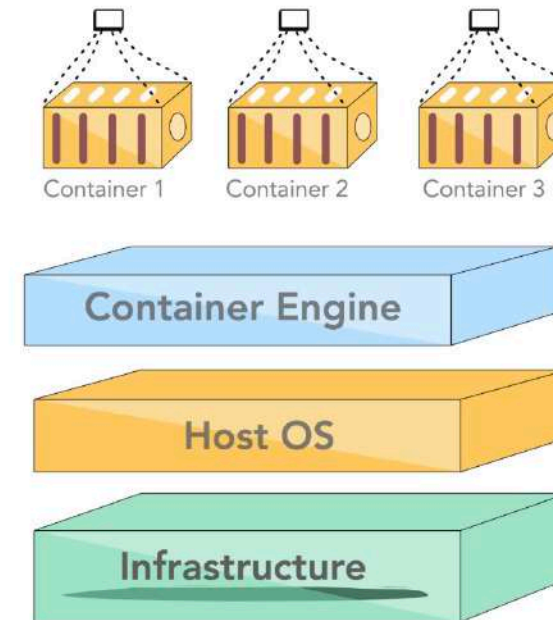
Enter Containers – A Better Way

Concept: Deliver a Complete Package

All-in-One:

- **OS:** Included
- **Runtime:** Included
- **Code:** Included
- **Dependencies:** Included

Simple: If the container runs on your laptop, it runs on the server





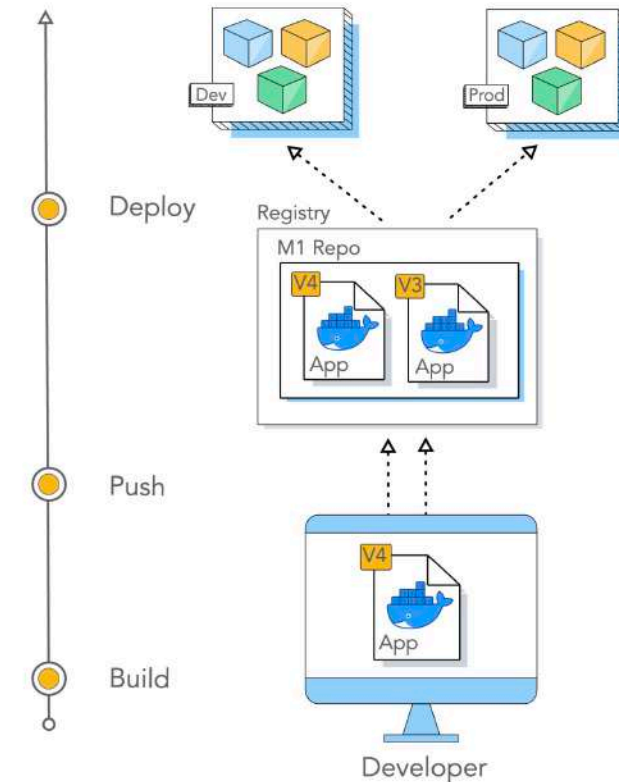
How do Containers work?

Idea: Package your application and its dependencies into a **container image**

Steps:

- **Build** – Create a container image with everything needed to run the app
- **Push** – Upload the image to a container registry
- **Deploy** – Download (Pull) the image and run it anywhere (local, data center, cloud)

Goal: Run the same container everywhere - no surprises!





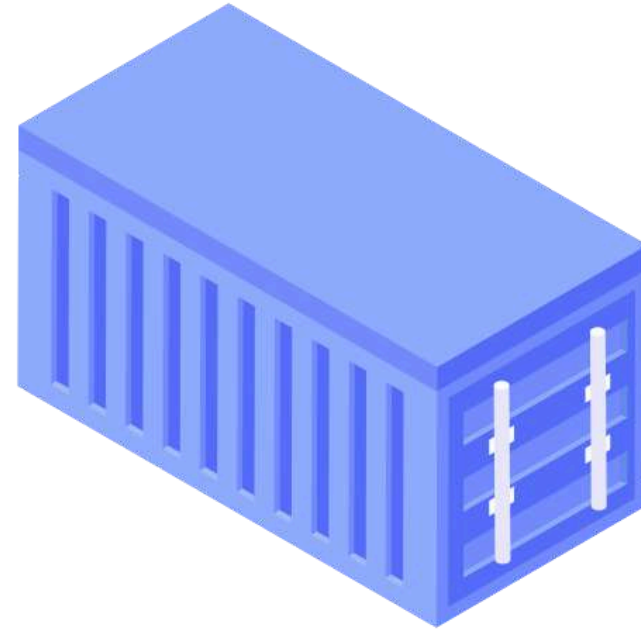
Component 1: Container Image

What is it?: A single file (essentially) containing:

- OS layer
- Your Runtime
- Your code
- Dependencies (Libraries)

Key Trait: Immutable (Read-only)

- Once built, it never changes
- Create new versions as needed - v1.0.0, v1.0.1, v1.1.0, ..(tags)





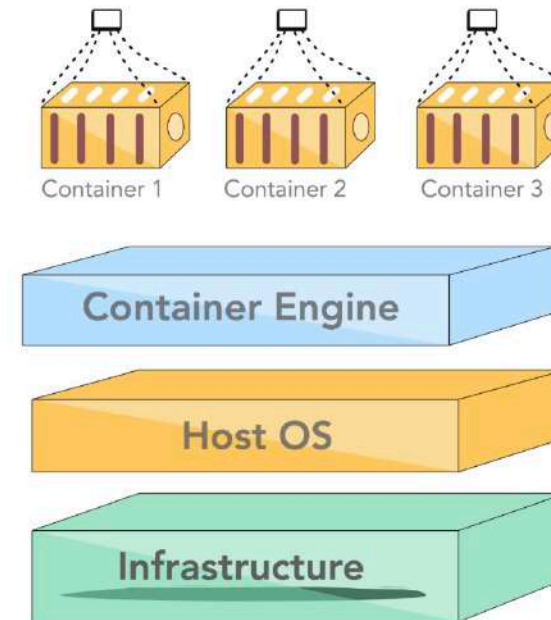
Component 2: Container Engine

Role: The software installed on your laptop or server that runs containers

Examples:

- **Docker:** The most famous one
- **Podman:** A popular alternative
- **Containerd:** The industry standard runtime

Job: "Take this image and run it as a container"





Component 3: Container Registry

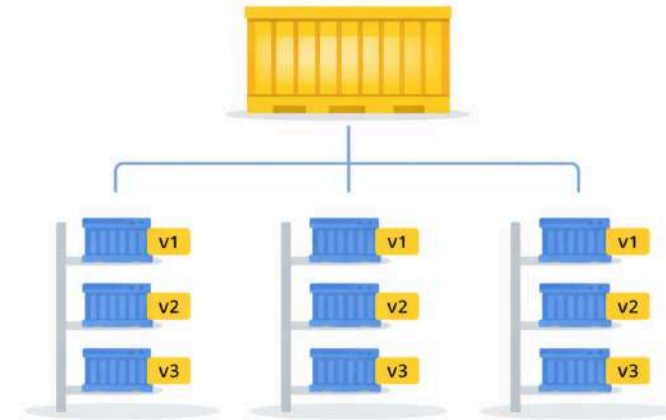
Role: A secure place to store your images

Public Registries:

- **Docker Hub:** The default public library

Private Cloud Registries:

- **AWS:** Elastic Container Registry
- **Azure:** Azure Container Registry
- **Google:** Google Artifact Registry





Containers – The Big Picture Recap

Step 1: Create a Container Image (Dev)

- Package code, runtime, and dependencies

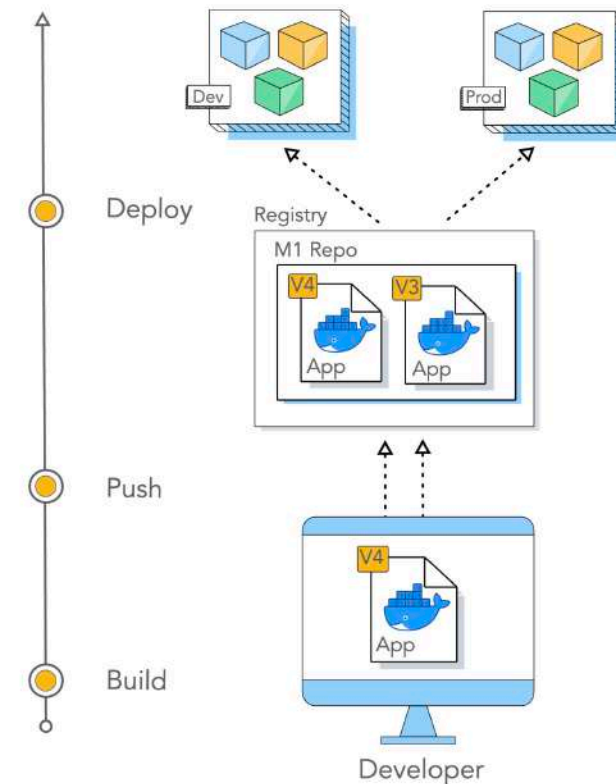
Step 2: Push it to Container Registry (Dev)

- Share the image across teams and environments

Step 3: Deploy to an Environment (Ops)

- Pull the image and run it as a container (using Container Engine)

Outcome: Same image, Same behavior,
Everywhere you run it





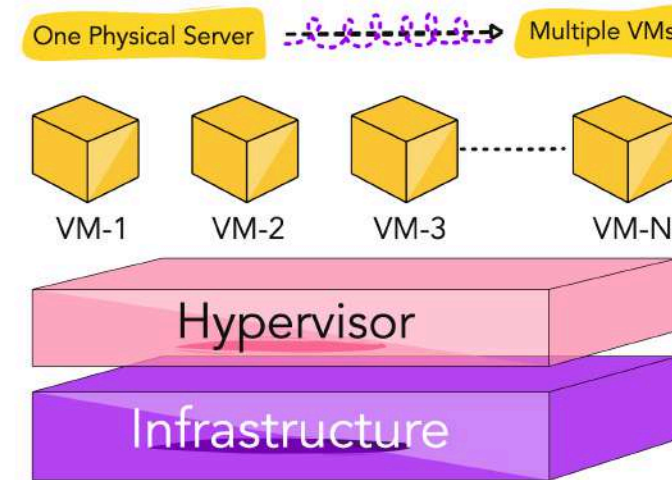
Virtual Machines - How do they work?

Key Layers: 3 key layers

- **Base Layer:** The physical server
- **Hypervisor:** Software that creates and manages VMs
- **VMs:** Each VM runs its own OS (Windows, Linux, ..)
 - Each VM contains application runtime, libraries, and the app

Key Benefits: Multiple VMs on one server →
Better resource utilization

Key Limitation: Each VM includes a full OS →
Heavy and slow to start





Containers - How do they work?

Base Layer: The physical server

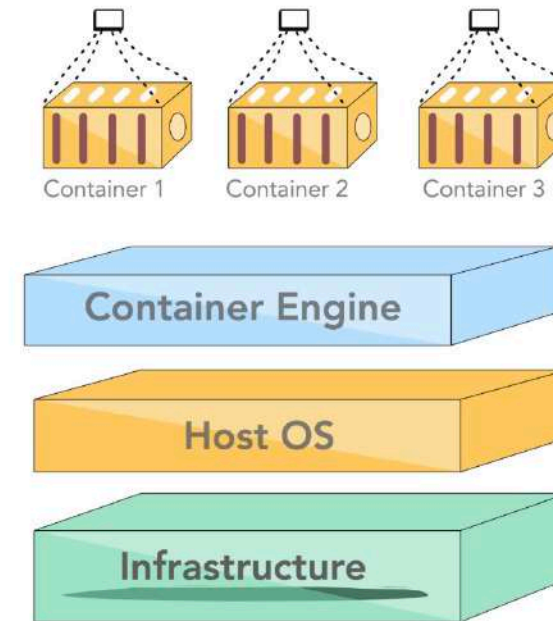
Host OS: Physical server runs a base OS

Container Engine / Runtime: Runs on top of the host OS

- Manages images, containers, etc.

Containers: Running instances created from container images

- Share the host OS kernel BUT isolated from other containers





Virtual Machines vs Containers

Feature	VMs	Containers
Architecture	Complete virtualized computers with their own OS, running on a hypervisor (Hardware Virtualization)	Lightweight, sharing the host OS core, running using Container Engine (OS layer Virtualization)
Performance	Resource-intensive (Each VM has full OS)	Lightweight (Host OS is shared)
Boot Time	Slower (Full OS startup)	Faster (Shares Host OS)
Resource Usage	Consumes more CPU, memory, storage	Efficient
Image Size	Large (includes OS)	Smaller (app + deps only)
Security	Stronger, separate OS for each VM	Shared OS kernel BUT Container engine ensures isolated processes for each container



Containers - A Summary

Standard Unit: Like shipping containers

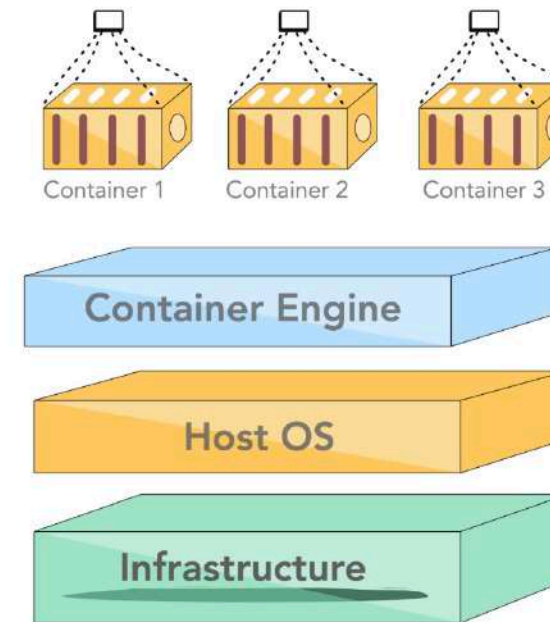
- **Before:** Different documents for Java, Python, NodeJs, ML, Analytics, etc.. (Manual)
- **After:** Everything is inside the container image
- **Ops Teams:** Only need to know how to run containers
 - **No Surprises:** Same steps everywhere for every app

Portable: Build once, run anywhere

- **Environments:** Local, on-prem data center, cloud

Fast Startup: Seconds or less

High Density: Many containers per server





in28minutes

Getting Started with Microservices



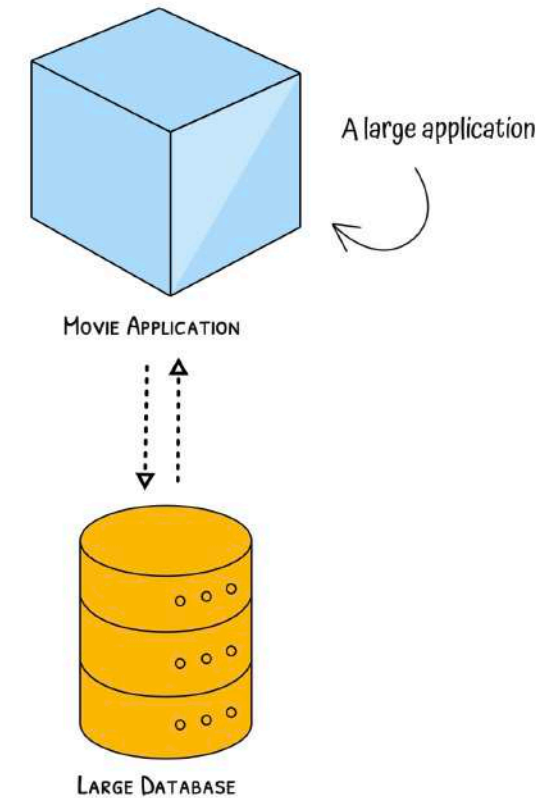
Understanding the Monolith Application

Monolith: A single large application

- **Database:** Typically connects to one large database
- **Codebase:** Can have millions of lines of code

Development: Entire team works on the same codebase

Challenge: Difficult to maintain and release





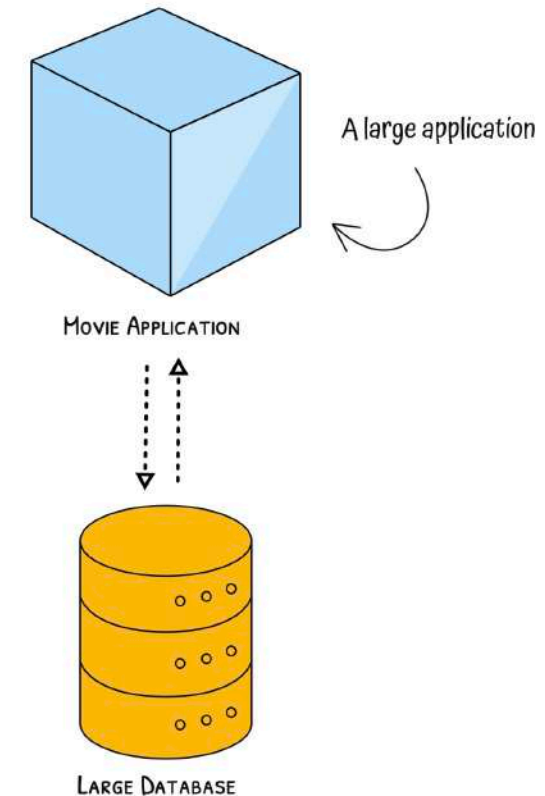
Challenges with Monolith Architecture

Deployment Complexity: Must deploy entire app for small changes

Release Cycles: Very long (months) due to regression testing

Scalability: Must scale entire app (expensive), not just components

Technology Lock-in: Hard to adopt new languages or frameworks



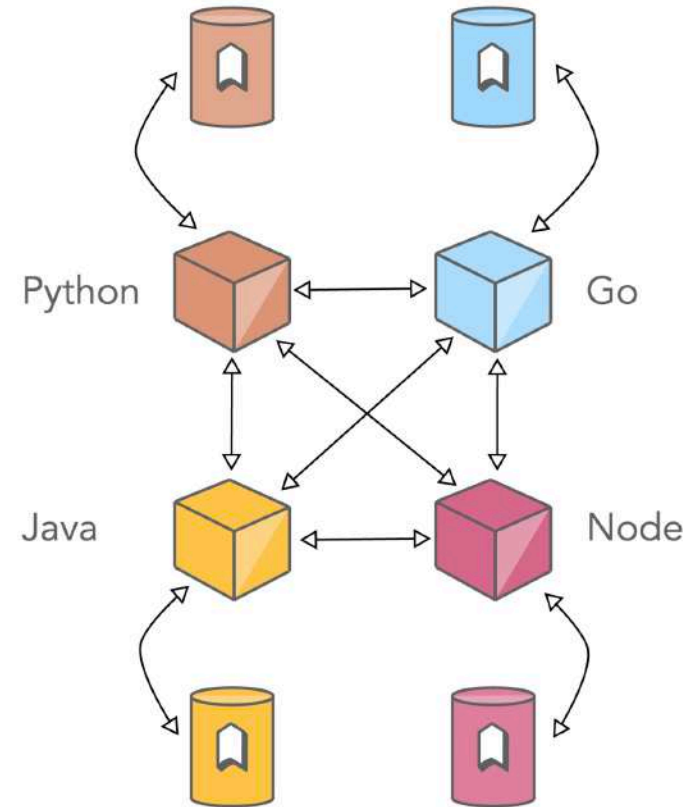


What Exactly is a Microservice?

Sam Newman: Small, autonomous services working together

James Lewis and Martin Fowler: An architectural style for developing apps as a suite of small, independently deployable, and scalable services

- Each running in its own process and organized around business capabilities
- These services communicate via lightweight mechanisms and feature decentralized data management and governance





Three Pillars of Microservices

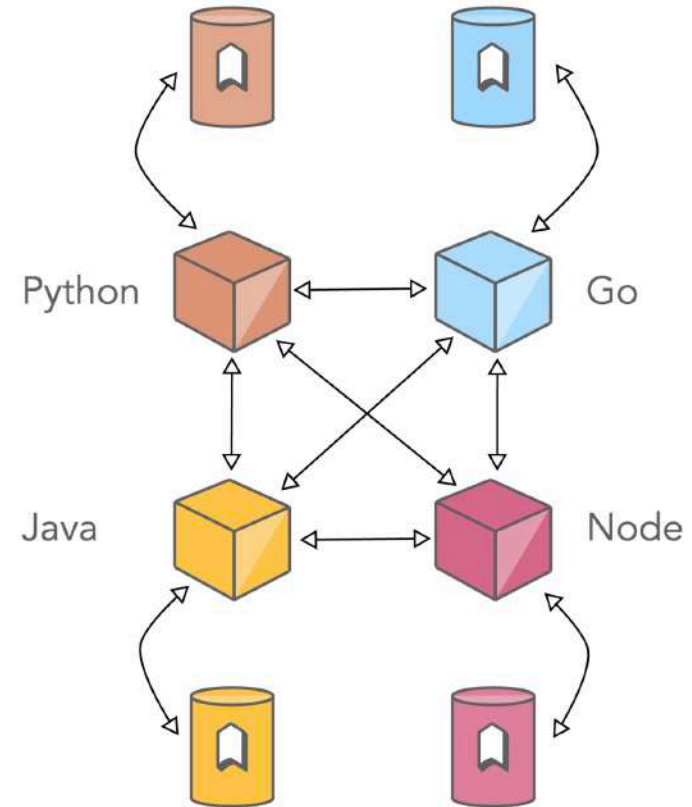
API-Based Communication: REST-first, lightweight service-to-service communication

Small Units: Well-chosen, independently deployable units

- **Boundaries:** Clearly defined functionality

Dynamic Scaling: Scale instances up/down independently

- **Benefit:** Adjust based on demand for specific services



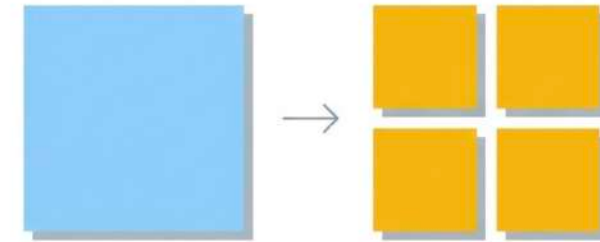


Monolith vs Microservices - An Example

Monolith: One massive app handling everything

Microservices: Decomposed into specific services

- **Movie Service:** Manages details and showtimes
- **Booking Service:** Handles tickets, seats, payments
- **Customer Service:** Handles profiles and authentication
- **Review Service:** Manages ratings and comments





Why Choose Microservices?

Flexibility: Adopt new technologies and processes easily

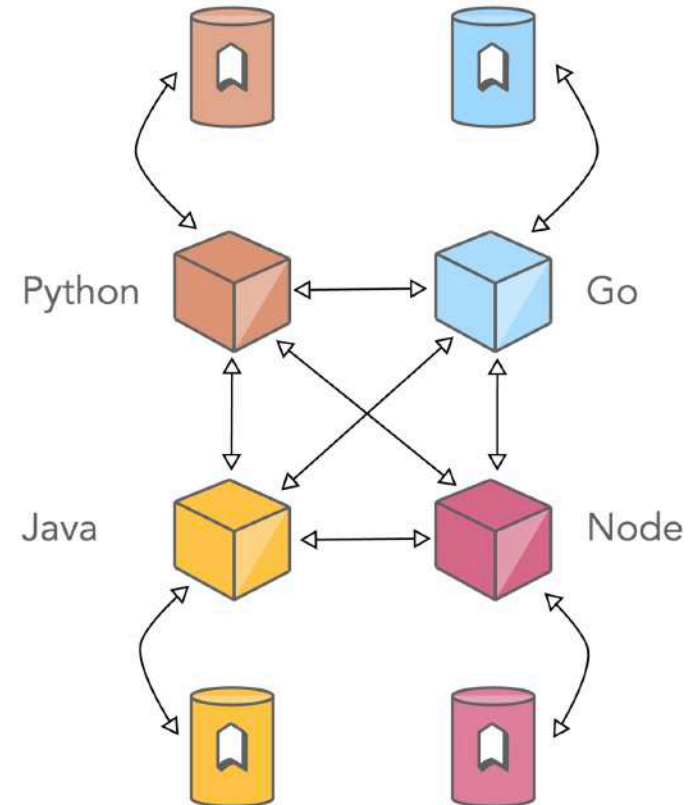
- **Innovation:** Experiment with different languages per service

Dynamic Scaling: Scale only what is needed

- **Cost:** More efficient than scaling a monolith

Faster Releases: Small independent units allow quick deployment

- **Agility:** Respond faster to market demands





in28minutes

Getting Started With

Microservices Challenges



Challenges With Microservices Architectures

Standardization: Need standardized operations

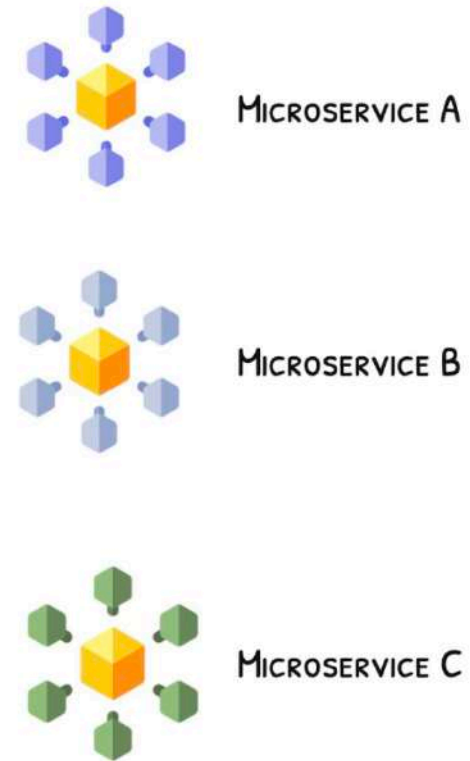
Auto Scaling: Services must scale automatically

Load Balancing: Distribute traffic between instances

Service Discovery: How do services find each other?

Release Management: Flexible releases

Resiliency: Handle failures automatically



Standardized Operations for Microservices

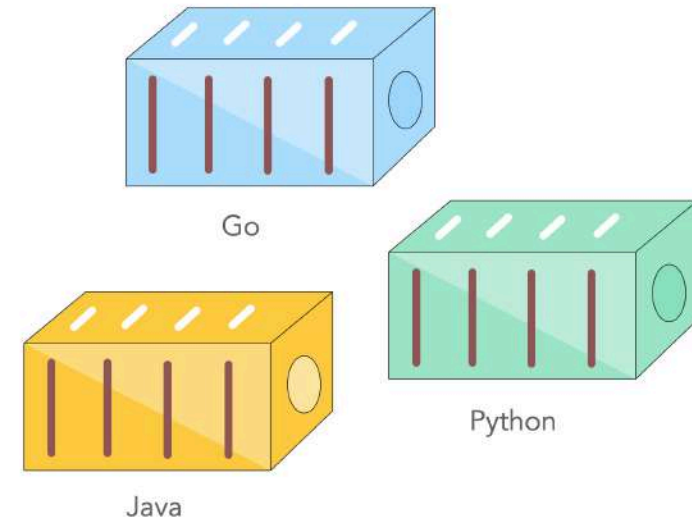


Challenges:

- **Diverse Tech Stack:** Java, Python, Node.js, etc.

Need: Standardized Operations

- Release Management
- Observability
 - Logs
 - Metrics
 - Traces
- Etc.





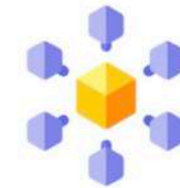
Automatic Scaling for Each Microservice

Multiple Microservices: A number of microservices with varying load

Varying no of instances: Each microservice will have different number of instances

Auto Scaling: Need to scale each microservice automatically based on load

Efficiency: Scale up and down automatically, saving costs



MICROSERVICE A



MICROSERVICE B



MICROSERVICE C



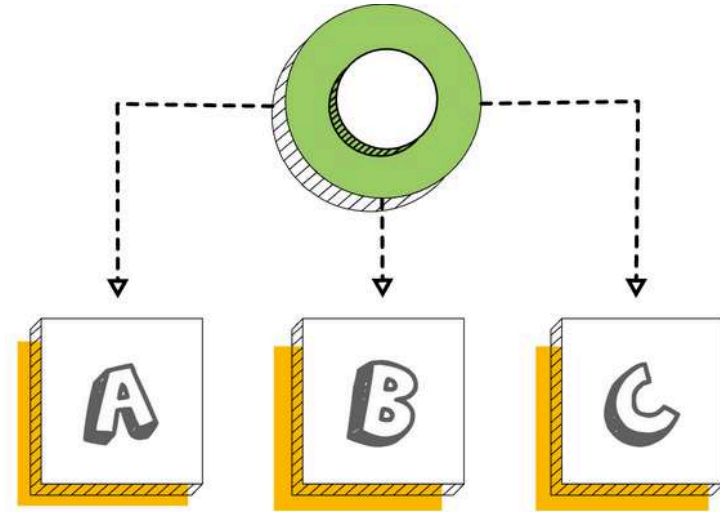
Load Balancing between Microservice Instances

Multiple Microservices: A number of microservices with varying load

Varying no of instances: Each microservice will have different number of instances

Load Balancing: Distribute traffic across instances

Performance: Ensure no single instance is overwhelmed





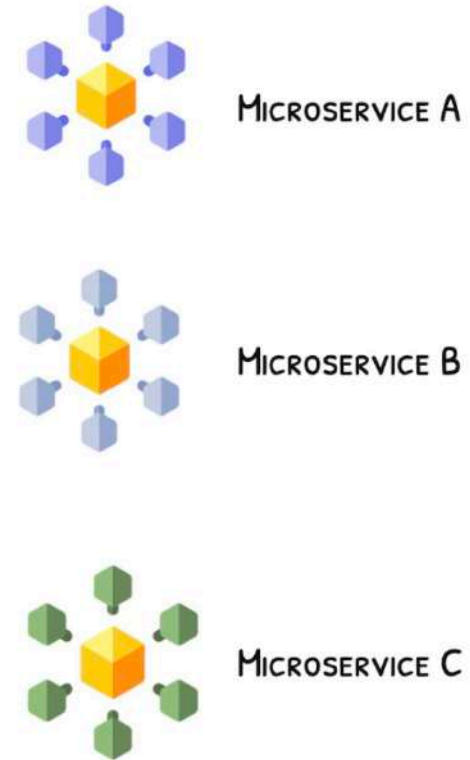
Help Microservices Locate Each Other - Service Discovery

Communication: Microservices need to talk to each other

Challenge: Microservice instances are created dynamically - as and when they are needed

- **Dynamic IPs:** IP addresses might keep changing
- **Best Practice:** Avoid hardcoding URLs

Need: Microservices should be able to locate each other whenever needed





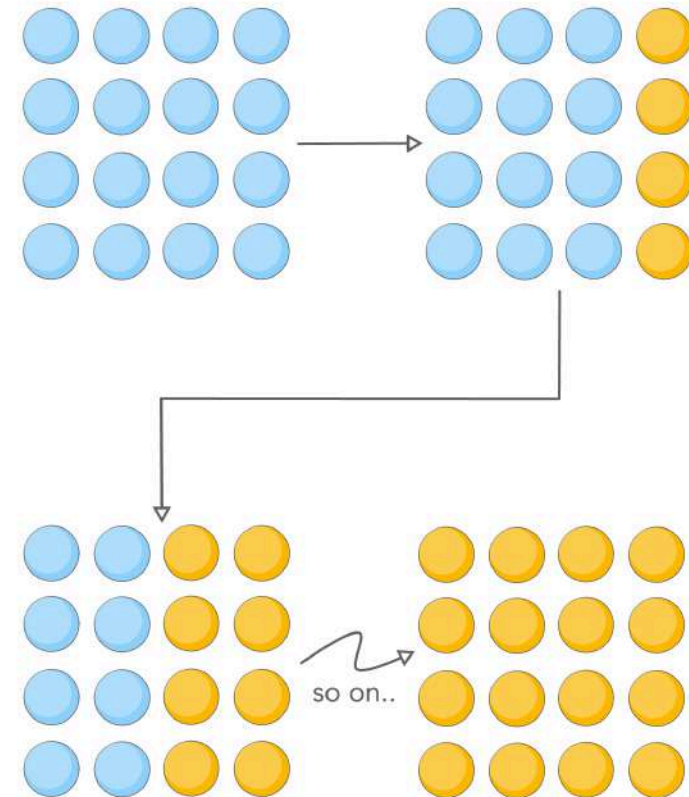
Manage Releases of Microservices

Frequent Releases: New versions of microservices are released very often

Need: Zero downtime releases with rollback possibilities

Example Strategy:

- **Rolling Updates:** Update instances in batches



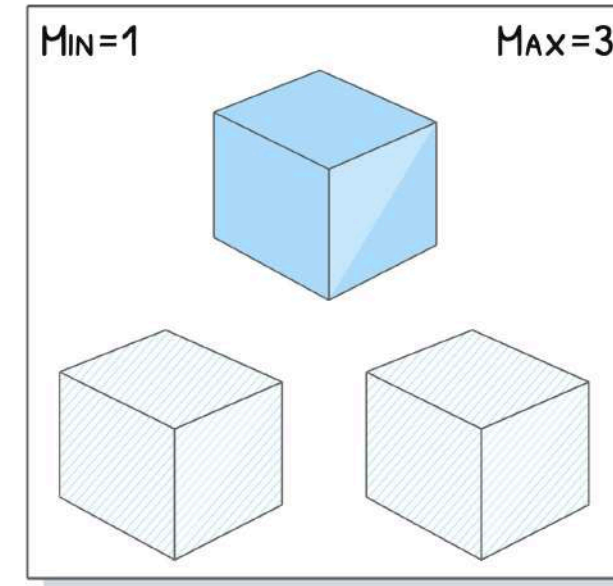


Resiliency - Identify and Act on Failures

Reality: Systems, nodes and applications fail often

Need: Recover automatically

- **Isolation of Failures:** Prevent one failure from crashing the whole system
- **Self-Healing:** Detect and recover from failures automatically





Challenges With Microservices Architectures - Summary

Standardization: Need standardized operations

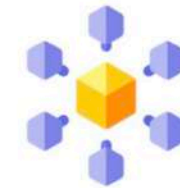
Auto Scaling: Each service must scale automatically

Load Balancing: Distribute traffic between instances

Service Discovery: How do services find each other?

Release Management: Flexible releases

Resiliency: Handle failures automatically



MICROSERVICE A



MICROSERVICE B



MICROSERVICE C



in28minutes

Exploring Solutions

Containers and Orchestration



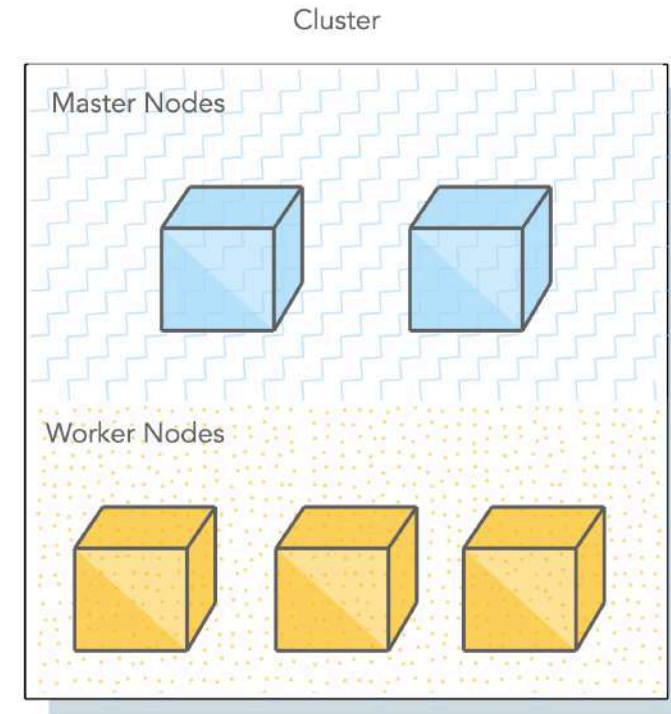
Challenges With Microservices Architectures - Solutions

Containers: Enable

- **Standardization:** Need standardized operations

Container Orchestration: Enable

- **Auto Scaling:** Scale services automatically
- **Load Balancing:** Distribute traffic between instances
- **Service Discovery:** Help services located other services
- **Release Management:** Flexible release strategies
- **Resiliency:** Handle failures automatically





Standardization with Containers

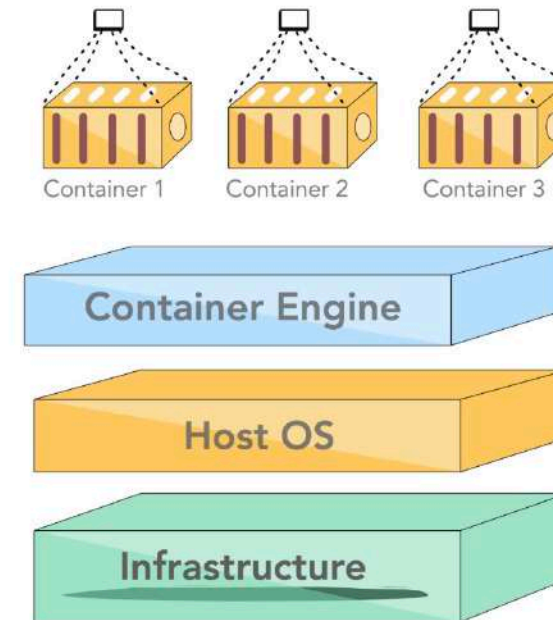
Diverse Tech Stack: Java, Python, Node.js, etc.

Need: Standardize Operations

- **Solution:** Containers
- **How?:** Build a container image for each microservice

Benefits:

- **Deployment:** Deploy all microservices the same way
- **Observability:** Standardize how services expose logs, metrics and traces



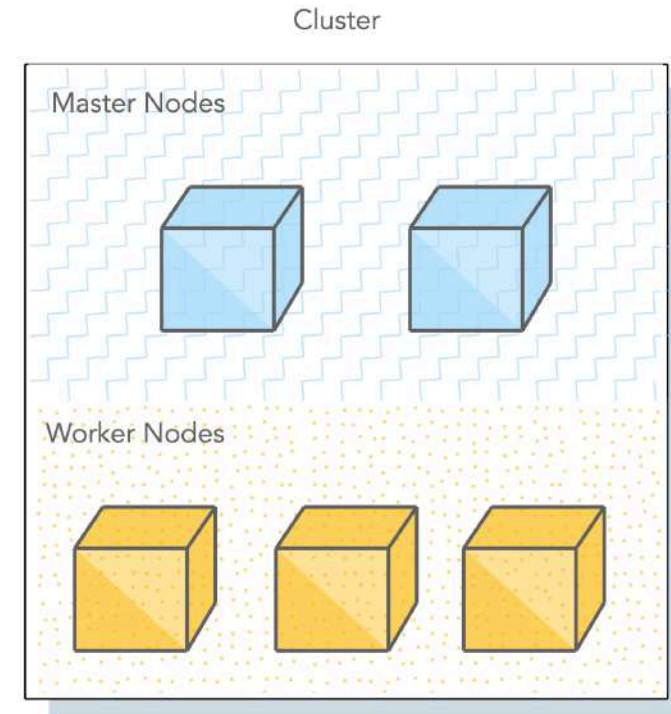


What is Kubernetes?

Kubernetes: An open-source container orchestration platform

How It Works (Simplified):

- You create a Cluster
- Declare your needs: **Microservice A** (3 instances), **Microservice B** (4 instances) and ...
 - Kubernetes deploys and manages these microservices
 - Kubernetes continuously monitors and reconciles the desired state





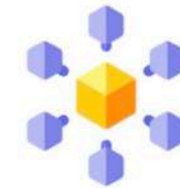
Automatic Scaling for Each Microservice

Multiple Microservices: A number of microservices with varying load

Auto Scaling: Need to scale each microservice automatically based on load

Solution: Kubernetes Auto Scaling Solutions

- **Declarative:** Tell Kubernetes that you need auto scaling for Microservice A



MICROSERVICE A



MICROSERVICE B



MICROSERVICE C



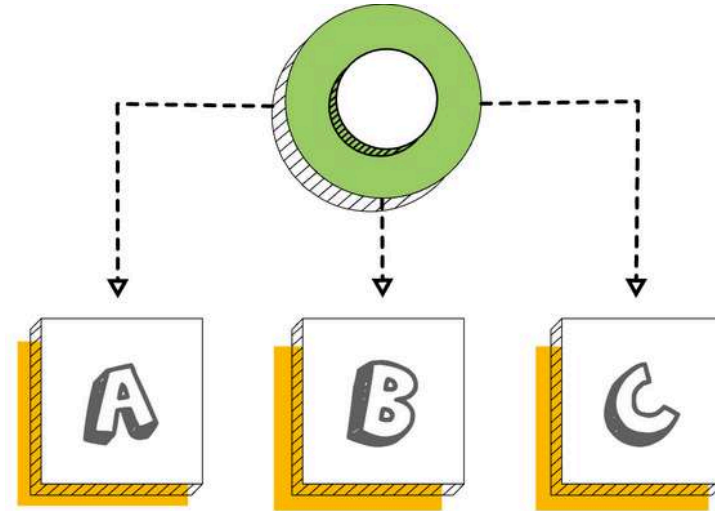
Load Balancing between Microservice Instances

Varying no of instances: Each microservice will have different number of instances

Need: Distribute traffic across instances

Solution: Kubernetes Service

- **How?:** Use Kubernetes Service Name to distribute traffic automatically among the microservices instances





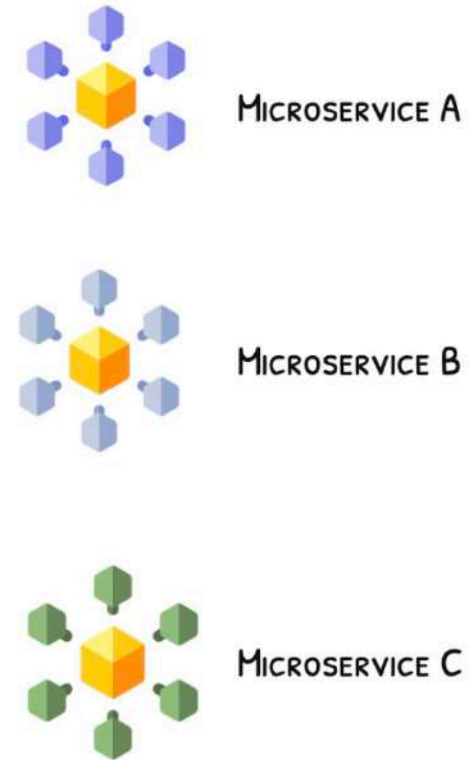
Help Microservices Locate Each Other - Service Discovery

Communication: Microservices need to talk to each other

Challenge: Dynamic IP addresses of containers

Solution: Kubernetes service discovery

- **Mechanism:** Use Kubernetes Service Name to discover other microservices
- **Benefit:** No need to hard-code IP addresses





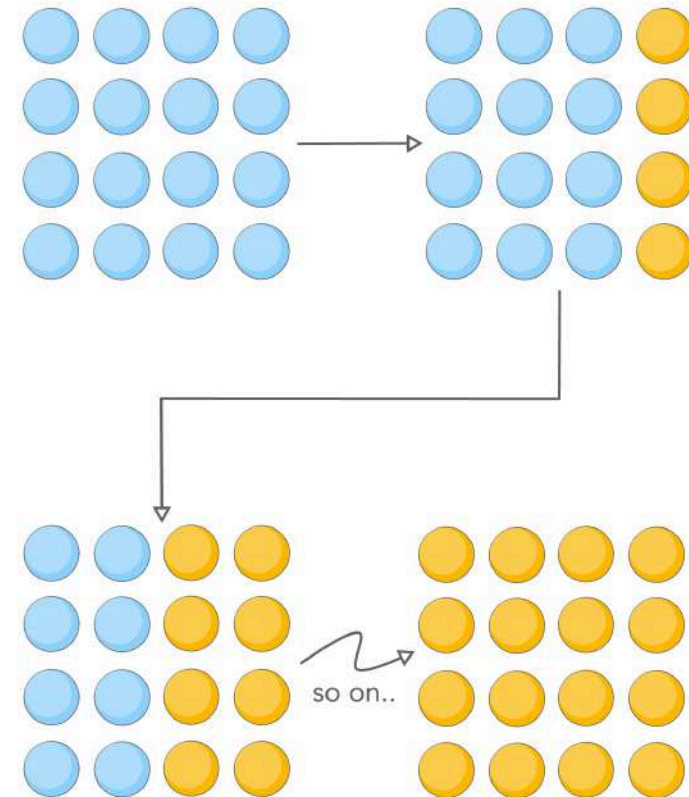
Manage Releases of Microservices

Challenge: Microservices need frequent releases with zero downtime

Solution: Kubernetes Deployments

Example Strategy:

- **Rolling Updates:** Update instances in batches





Resiliency - Identify and Act on Failures

Reality: Systems, nodes and applications fail often

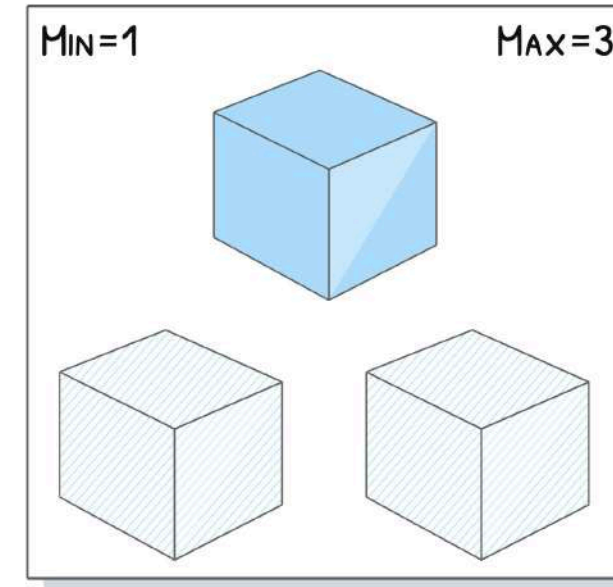
- **Need:** Recover automatically

Isolation: Prevent one failure from crashing the whole system

- **Solution:** Kubernetes ensures that a failure of microservice instance does not impact other microservices

Self-Healing: Restart or replace failed containers

- **Solution:** Kubernetes health checks



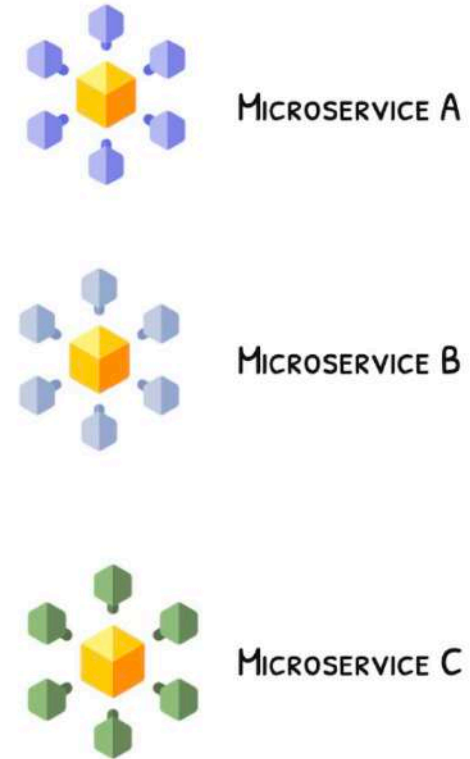
Challenges With Microservices Architectures - Solutions



Standardization: Containers (Docker)

Container Orchestration: With Kubernetes

- **Auto Scaling:** Kubernetes Auto Scaling Solutions
- **Load Balancing:** Kubernetes Services
- **Service Discovery:** Kubernetes Service Discovery
- **Release Management:** Flexible Deployment Strategies
- **Resiliency:** Kubernetes Self-Healing



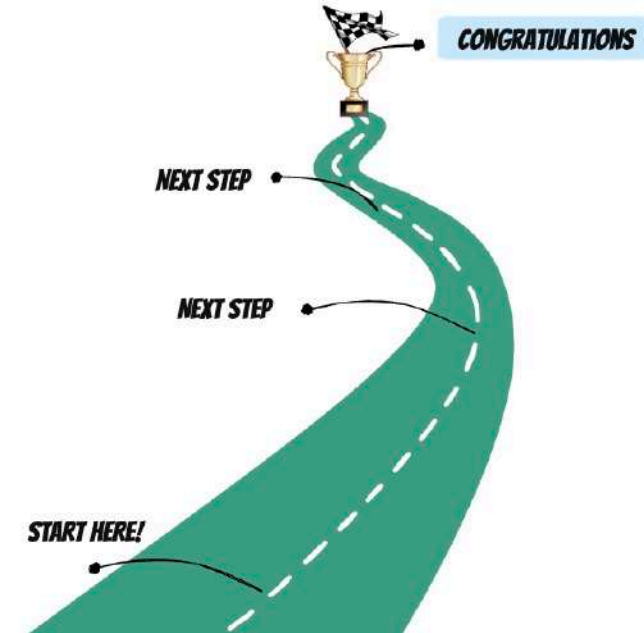


Summary - Journey to Microservices

The Journey:

- **Problem:** Challenges with monolith applications
- **Solution:** How microservices address the challenges
- **New Challenges:** Complexity introduced by microservices
- **Solution:** Containers and container orchestration

Key Outcome: Understand microservices and the need for containers and orchestration





in28minutes

Google Cloud

Compute Managed Services



Google Cloud Managed Services for Compute

Service	Details	Category
Google Compute Engine	Create VMs and VM groups	IaaS
Google App Engine	Build and deploy highly scalable applications on a fully managed platform	PaaS (Containers, Serverless)
Google Cloud Run	Build and deploy highly scalable containerized applications. Does NOT need a cluster!	PaaS (Containers, Serverless)
Google Cloud Run Functions	Build event driven applications using simple, single-purpose functions	PaaS (Functions, Serverless)
Google Kubernetes Engine	Orchestrate containerized microservices on Kubernetes Needs advanced cluster configuration and monitoring	PaaS (Container Orchestration)



Scenarios - Google Cloud Compute Services

Scenario	Google Cloud
How do you create Virtual Machines?	Google Compute Engine
How do you create a group of similar VMs?	Managed Instance Groups
How do distribute load among VMs?	Cloud Load Balancing
What is the easiest way to run a single container based application in Google Cloud?	Google Cloud Run
How do you orchestrate containers in Google Cloud?	Google Kubernetes Engine (GKE)
How do you build serverless event driven functions in Google Cloud?	Cloud Run Functions



in28minutes

Getting Started with Google App Engine



Getting Started with App Engine

Scenario: Deploy an app quickly (even without containerizing) to Google Cloud

- **Solution:** App Engine

Features:

- **Standard:** Deploy Go, Java, .NET, Node.js, PHP, Python, Ruby, .. apps using pre-configured runtimes
- **Flexible:** Deploy Apps built in any language
- **Deploy to Scaling:** Handles everything
- **Advanced Features:** Automatic LB, Auto scaling, App versioning, Traffic splitting, App health monitoring





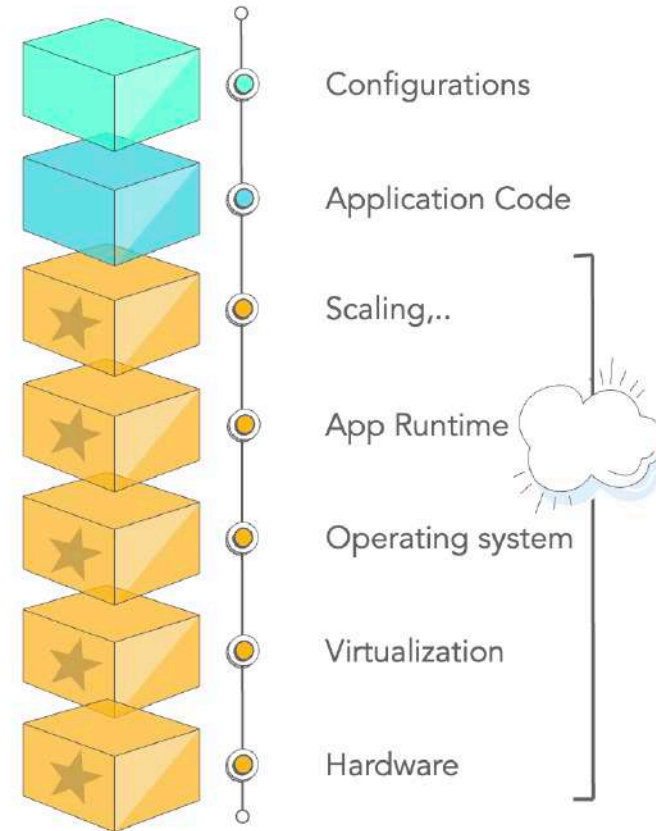
Compute Engine vs App Engine

Compute Engine: IaaS

- MORE Flexibility
- BUT MORE Responsibility
 - Choosing Image, Installing Software, Choosing Hardware
 - Fine grained Access/Permissions (Certificates/Firewalls)
 - Availability etc

App Engine: PaaS

- Serverless
- LESSER Responsibility
- LOWER Flexibility

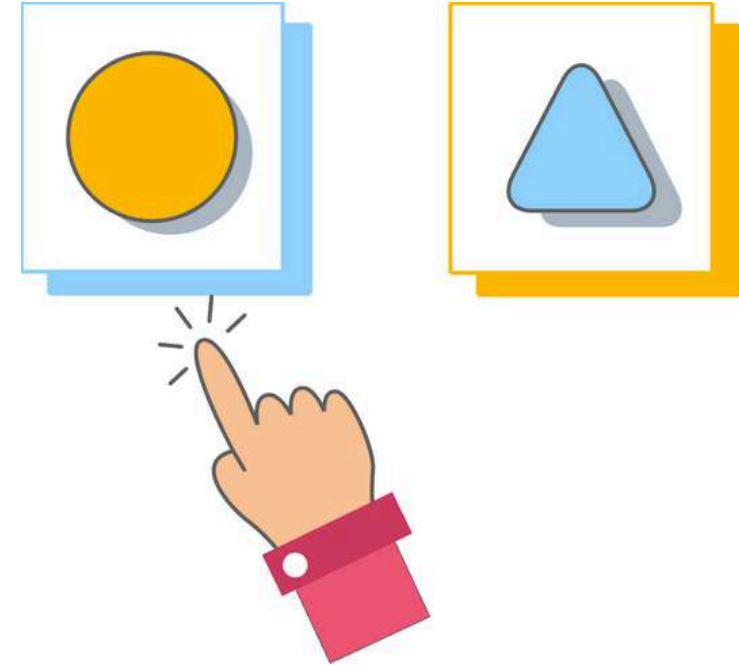




App Engine environments - Standard

Standard: Applications run in language specific sandboxes

- Supported Language Examples: Java, Python, PHP, Node.js, Ruby, Go
- Instance startup time: Seconds
- Complete isolation from OS/Disk/etc.
- Recommended if your want to:
 - Run your apps at very low cost (Pay only for what you need and when you need it. Example: scale to 0 instances when there is no traffic) OR
 - Your app experiences sudden and extreme spikes of traffic and you need immediate scaling





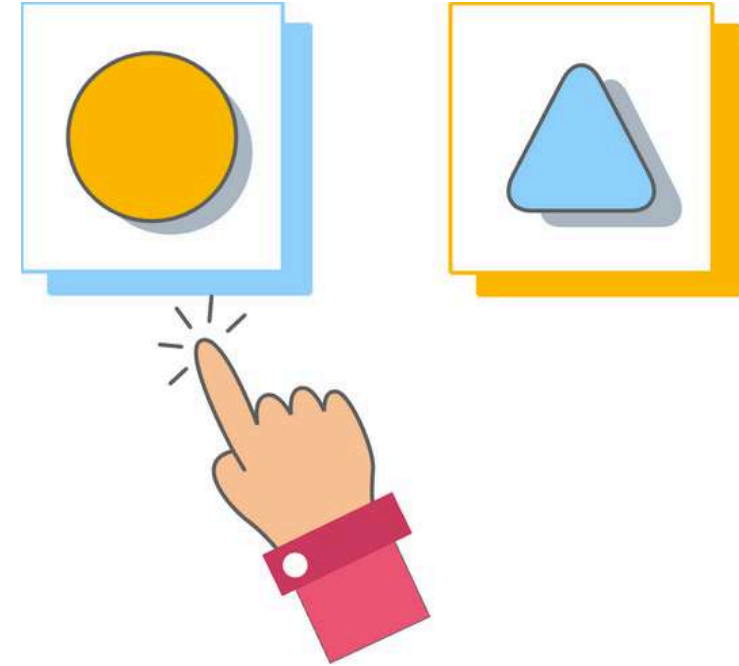
App Engine environments - Flexible

Flexible: Application instances run within containers

- **GCE VMs:** Makes use of GCE VMs
- **No Runtime Restrictions:** Support ANY runtime
- **Built-in support:** For Python, Java, Node.js, Go, Ruby, PHP, .NET
- **Get Access to:** SSH, Background processes and local disks

Recommended For Containerized Apps:

- Receiving consistent traffic (gradual scaling)
- Needs access to Background processes





App Engine environments - Standard vs Flexible

Feature	Standard	Flexible
Makes use of GCE VMs	No	Yes
Key Pricing Factors	Instance hours	vCPU, Memory & Persistent Disks
Scaling to zero	Yes	No. Minimum one instance
Instance startup	Seconds	Minutes
Rapid Scaling	Yes	No
SSH for debugging	No	Yes



App Engine - Application Component Hierarchy

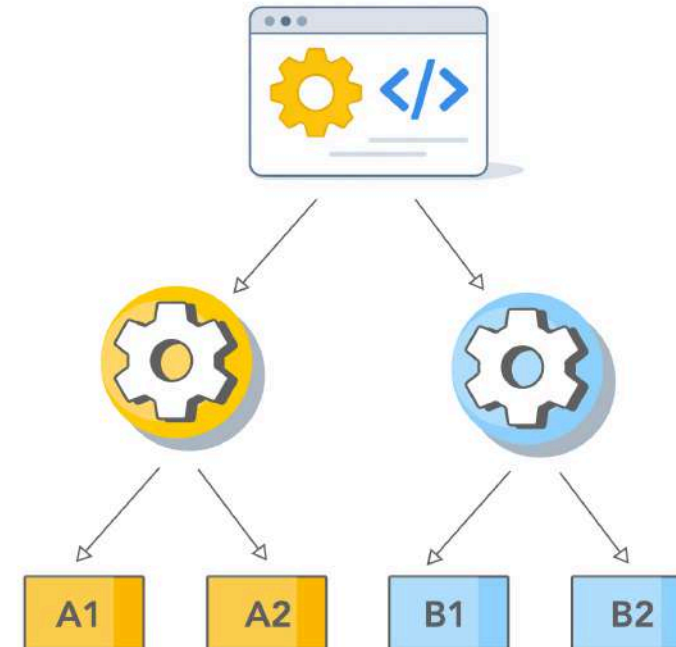
Application: One App per Project

Service(s): Multiple Microservices or App components

- You can have multiple services in a single application
- Each **Service** can have different settings
- Earlier called Modules

Version(s): Each version associated with code and configuration

- Each **Version** can run in one or more instances
- Multiple versions can co-exist
- Options to rollback and split traffic





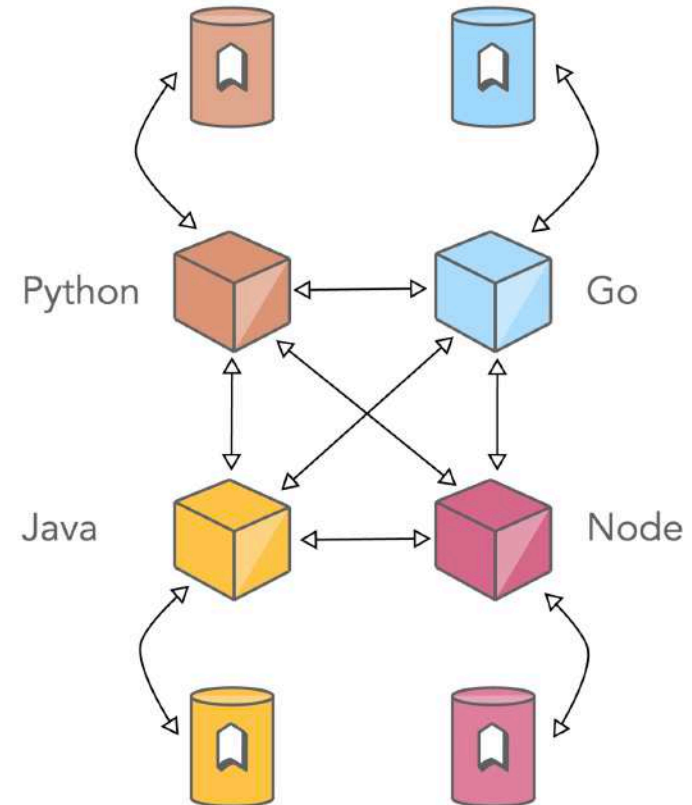
App Engine - Remember

Regional: Services deployed in one region across zones

- You **CANNOT** change an Application's region

Good option: For simple microservices (multiple services)

- Use **Standard** when you are using supported languages
- Use **Flexible** if you are building containerized apps
 - **ATLEAST one instance** is always running when using **Flexible**
 - **Go for Standard** if you want to scale down to **zero** instances when there is NO load





Getting Started with App Engine - Scenarios

Scenario	Solution
Which of these minimizes operations (App Engine or Compute Engine?)	App Engine
Maximum Number of Applications in a Project	One
You want rapid scaling. Should you go for App Engine Standard or Flexible?	Standard
Which of these supports scaling to zero - App Engine Standard or Flexible?	Standard
Explain the structure - You want to create multiple versions of a Microservice using App Engine	Create a service for each microservice. Each service can have multiple versions.



in28minutes

Getting Started with Cloud Run



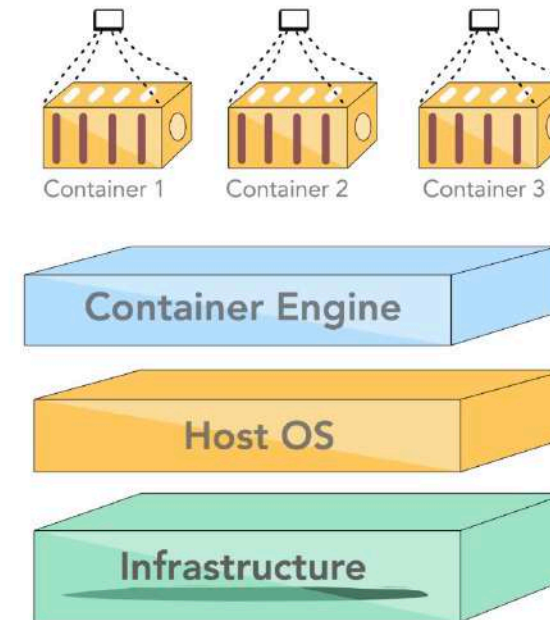
Getting Started with Cloud Run

Cloud Run: Deploy Containers in Seconds

- Built on top of an open standard - **Knative**
- **Fully managed** serverless platform
 - ZERO infrastructure management
 - Pay-per-use (Key cost factors: Used CPU, Memory and Request count)

Fully integrated **end-to-end developer experience**:

- **No limitations**: in languages, binaries and dependencies
- **Simple Integrations**: Cloud Build, Cloud Monitoring, Cloud Logging etc.

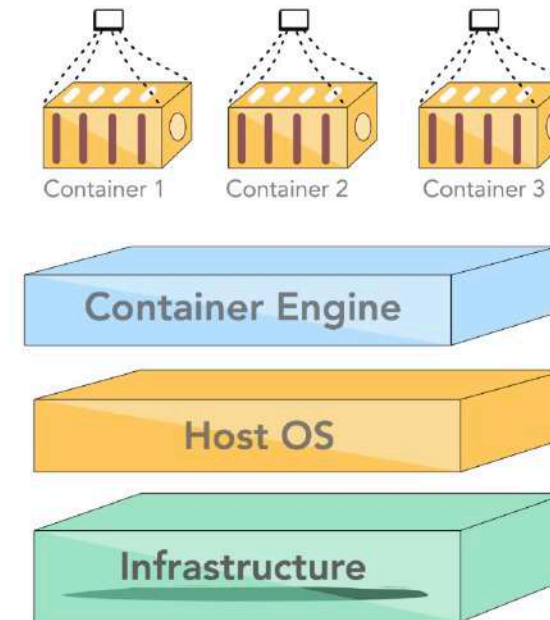




App Engine vs Cloud Run

Newer and More Flexible: Cloud Run is the newer, more flexible alternative

- **Built on Experience:** Cloud Run builds upon over a decade of operational learnings from App Engine
- **Deployment Options:** Both support deploying from source code or container images
- **Scaling and Cost:** Both scale to zero (except App Engine Flexible). However, Cloud Run offers faster startup times and superior cost efficiency for spiky traffic.
- **Example Features in Cloud Run But NOT in App Engine:** Multi Region Support, Committed Use Discounts (CUDs)





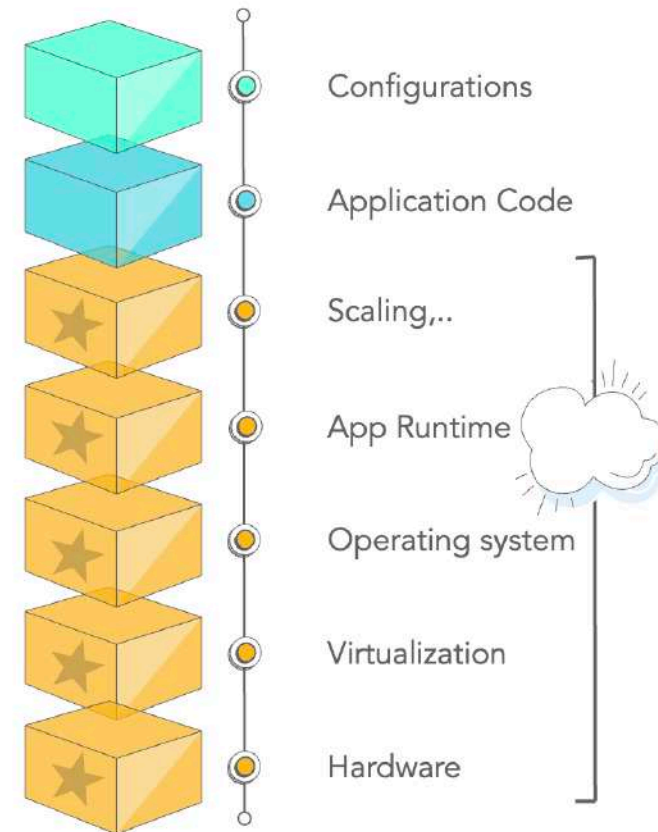
Compute Engine vs Cloud Run

Compute Engine: IaaS

- MORE Flexibility
- BUT MORE Responsibility
 - Choosing Image, Installing Software, Choosing Hardware
 - Fine grained Access/Permissions (Certificates/Firewalls)
 - Availability etc

Cloud Run: PaaS (Serverless Container Platform)

- Serverless
- LOWER Flexibility
- LESSER Responsibility





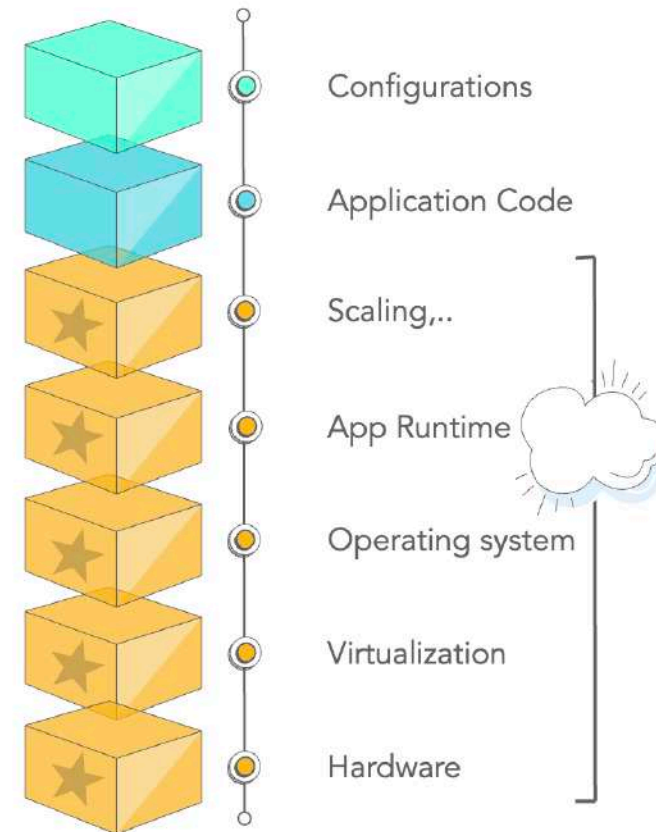
Advantages of Cloud Run

Language neutral: Applications can be built using any programming language or framework

Minimal ops: No infra, OS, or runtime management

Automatic scaling: Handle variable traffic efficiently (Scale to zero)

Cost efficient: Pay only for actual usage





Getting Started with Cloud Run - Scenarios

Scenario	Solution
Which of these is the newer, more flexible serverless option - App Engine or Cloud Run?	Cloud Run
Hosting containerized workloads with highly intermittent or spiky traffic at absolute minimum cost	Cloud Run
Migrating existing containerized application to Google Cloud with minimal effort	Cloud Run
Deploying a Knative-compatible application with zero infrastructure management	Cloud Run
Deploying a simple Python application with low or unpredictable traffic cost-effectively without managing runtimes or web servers	Cloud Run



Typical Steps in Using Cloud Run

Containerize: Package a container image

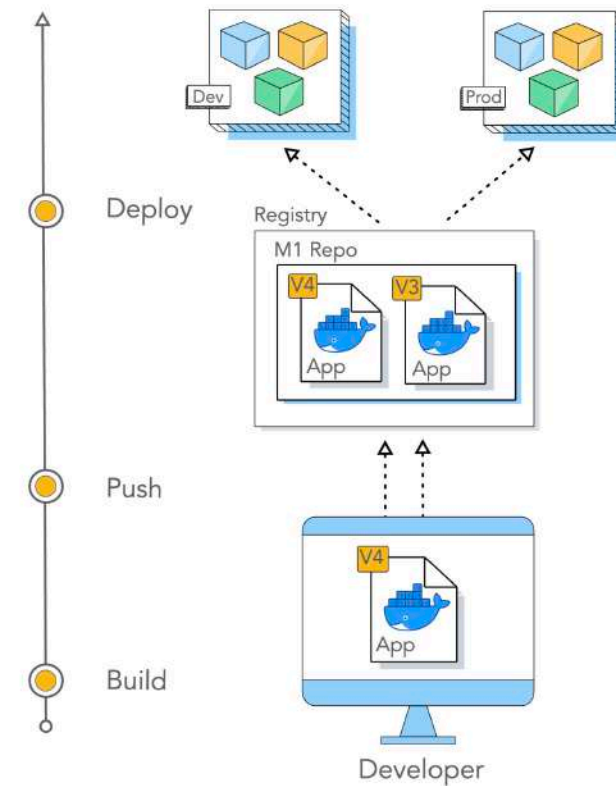
- OR let Cloud Run build a image from source code using a Dockerfile or buildpacks

Push: Push the container image to Artifact Registry

Deploy: Run your container on Cloud Run

Automatic scaling: Scales based on incoming requests

Release management: Deploy versions and split traffic





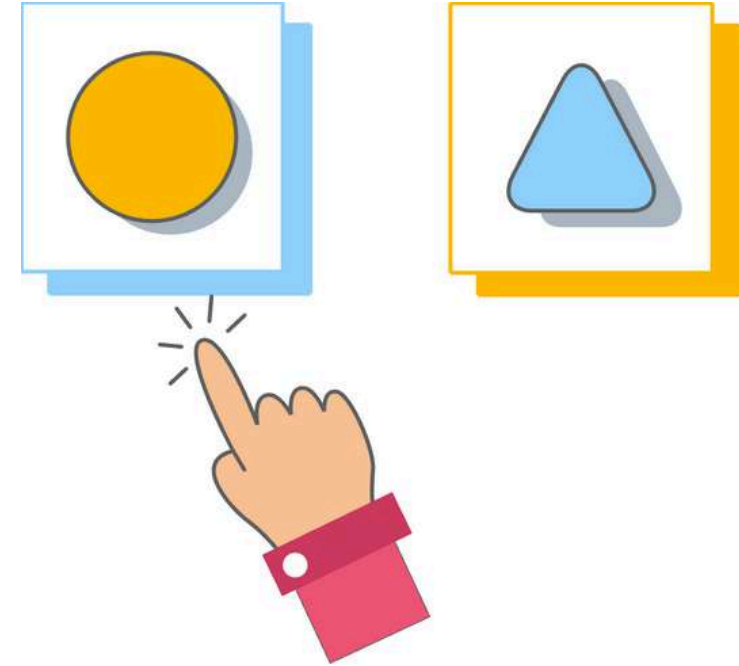
Cloud Run - Multiple Options

Web apps & APIs - Cloud Run Services

Handle Event Triggers (From Pub/Sub, Storage) - Cloud Run Functions

Run a batch job (Run-to-completion) - Cloud Run Jobs

Perform continuous background work - Cloud Run Worker Pools





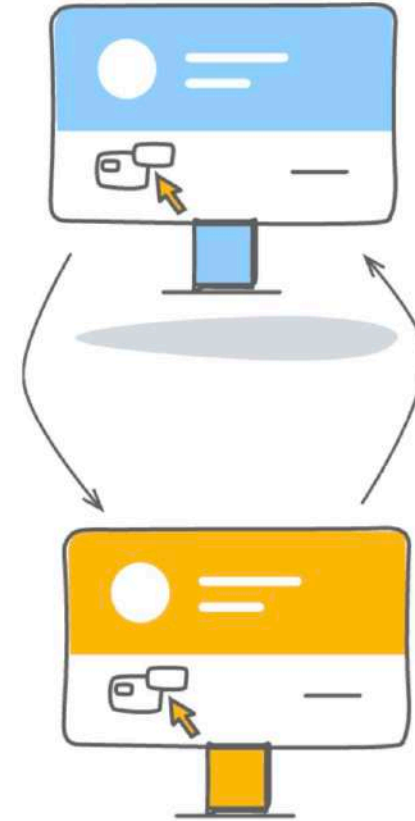
Cloud Run Services

Use for web apps & APIs: Handle HTTP/2, WebSockets, gRPC, etc.

Auto scaling: Automatically scales instances based on load (Scale to zero when there are no requests)

Traffic management: Seamlessly deploy new revisions, roll back instantly, or split traffic

Target Workloads: Ideal for e-commerce backends, REST APIs, and simple microservices





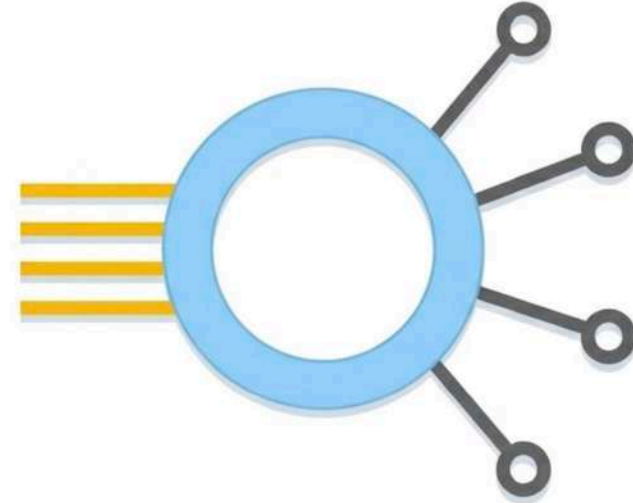
Cloud Run Functions

Lightweight, Single-Purpose Logic: Deploy small, focused functions

Event-Driven Architecture: Automatically triggers in response to cloud events

Source-to-Deploy Workflow: Deploy code directly; Google handles the underlying work

Example Workloads: Generating image thumbnails or sending an email when a file is uploaded to Cloud Storage



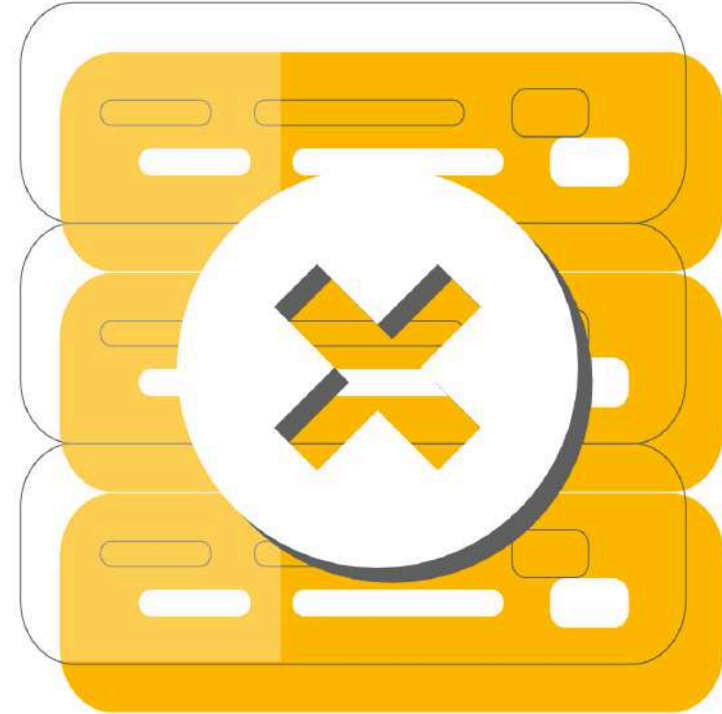


Cloud Run Jobs

Run-to-Completion Tasks: Built specifically for workloads that execute a finite set of steps and then cleanly shut down

Time-Based Scheduling: Seamlessly integrates with Cloud Scheduler to run routine tasks automatically on a schedule

Example Workloads: Nightly database backups, end-of-month financial reporting, or database migrations





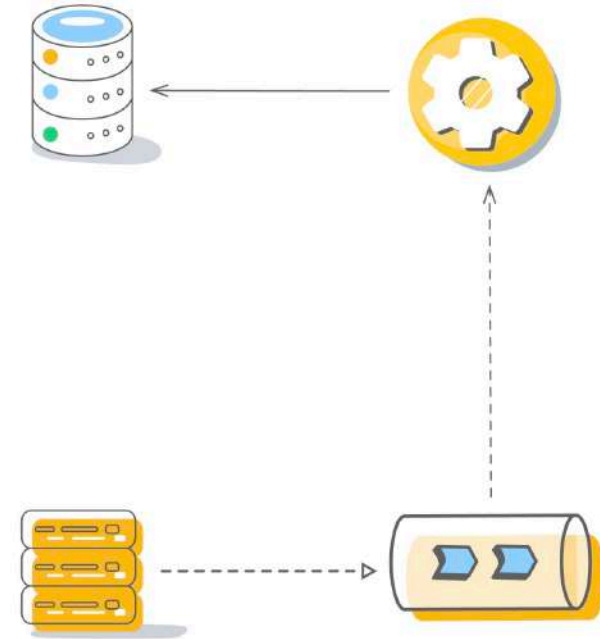
Cloud Run Worker Pools

Continuous Background Workers: Built for non-request, pull-based workloads with no public HTTP endpoints

- **Streamlined Queue Consumption:** Ideal for architectures that pull tasks directly from message brokers (e.g., Pub/Sub Pull, Apache Kafka).

Optimized Costs: Eliminates the overhead of public endpoints and load balancers

Example Workloads: Process messages continuously from Pub/Sub, Kafka, or other message queues





Cloud Run - Services vs Jobs vs Worker Pools vs Functions

Scenario	Recommendation
Build a web application, REST API, or handle streaming connections (WebSockets/gRPC)	Cloud Run Service
Run continuous background workers consuming messages from Pub/Sub or Kafka	Cloud Run Worker Pools
Execute a run-to-completion task (batch data processing, database migration)	Cloud Run Job
Run a scheduled task like a daily cleanup script or report generation	Cloud Run Job <i>(via Cloud Scheduler)</i>
Deploy simple, event-driven lightweight logic without managing containers	Cloud Run Function



Cloud Run - Component Hierarchy

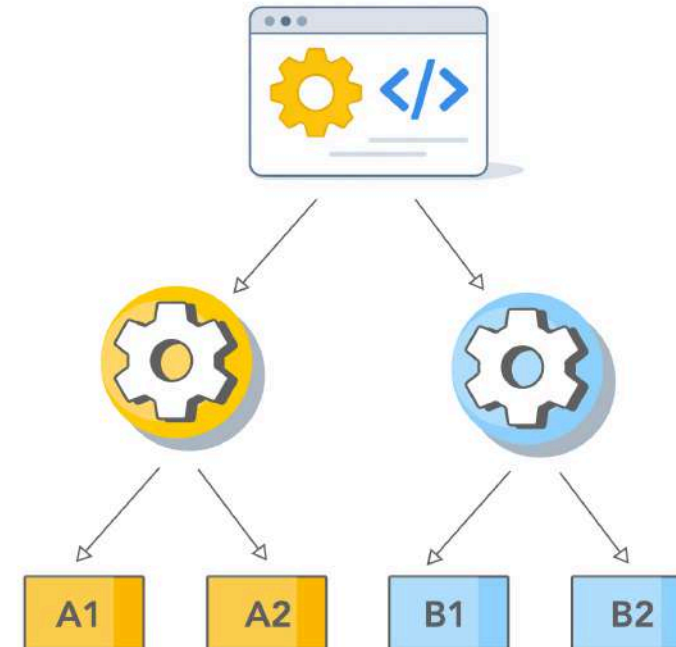
Project: The Google Cloud boundary wrapping your resources

Service: The top-level Cloud Run resource representing a microservice or application

Revision: Represent a Service Version

- Multiple revisions can co-exist simultaneously
- Allows for flexible release management and granular traffic splitting

Instance: The underlying container runtime environment created to handle requests for a specific Revision





in28minutes

Getting Started with Kubernetes



Challenges With Microservices Architectures

Large No of Services: Need for automation

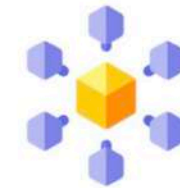
Auto Scaling: Must scale automatically

Load Balancing: Distribute traffic between instances

Release Management: Flexible releases

Resiliency: Automatically recover from failures

And a lot's more: Service Discovery, Centralized Configuration Management, etc.



MICROSERVICE A



MICROSERVICE B



MICROSERVICE C



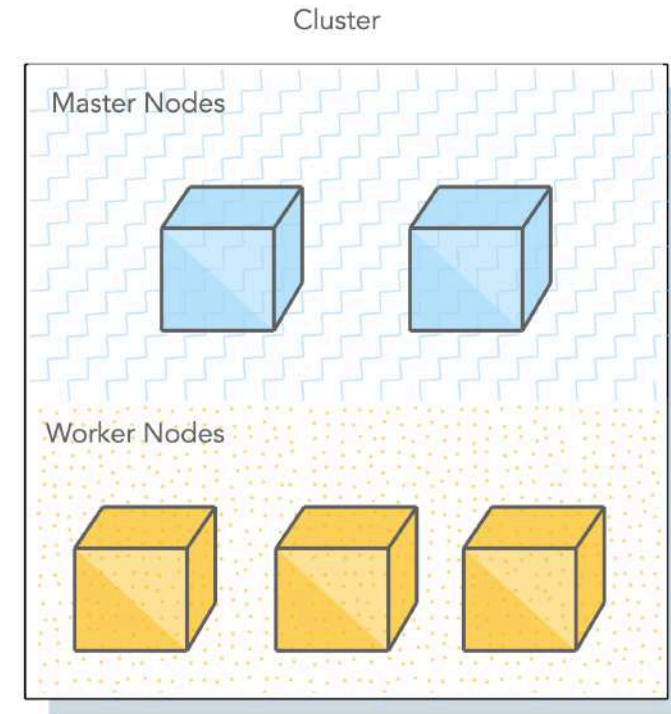
What is Kubernetes?

Kubernetes: An open-source container orchestration platform

- **Goal:** Automate deployment, scaling, and management of containerized apps
- **Origin:** Created at Google, now managed by CNCF

How It Works (Simplified):

- Create a Cluster
- Declare your needs: I need 2 instances of **Microservice A**, 5 instances of **Microservice B**, etc.
- Kubernetes will do its magic:
 - Deploys the containers
 - Monitors their health





Understanding Kubernetes Architecture

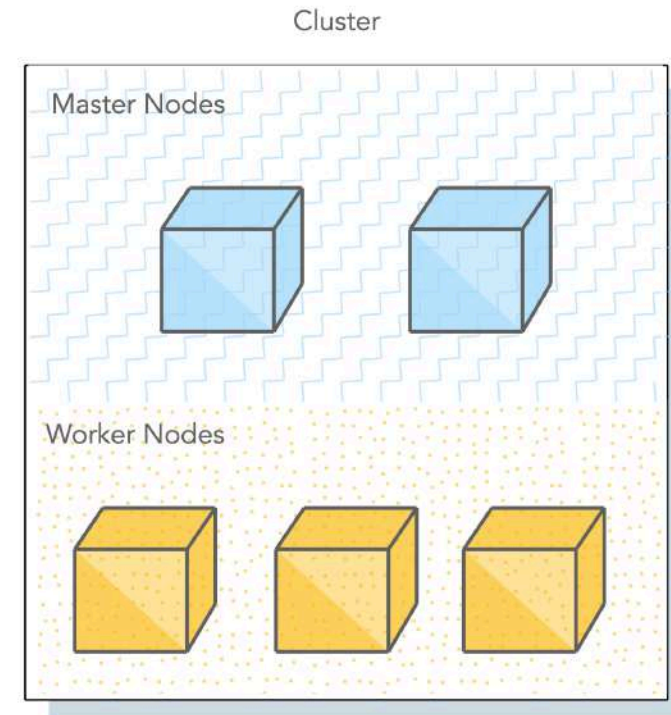
Control Plane (Master Nodes): Manages the cluster

Worker Nodes: Runs your applications

Declare your needs: I need 2 instances of **Microservice A** (and ..)

- Control plane schedules workloads
- Monitors and ensures high availability

Recommendation: Multiple Master Nodes For High Availability





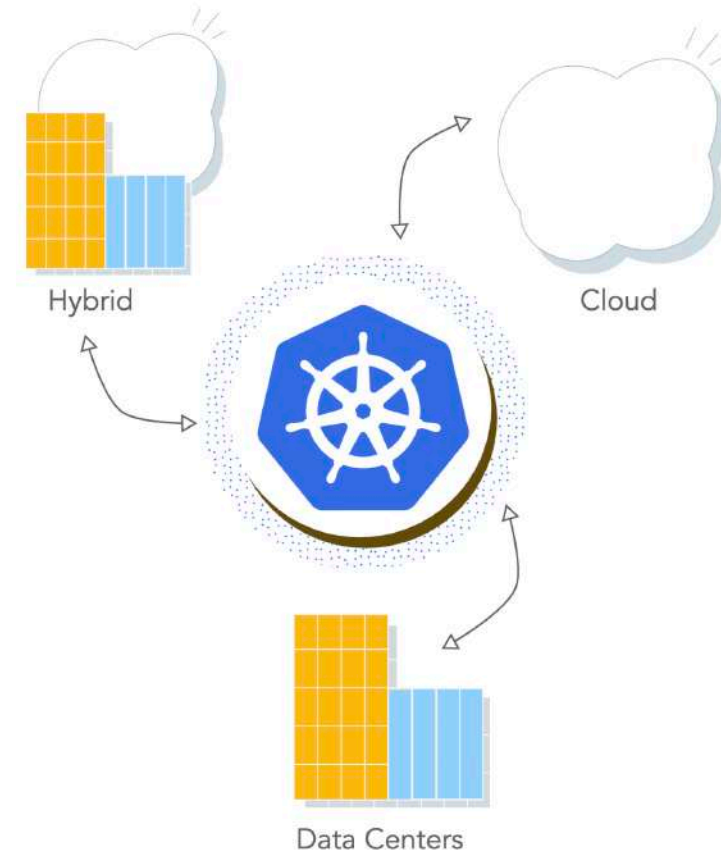
Kubernetes is Cloud and Platform Agnostic

Run Anywhere: On-premises, Public Cloud, Hybrid, or Edge

- **On-premises:** Use OpenShift, Rancher, etc.
- **Cloud:** AWS (EKS), Azure (AKS), Google Cloud (GKE)
- **Hybrid Cloud:** Combine on-prem and cloud clusters

Provides: Portability and scalability

- **Avoid Vendor Lock-in:** Works across providers
- **Language Agnostic:** Works with any language or framework
- **Run any workload:** Microservices, batch jobs, ML, analytics, etc.





Why Kubernetes Is Called Declarative

Desired State: You tell what you want

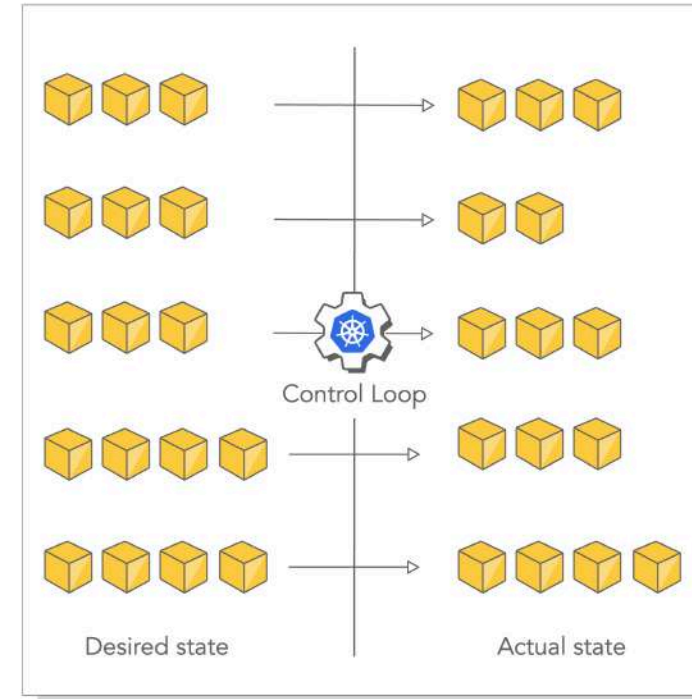
- Example: “Run 3 instances of Microservice A”

Actual State: Current state of the cluster

- Example: “2 instances of Microservice A are running in the cluster”

Control Loop: Reconciliation Loop

- **Continuous Comparison:** Is actual state $\neq$ desired state?
- **If there is a difference:** Automatically takes action to fix it





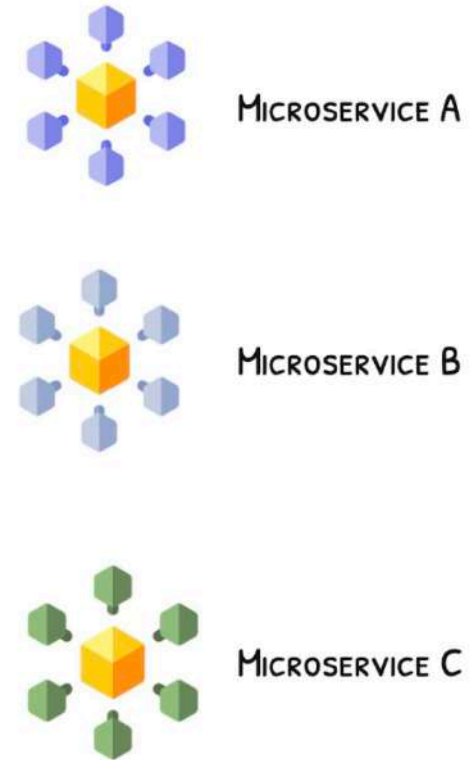
Summary - Getting Started with Kubernetes

Large Number of Services: Strong need for automation in microservices architectures

Architecture: Master Nodes (Control Plane) + Worker Nodes

Declarative: Define desired state, Kubernetes makes it happen (and maintains the desired state)

Flexible: Runs anything, anywhere with zero vendor lock-in





in28minutes

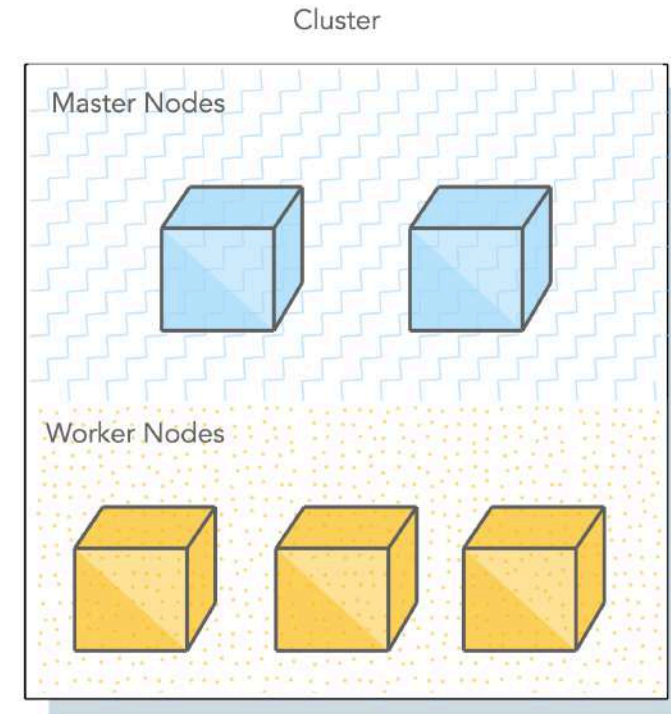
Getting Started with Managed Kubernetes



Managing Kubernetes is NOT Easy

Kubernetes Clusters: Not easy to setup and manage

- **Infrastructure Setup in Data Centers:** Servers + Networking + Security
- **Kubernetes Installations:** Master Nodes + Worker Nodes
- **Deploying Your Applications:** Deploy and manage applications
- **Kubernetes Configuration:** Scaling of Nodes, networking, storage
- **Observability & Troubleshooting:** Logs, metrics, alerts and cluster health





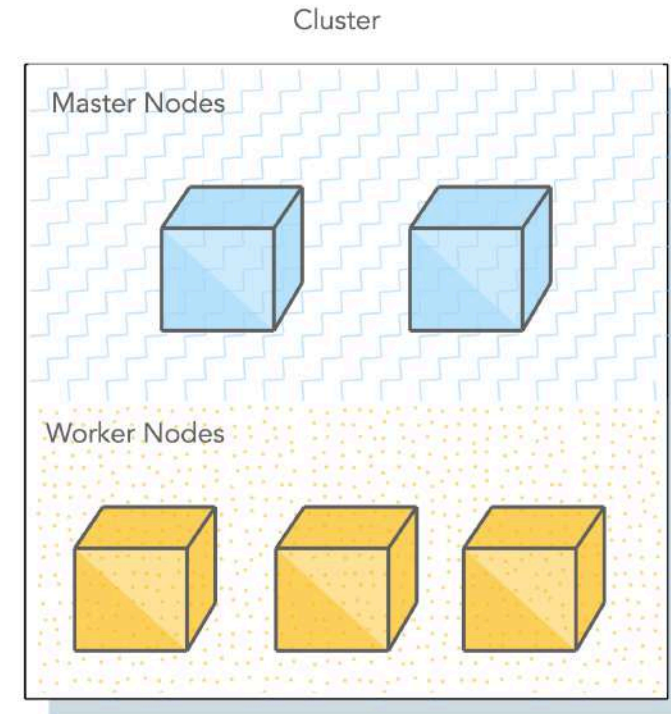
What is Self Managed Kubernetes (On-premises)?

Self-Managed: You manage everything

- **Infrastructure:** Setup physical servers in data centers
- **Kubernetes Setup:** Install control plane and worker nodes
- **Kubernetes Configuration:** Scaling of Nodes, etc.
- **Workloads:** Deploy and manage applications
- **Upgrades:** Maintain K8S versions and patch clusters
- **Observability & Troubleshooting:** Logs, metrics, alerts and cluster health

Pro: Full control

Con: High operational overhead





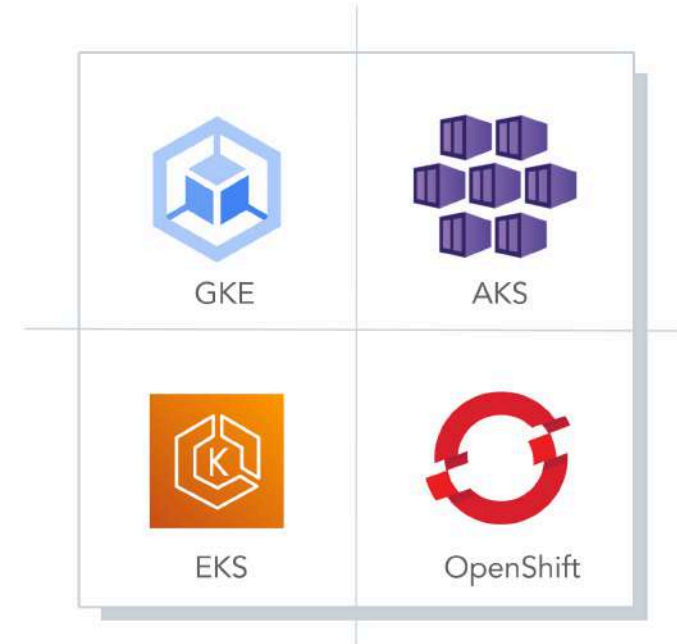
What is Cloud Managed Kubernetes?

Cloud Managed: Cloud provider manages infrastructure and Kubernetes control plane

- **Cloud Provider manages:** Infrastructure, Kubernetes Installation, Control plane, availability, observability and upgrades
- **You manage:** Kubernetes configuration (Node type, scaling, etc.) and workloads

Benefit: Reduced operational effort

- **Avoid undifferentiated heavy lifting:** Focus on your applications
- **Built-in features:** Built-in observability and security integrations





Managed Kubernetes – How It Solves Real Problems

Challenge	How Managed Kubernetes Helps
Setting up infrastructure in data centers	Cloud provider provisions and manages compute, networking and storage
Installing Kubernetes	Control plane, storage and networking are preconfigured
Node maintenance	Nodes are auto-patched, upgraded and auto-scaled
Observability setup	Integrated dashboards and alerts for logs, metrics and traces
High ops overhead	Teams focus on applications instead of infrastructure work
Cost management	Autoscaling ensures you pay only for resources you actually need



Managed Kubernetes Services in the Cloud

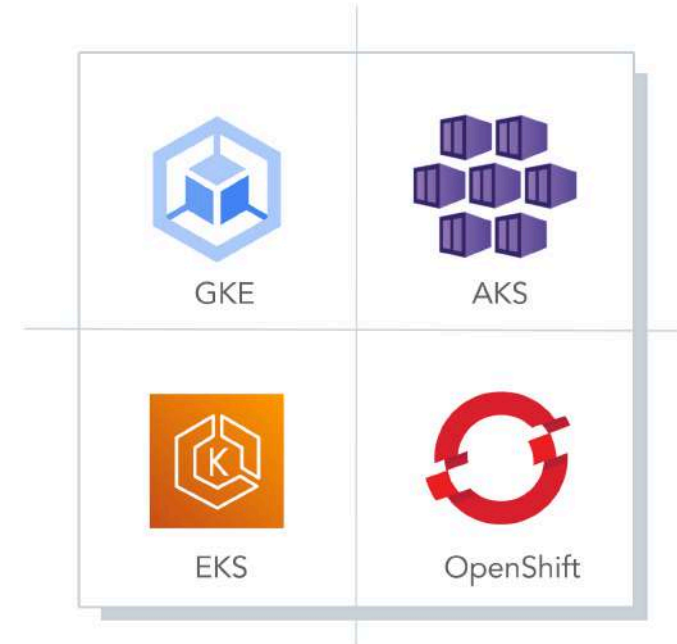
GKE: Google Kubernetes Engine - Managed Kubernetes from Google Cloud

EKS: Amazon Elastic Kubernetes Service - Managed Kubernetes from AWS

AKS: Azure Kubernetes Service - Managed Kubernetes from Microsoft Azure

OpenShift: Enterprise Kubernetes platform from Red Hat

- **Available as Managed Service:** Offered on multiple clouds





in28minutes

Getting Started with

Google Kubernetes Engine



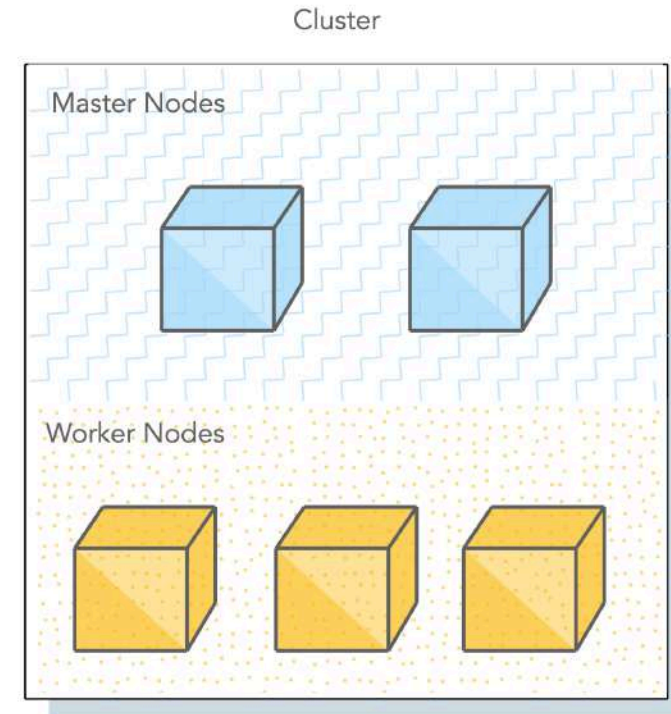
What Is Google Kubernetes Engine (GKE)?

Full Managed Kubernetes: Fully managed Kubernetes service from Google Cloud

- **Purpose:** Run, scale, and manage containerized applications with minimal effort
- **Design:** Based on Google's operational experience with large-scale container workloads

Modes:

- **Standard Mode:** Customer decides the number and type of worker nodes
- **Autopilot Mode:** Google manages everything in the cluster





GKE Standard vs GKE Autopilot – A Quick Comparison

Feature/Task	GKE Standard	GKE Autopilot
Setting up infra. - servers etc.	Google Cloud's Responsibility	Google Cloud's Responsibility
Installing Kubernetes	Google Cloud's Responsibility	Google Cloud's Responsibility
Setting up Control Plane & Worker Nodes	Google Cloud's Responsibility	Google Cloud's Responsibility
Configuration of Worker Nodes (Type, Scaling)	Customer's Responsibility	Google Cloud's Responsibility
Workloads	Customer's Responsibility	Customer's Responsibility
Key Cost Factor	Nodes (You pay for entire node)	CPU/Memory requests
Operational effort	Higher	Very low
Best for	Custom infra & special workloads	Most other workloads



Using GKE Standard Mode – An Overview

1: Create Cluster

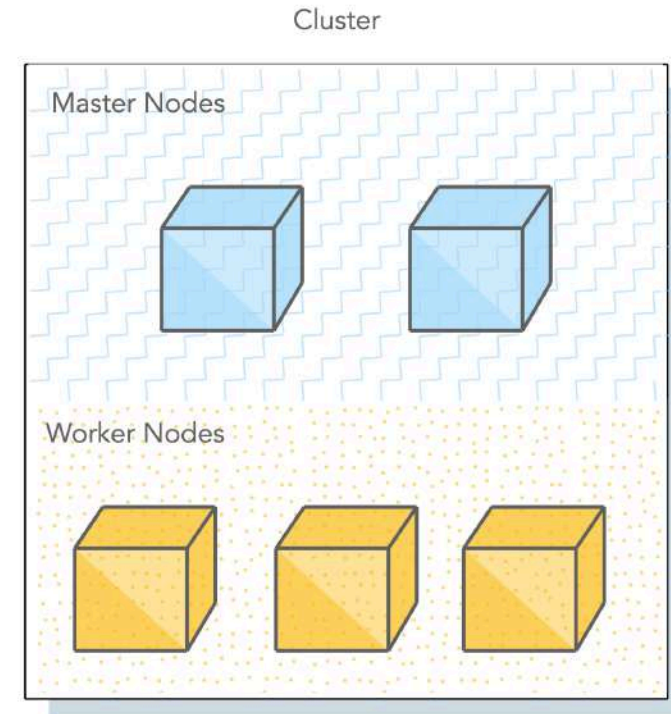
- A: Choose Region and/or Zone
- B: Configure nodes (no of nodes, machine type, size)

2: Deploy Applications

3: Manage Applications

- A: Manage Nodes Manually

Key Cost Factor: Nodes (You pay for entire node)





Using GKE AutoPilot Mode – An Overview

1: Create Cluster (No need to configure nodes)

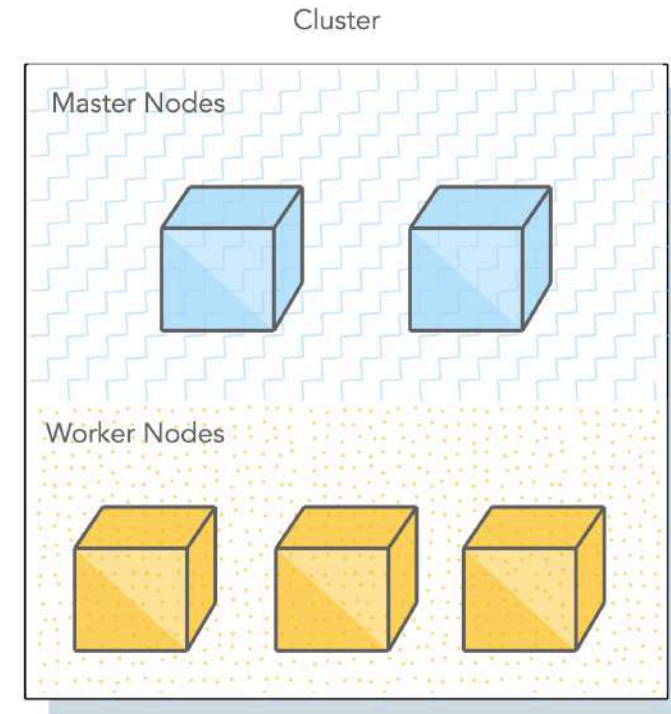
- A: Choose Region
- B: Take Defaults

2: Deploy Applications

3: Manage Applications

- No need to manage nodes

Key Cost Factor: CPU/Memory requests





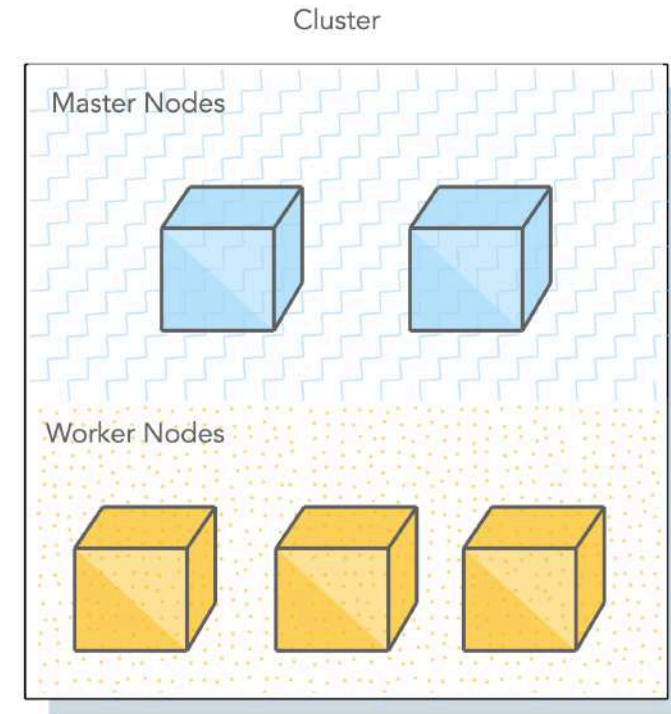
GKE Autopilot Mode – What It Is

Operating model: A Kubernetes mode where Google manages everything in the cluster for you

Nodes: Automatically created, sized, secured, and repaired

Billing: Primarily based on requested CPU and memory

GOAL: Reduce Kubernetes operational effort to ZERO!





GKE Autopilot Mode – Why Teams Use It

Less management: No node configuration, patching, or right-sizing

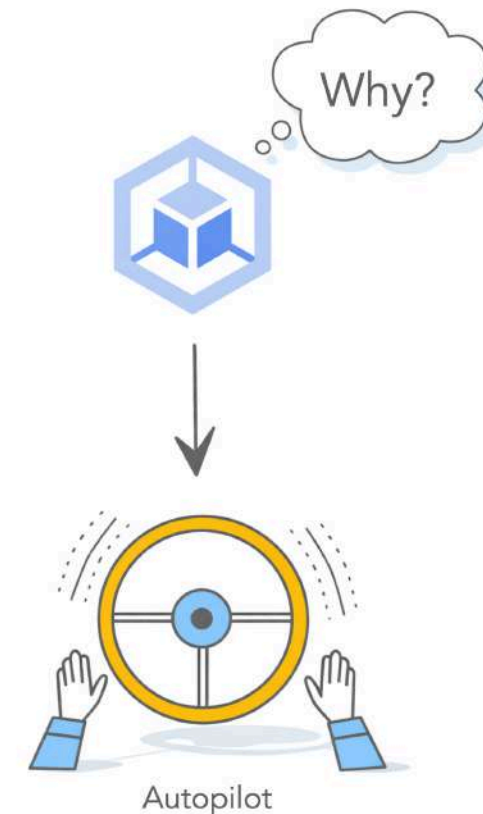
- Upgrades are automatic and safe
- Security best practices are enforced by default

Higher efficiency: Automatic resource scaling

- No idle VM cost because billing is based on requests

Better reliability: Resilience built into the platform

- Automatic monitoring and repairs





GKE Standard – Why Teams Still Use It

Custom Infrastructure Control: Full control over node configuration

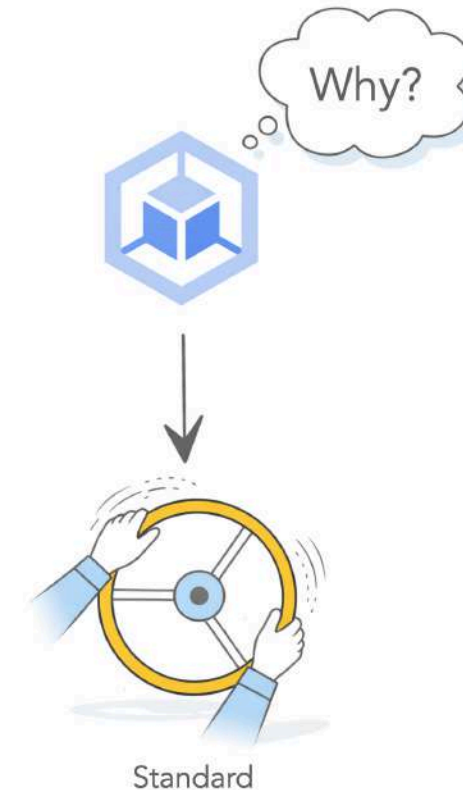
- Choose machine types, disk types, etc.
- Attach GPUs or TPUs for ML and HPC workloads

Node-Level Access: For direct control

- SSH access to nodes, Install custom agents

Run Hybrid Clusters: On-prem integration

Spot VMs: Run fault-tolerant, non-immediate workloads on Spot VMs





Kubernetes - A Journey - Getting Started

Step	What You Do	Example Command
1	Create a Cluster	<code>gcloud container clusters create my-cluster</code>
2	Log in to Cloud Shell	Open Cloud Shell in the console
3	Connect to the Cluster	<code>gcloud container clusters get-credentials my-cluster --zone us-central1-a --project solid-course-258105</code>
4	Deploy your microservice	<code>kubectl create deployment hello-world-rest-api --image=in28min/hello-world-rest-api:0.0.1.RELEASE</code>
	Expose as a service	<code>kubectl expose deployment hello-world-rest-api --type=LoadBalancer --port=8080</code>
5	Increase instances	<code>kubectl scale deployment hello-world-rest-api --replicas=2</code>
6	Enable auto-scaling for the microservice (HPA)	<code>kubectl autoscale deployment hello-world-rest-api --max=10 --cpu=70%</code>



Kubernetes - A Journey - The End!

Step	What You Do	Example Command
7	Increase number of nodes (Standard Mode Only)	<code>gcloud container clusters resize my-cluster --node-pool my-node-pool --num-nodes=5</code>
8	Enable auto-scaling for the Kubernetes cluster (Standard Mode Only)	<code>gcloud container clusters update cluster-name --enable-autoscaling --min-nodes=1 --max-nodes=10</code>
9	Delete the microservice	<code>kubectl delete service SERVICE_NAME</code> <code>kubectl delete deployment DEPLOYMENT_NAME</code>
10	Delete the cluster	<code>gcloud container clusters delete CLUSTER_NAME</code>



Key Kubernetes Logical Concepts - Simplified

Deployment: Manages your app (Defines container image and number of instances)

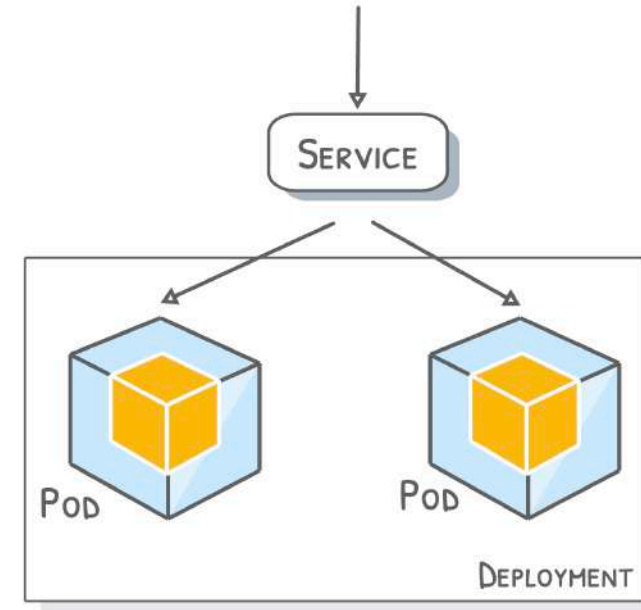
- `kubectl create deployment hello-world-rest-api --image=CONTAINER_IMAGE --replicas=2`

Pod: Created when you create a deployment

- 2 instances => 2 pods

Service: Expose pods in a deployment to outside world

- `kubectl expose deployment hello-world-rest-api --type=LoadBalancer --port=8080`





Key Kubernetes Concepts – Quick Review [1/2]

K8S Concept	Simple Explanation
-------------	--------------------

Control Plane	The “brain” of the cluster (Master Nodes)
---------------	---

Worker Nodes	Machines that run your applications
--------------	-------------------------------------

Deployment	Manages Your App - <code>kubectl create deployment hello-world-rest-api --image=CONTAINER_IMAGE</code>
------------	--

Pod	Created when you create a deployment. Smallest deployable unit in Kubernetes.
-----	---

Service	Expose pods in a deployment to outside world. <code>kubectl expose deployment hello-world-rest-api --type=LoadBalancer --port=8080</code>
---------	---



Key Kubernetes Concepts – Quick Review [2/2]

K8S Concept	Simple Explanation
GKE Autopilot Mode	A Kubernetes mode where Google manages everything in the cluster for you. Nodes are automatically created, sized, secured, and repaired. You just need to worry about the workloads.
Horizontal Pod Autoscaling (HPA)	Automatically increases or decreases the number of pods based on CPU utilization, memory usage, or custom metrics
Cluster Autoscaling	Automatically adds or removes worker nodes based on load



Google Kubernetes Engine - Introduction - Scenarios

Scenario	Solution
You want to keep your costs low	Consider AutoPilot mode If using Standard mode - Consider Spot VMs & Committed-use discounts. Choose right machine type to fit your workload type.
You want an efficient, completely auto scaling GKE solution	Configure Horizontal Pod Autoscaler for deployments and Cluster Autoscaler for node pools (or go for GKE AutoPilot Mode)
The smallest unit in Kubernetes, running your microservice	Pod
A stable way to expose pods in a deployment to outside world	Service
A Kubernetes mode where Google manages everything in the cluster	GKE Autopilot Mode



in28minutes

Getting Started with Artificial Intelligence

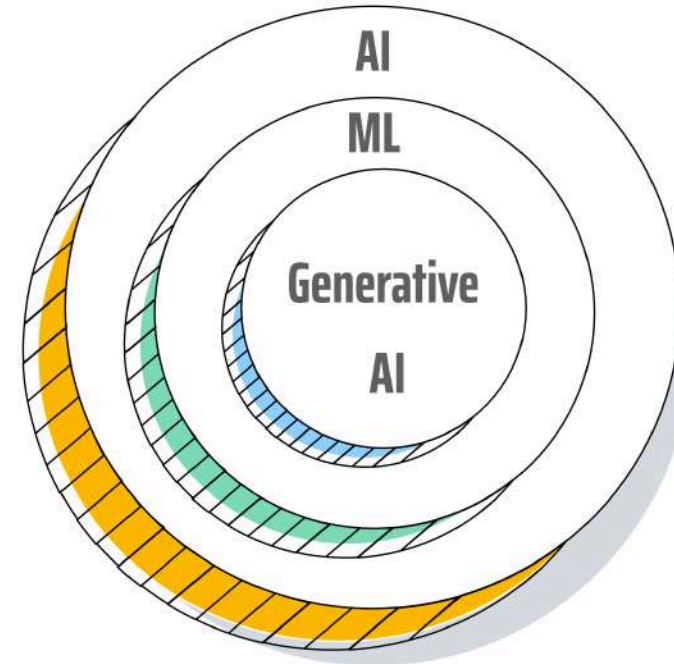
Section Overview - Introduction to AI, ML and Generative AI



Focus: What will we learn?

- What is Artificial Intelligence?
- What is Machine Learning?
- How does Machine Learning work?
- Machine Learning vs Traditional Programming
- What is Generative AI?
- How is Generative AI different from Traditional AI and ML?

Foundation: Build the core concepts needed for the rest of the course





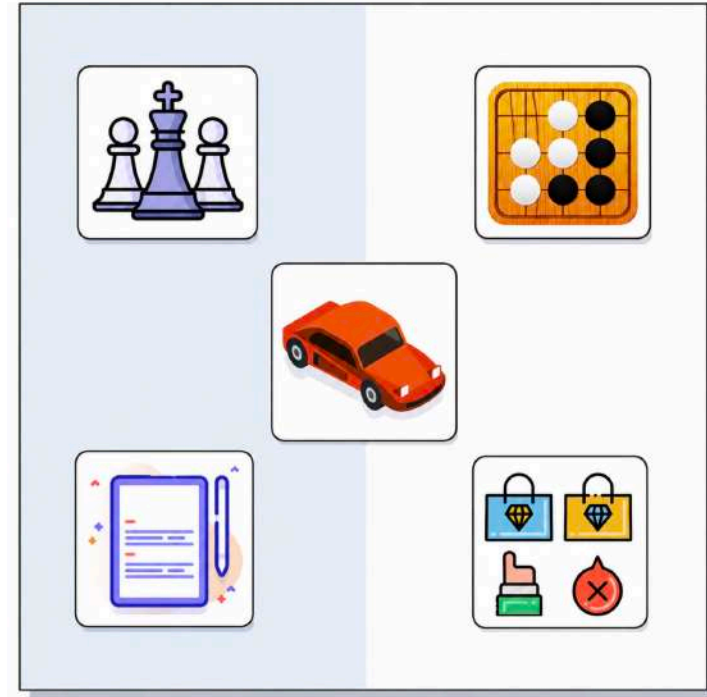
What is Artificial Intelligence?

Humans are great at:

- **Decision-Making:** Choose from multiple options
- **Visual Perception:** Recognize faces and objects
- **Language Understanding:** Understand written text
- **Speech Recognition:** Understand spoken language
- And a lot more..

Artificial Intelligence: Theory & development of computer systems able to perform tasks normally requiring human intelligence

- **Simplified Goal:** Make computers do things that normally require human intelligence





Are All AI Systems Equally Intelligent?

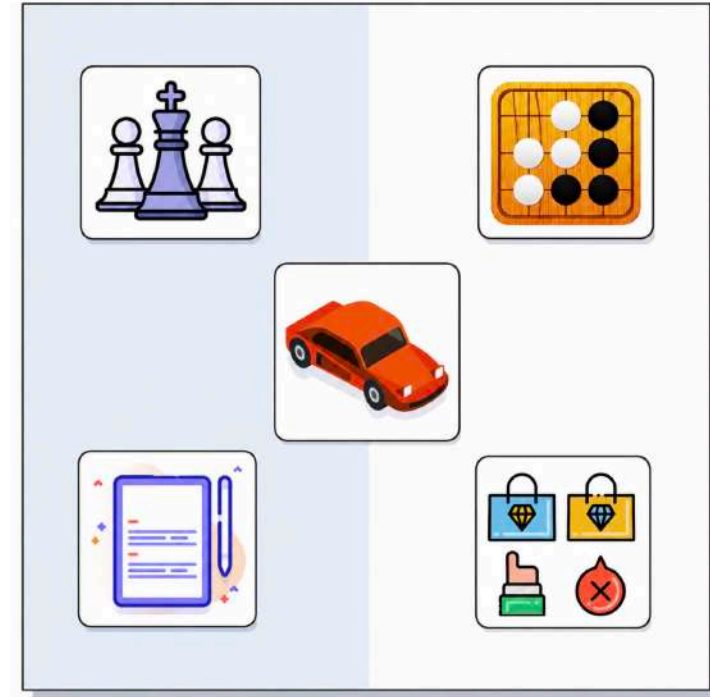
AI can have different levels of capability

Some AI systems are designed to:

- Perform **one specific task**
- Perform **many different tasks**

This leads to two broad types of AI:

- Strong AI
- Narrow AI





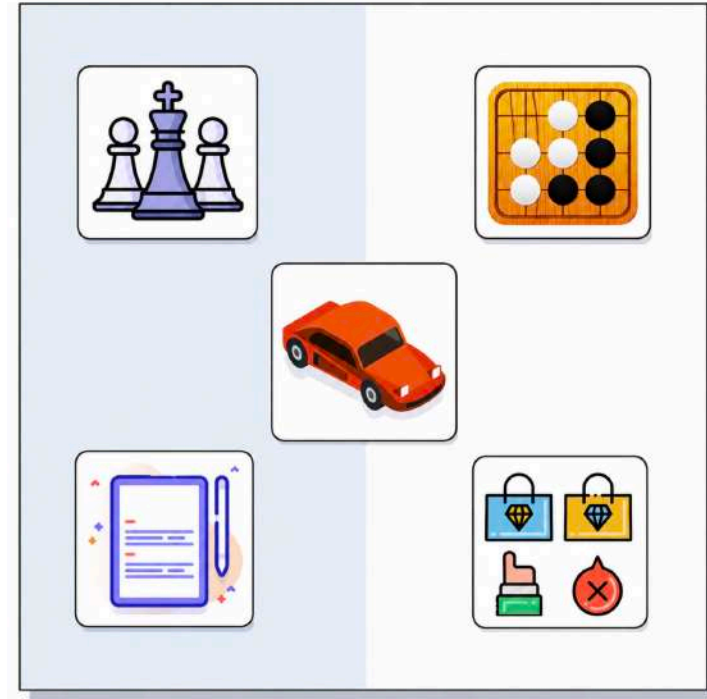
Understanding Types of AI - Strong AI

Strong AI: Goal is to create machines that are as intelligent as humans

Capabilities: Solve problems, learn, and plan for the future

- Expert at everything
- Make complex decisions
- Learns like a child

Also Known As: General AI or Artificial General Intelligence (AGI)





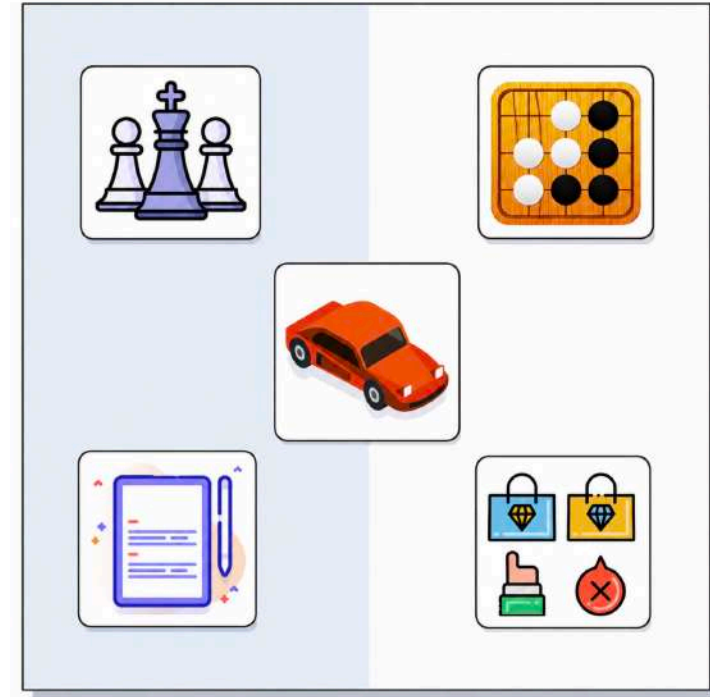
Understanding Types of AI - Narrow AI

Narrow AI: AI focused on a specific task

Examples:

- **Self-driving Car:** Can drive but cannot play chess
- **Chess App:** Can beat champions but cannot drive
- **Weather Predictor:** Knows weather but cannot write poetry

Also Known As: Weak AI





Getting Started with AI - Scenarios

Scenario	Solution
What is the Goal of AI?	Make computers do things that normally require human intelligence
Strong or Narrow AI: Build a system that can learn any game, write poetry, and drive a car like a human	Strong AI (General AI)
Build a computer system that focuses on a specific task such as self-driving, virtual assistants, or object detection	Narrow AI (or Weak AI)
The AI used in your smartphone to recognize your face	Narrow AI



Why are we talking a lot more about AI now?

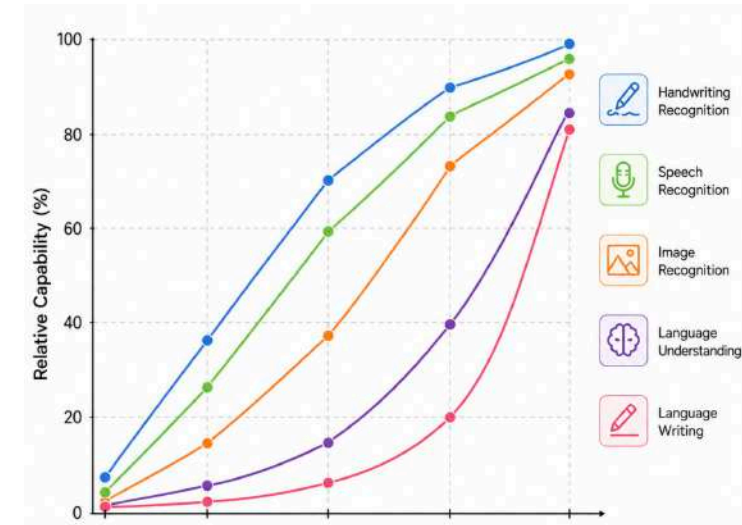
AI is Not New: Researchers have been working on AI since 1950s

Question: Why are we talking a lot more about AI now?

- **Answer:** AI capabilities improved dramatically in recent decades

Question: How can we measure progress in Artificial Intelligence?

- **Early Answer:** The **Turing Test**





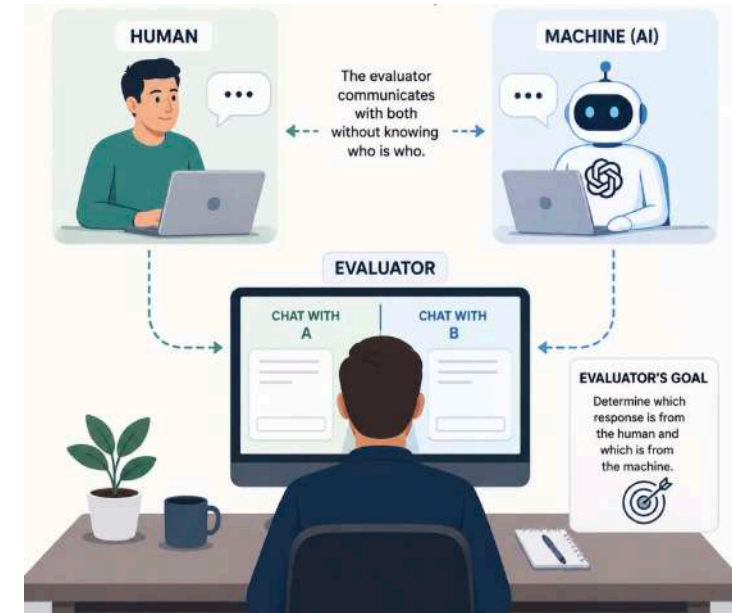
What is Turing Test?

Goal: Check if a AI system can communicate like a human

- 1: An evaluator chats with a human and a AI system
- 2: The evaluator asks questions through text
- 3: Both respond to questions
- 4: Evaluator tries to identify the machine

Early AI: AI systems struggled to imitate human conversation

Today: Advanced AI systems can often produce human-like responses





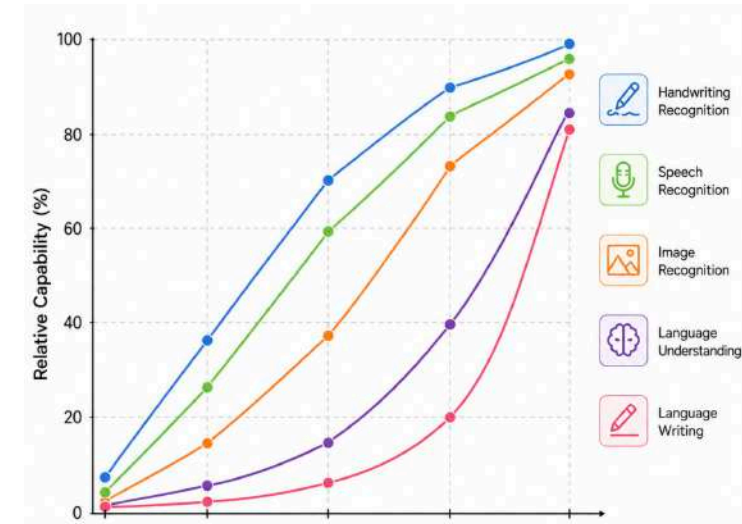
AI Has Made a Lot of Progress

AI Improved Rapidly: Across many areas

- **Image Recognition:** Recognizing object in images
- **Speech Recognition:** Convert spoken words into written text
- **Language Generation:** Large language models are capable of fluent writing
- And a number of others

How: How did this progress happen?

- Coming up!





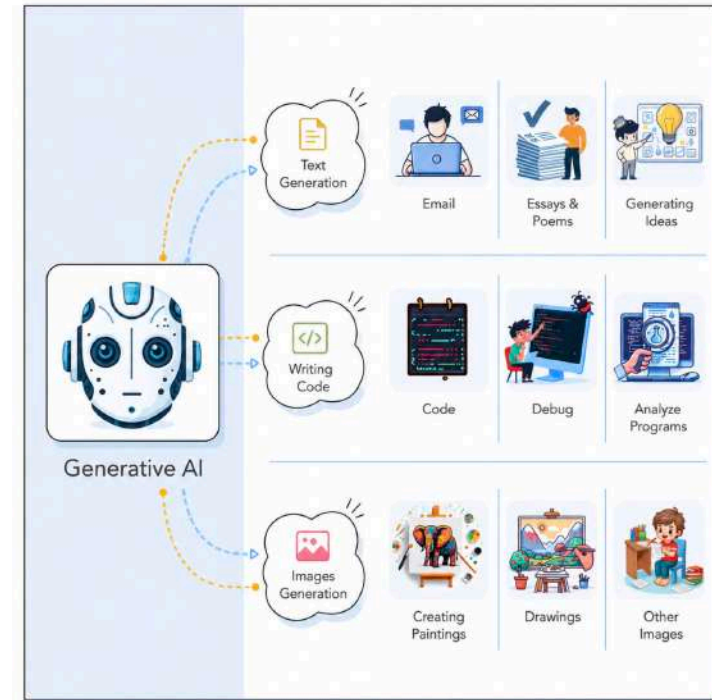
Playing with Generative AI

Checklist: Generate a list of items I need for a 15-day Everest Base Camp trek

- Update the list assuming I will stay at a tea house as part of a guided group

Multimodal: Let's create an image showing the atmosphere at a cricket match

- A vibrant, high-energy night scene at an Indian Premier League (IPL) cricket match. Focus on the electric atmosphere. Show fans wearing team jerseys, waving flags, and cheering loudly. Include digital scoreboards and a sense of celebration.



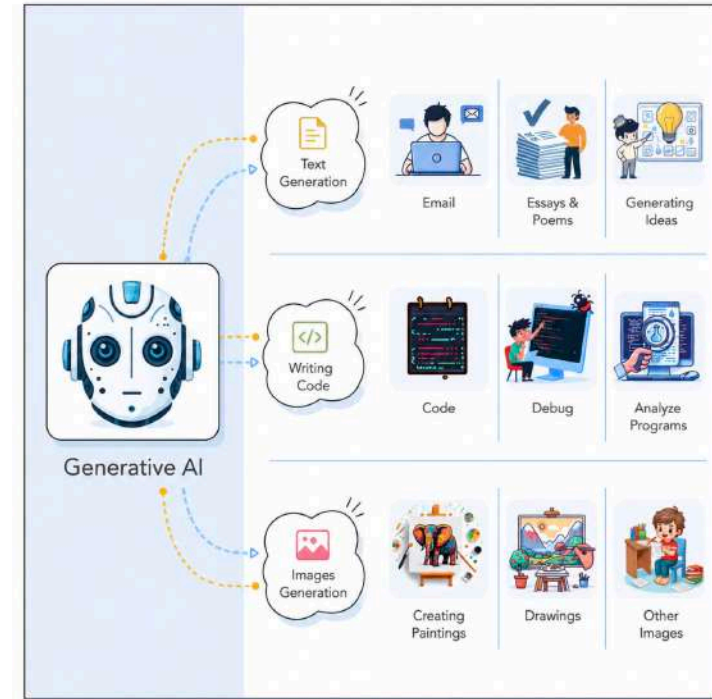


Playing with Generative AI - Observations

Powerful, Not Perfect: Can produce incorrect, biased, or inappropriate responses

- **Use with Judgment:** Outputs should be reviewed and verified
- **Still Valuable:** When used with the right expectations, Generative AI can be extremely useful

How does it work?: Let's start with Machine Learning and build our way to Generative AI





in28minutes

Getting Started with Machine Learning



How is Traditional Programming Done? [1/2]

Approach: Rule-based logic

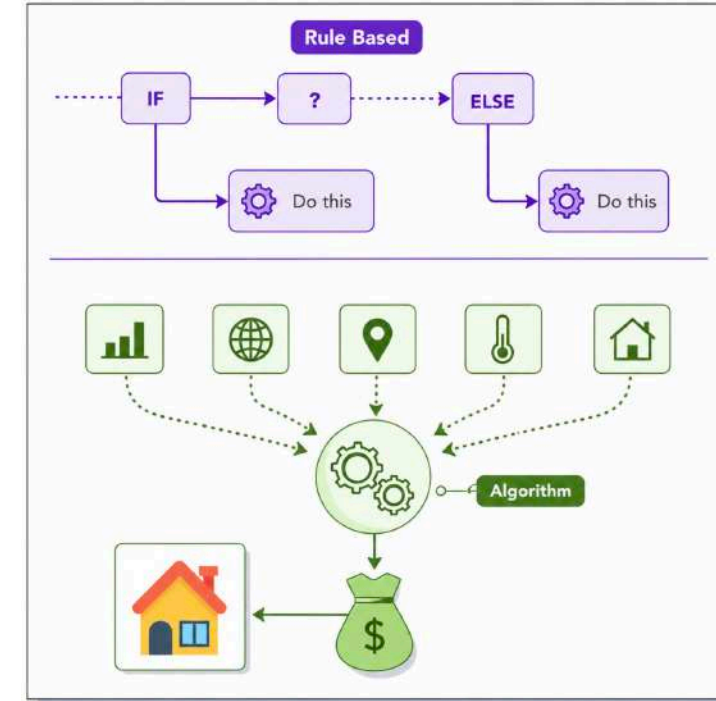
Mechanism: You explicitly tell the computer what to do

Flow:

- **Input:** Data
- **Program:** Rules such as IF this DO that
- **Output:** Result

Example: Calculating tax

- **Rule:** If income > 50k, tax is 20%





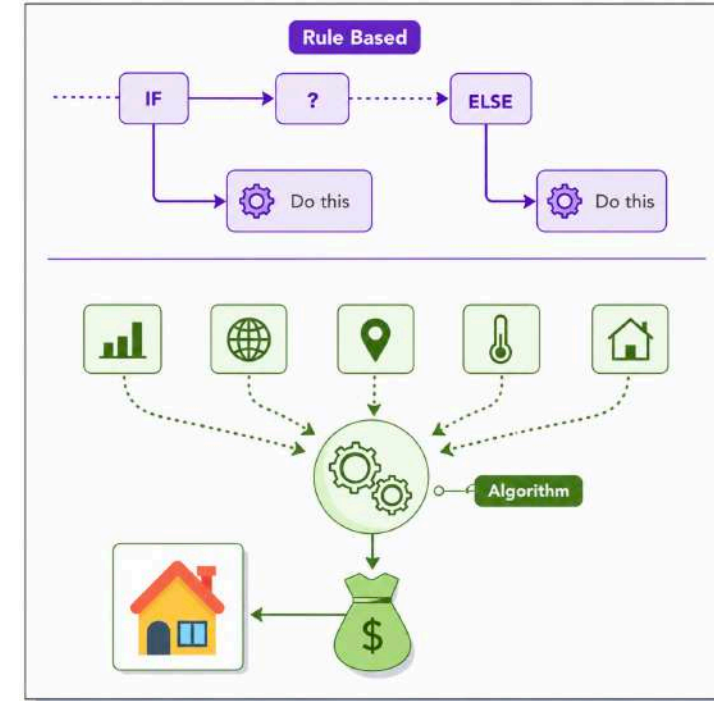
How is Traditional Programming Done? [2/2]

Example: Predicting house price

Approach: You design an algorithm that considers all key factors

- Location
- Home size
- Age
- Condition
- Market

Result: The computer does exactly what you tell it, nothing more and nothing less





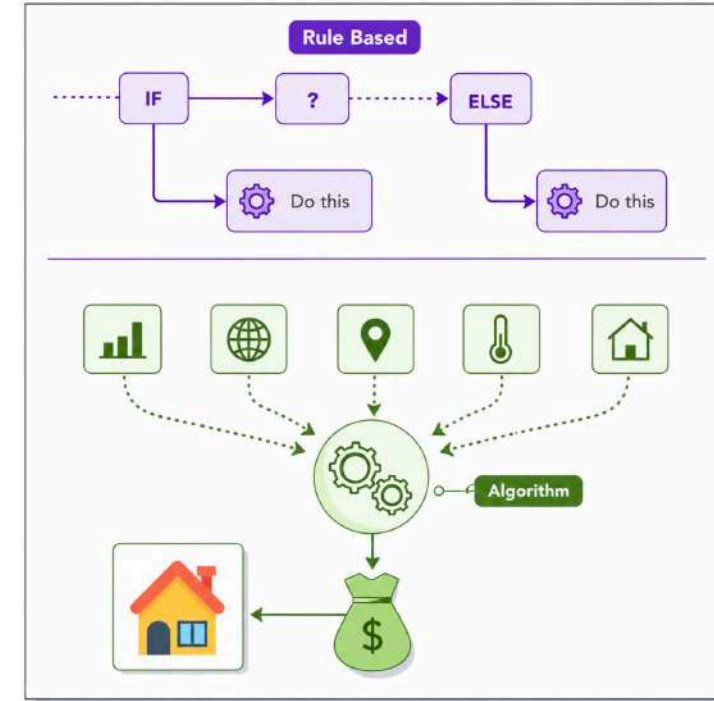
What is the Limitation of Traditional Programming?

Traditional Programming: Input --> Program (Rules or Algorithm) --> Output

Think About This: Can you write the rules to recognize a cat?

- **Example:** If pixel 1 is light and pixel 2 is dark, and so on
- **Reality:** Almost impossible to hand-code such kind of rules

Result: We hit a wall with traditional coding





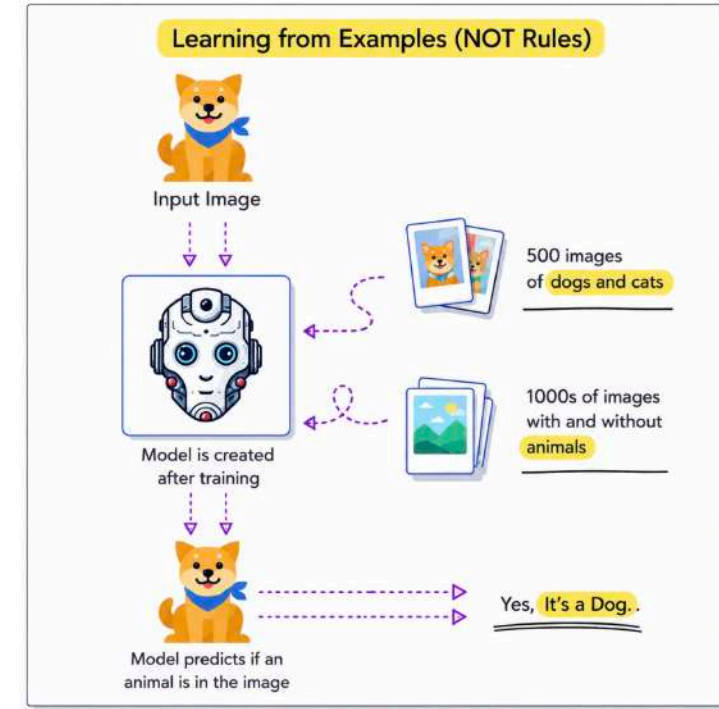
How Does Machine Learning Work?

What Happens?: ML learns from data

Machine Learning: Give examples, ML finds patterns, then predicts the outcome

Example:

- House Price Prediction: Learn from past sales data





Machine Learning - Simplified Workflow

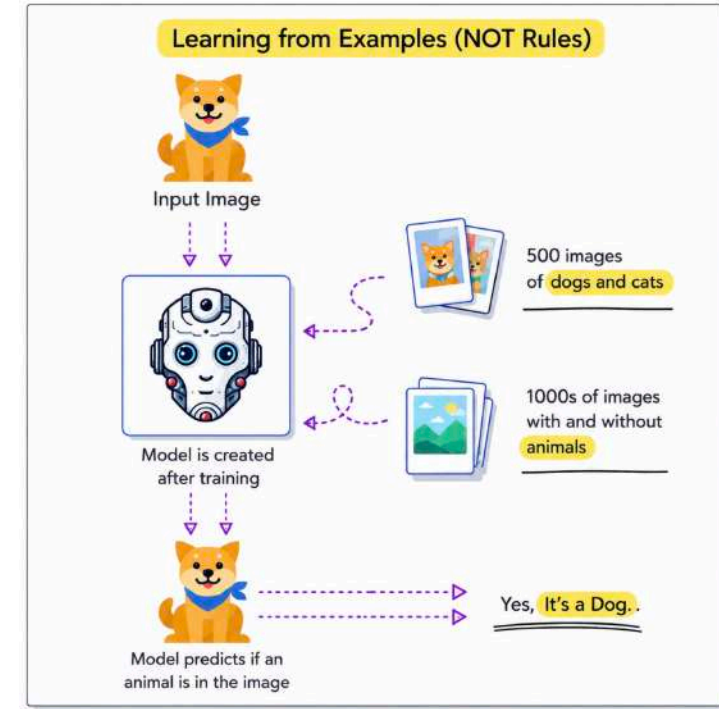
Step 1 - Obtain Data: Provide examples

- **Example:** Photos of cats, dogs and other animals with labels

Step 2 - Train Model: The system analyzes images to find patterns

- **Learned Patterns:** Shapes, edges, and textures

Step 3 - Predict: Give it a new photo and get a prediction





Traditional Programming vs Machine Learning

Traditional Programming

Programmer writes the rules (program)

Input → Program (Rules or Algorithm) → Output

Works well when rules are known

Example: Tax calculation

Example: Payroll processing

Machine Learning

Model learns the rules from data

Input + Expected Outputs → Training → Model

Works well when rules are difficult to define

Example: House price prediction

Example: Spam email detection



Machine Learning Fundamentals - Scenarios

Scenario	Solution
Developer writes logic to tell the computer what to do	Traditional Programming
System finds patterns from data that humans cannot explicitly define	Machine Learning
Category of AI that focuses on learning from data	Machine Learning
How is ML different from traditional programming?	Traditional Programming uses rules, ML uses examples



in28minutes

Getting Started with

Machine Learning Stages



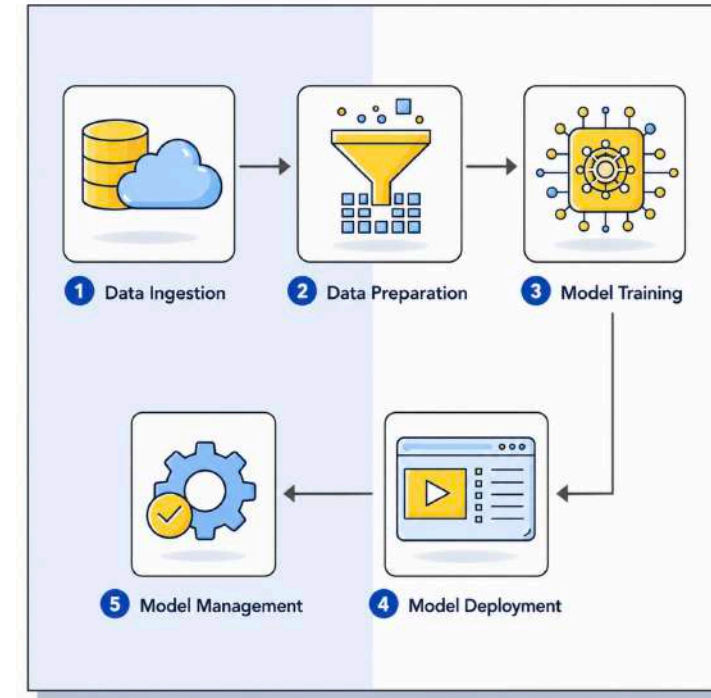
ML Lifecycle: Data Related Stages

Data Ingestion: Collect raw data from different sources

- **Example:** Streaming user activity from a mobile app
- **Example:** Pulling transaction logs from a sales system

Data Preparation: Clean and format data for training

- **Example:** Remove duplicates
- **Example:** Handle missing values in patient health data





ML Lifecycle: Model Related Stages

Model Training: Use data to train the model

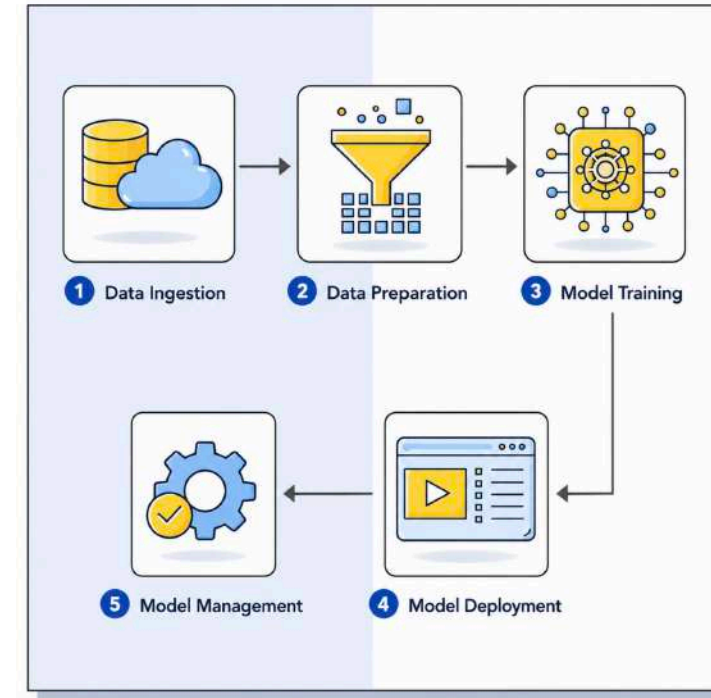
- **Example:** Train a fraud detection model with transactions
- **Example:** Train a price prediction model for used cars

Model Deployment: Make the trained model available to users

- **Example:** Deploy a chatbot in production (Inference)

Model Management: Monitor and maintain the model after deployment

- **Example:** Update the model as new financial data comes in (Re-Training)





ML Lifecycle Recap: End-to-End Example

Use Case: ML model to detect spam in email

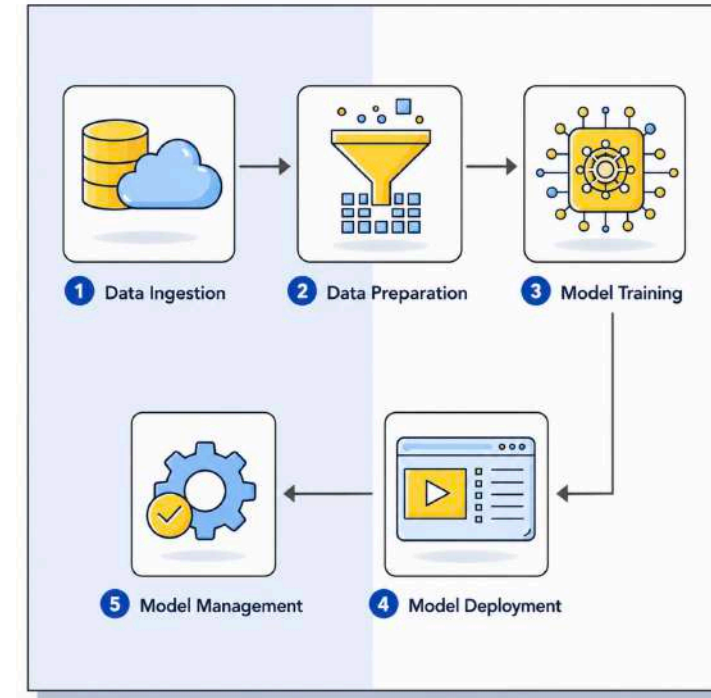
1: Collect email history

2: Prepare labels

3: Train with examples

4: Deploy into email platform

5: Manage performance





Identify the Stage in ML Lifecycle - 1

Activity	Stage
Pulling product transaction logs from a retail store's log store	Data Ingestion
Streaming real-time user activity from a mobile app	Data Ingestion
Removing duplicate customer records from the training dataset	Data Preparation
Filling in missing values in patient data using averages	Data Preparation
Labeling emails as spam or not spam for training	Data Preparation
Using thousands of labeled housing records to teach a model to predict price	Model Training
Training a model to detect credit card fraud using past transaction data	Model Training



Identify the Stage in ML Lifecycle - 2

Activity	Stage
Making your trained model available via an API to a web application	Model Deployment
Deploying an image classification model into a mobile app	Model Deployment
Monitoring how accurate your chatbot is after deployment	Model Management
Updating your model with recent data after noticing performance drop	Model Management
Creating a dashboard that displays model inference accuracy trends	Model Management



in28minutes

Getting Started with Generative AI



Generative AI - Generates New Content

Goal: Generate New Content

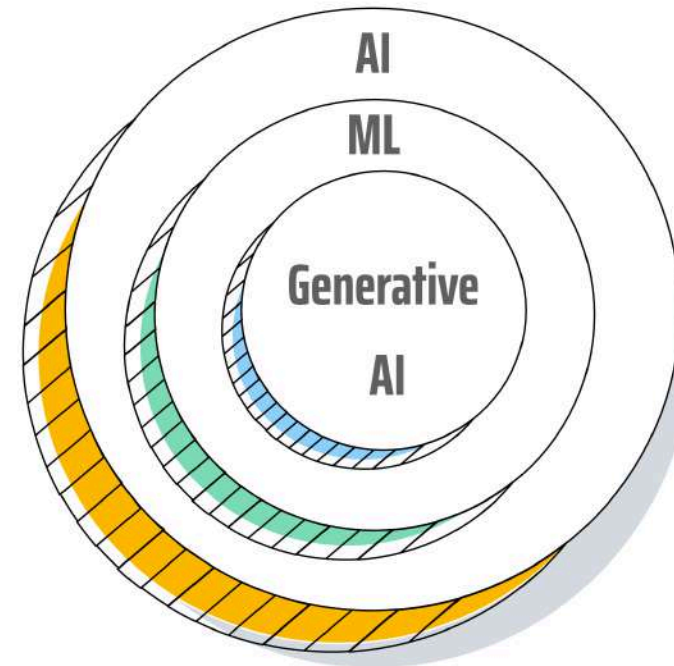
- Instead of making predictions, Generative AI focuses on creating new content

Text Generation: Writing e-mails, essays & poems. Generating ideas.

Code Generation: Generate code snippets. Create unit tests.

Media Creation: Design images. Create instructional videos.

And a lot more features!



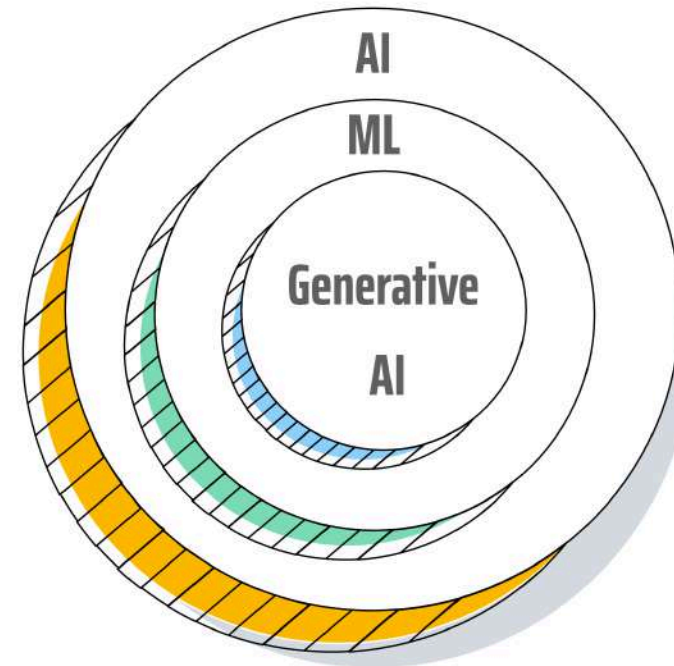


Generative AI - How is it different?

Artificial intelligence (AI): Create machines that simulate human-like intelligence and behavior

Machine learning (ML): A subset of AI where machines learn from data

Generative AI: AI that learns patterns from data to **generate new content**





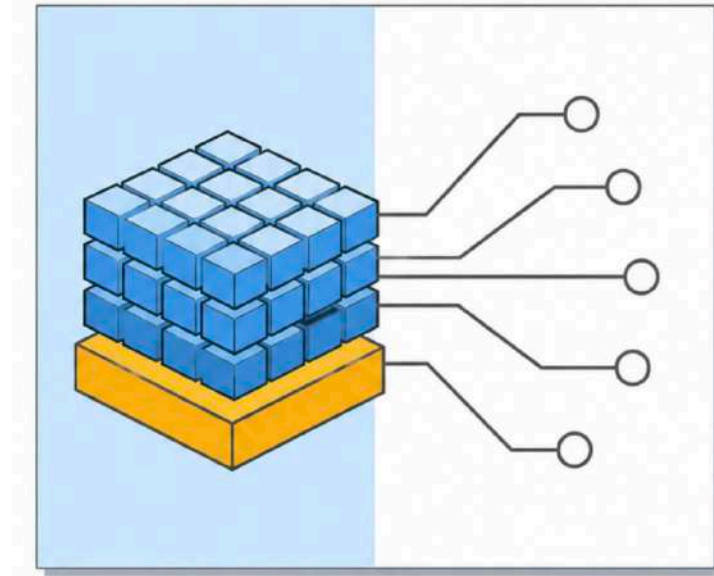
Foundation of Generative AI - The Foundation Models

Foundation Model: The core model powering many generative AI applications

- **One Model, Many Tasks:** Answer questions, summarize, generate code, create content
- **Examples:** GPT, Gemini, Claude, Llama

Pre-trained: Trained on massive amounts of text, code, images, and other data

- **Expensive to Train:** Requires significant compute, time, and data
- **Easy to Reuse:** Build applications without training a model from scratch



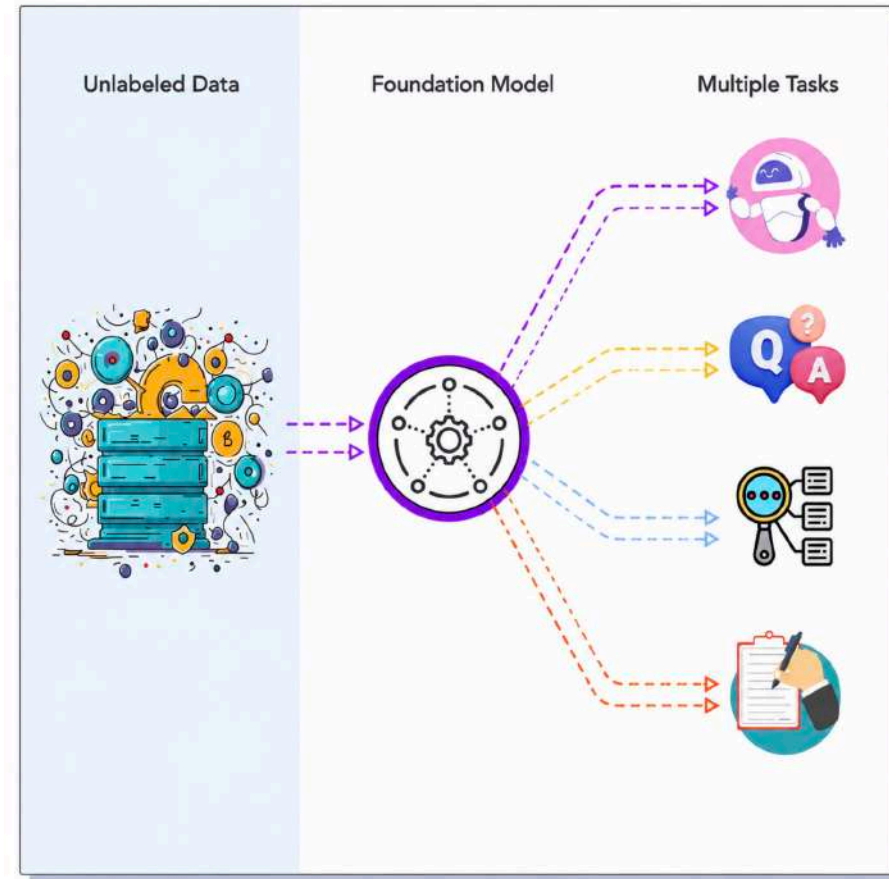


Training Foundation Models is Complex

Complex Training Process: Training is computationally intensive and complex

- **Massive Datasets:** Books, code, web pages, images, captions, and more
- **Specialized Hardware:** GPUs, TPUs, distributed storage, and parallel processing
- **Takes Time:** Training runs for weeks or months

Outcome: Broad general knowledge - understands how humans write, speak and code





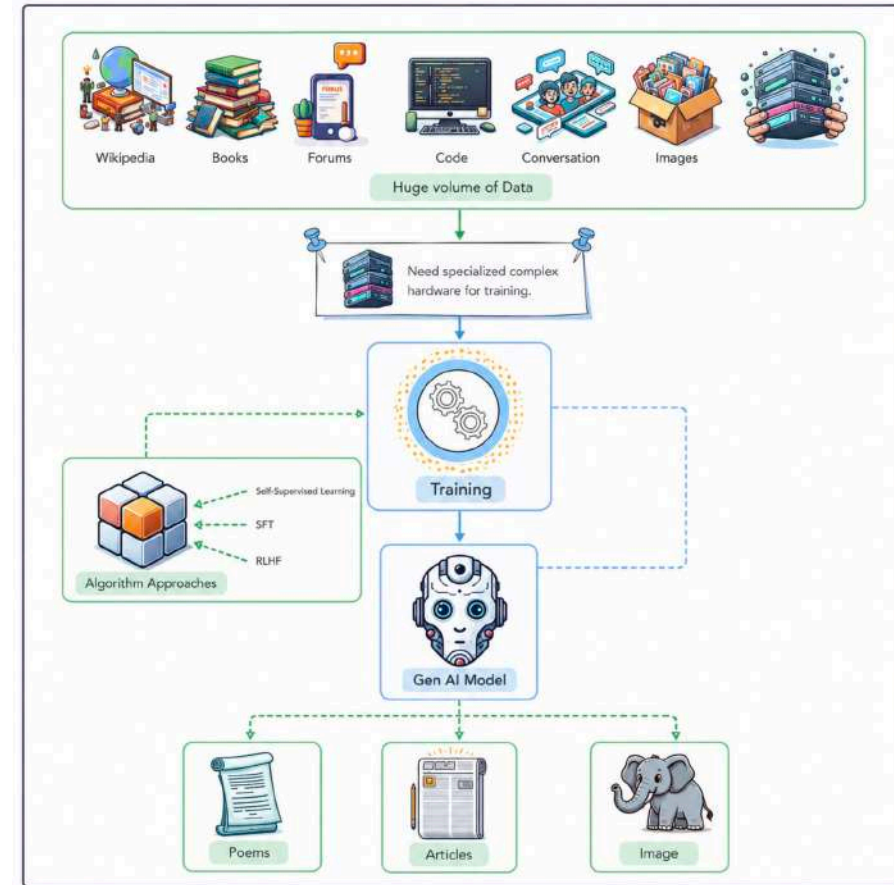
Foundation Models Are Multi Task Models

Traditional ML Models:

- Needed task specific training
- Multiple tasks => Multiple trainings => Multiple models

Foundation Models: Multi Task Models

- Trained once (called pre-training)
- Same model can be used for multiple tasks
 - Chatbot, Classification, Summarization, etc.
- Some models are multi modal as well: Text, video, audio, image...





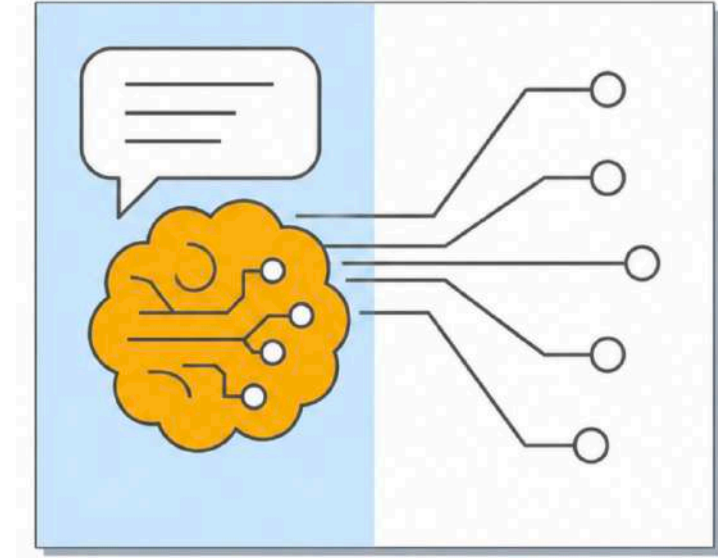
Text Foundation Models are called Large Language Models

Trained on **huge volumes** of diverse text data

- Books, articles, websites, code, and more

Learn to **predict the next token** in a sequence

- Example: "The cat sat on the ..."
- Model gives probabilities:
 - "mat" (40%), "table" (20%), "chair" (20%), "moon" (10%)
- It usually picks the most likely token — but we can tune settings to make it more creative
 - e.g., using the temperature parameter



Use cases: Chat, summarization, translation, Q&A, ..



Diffusion Models: Turning Noise into Art

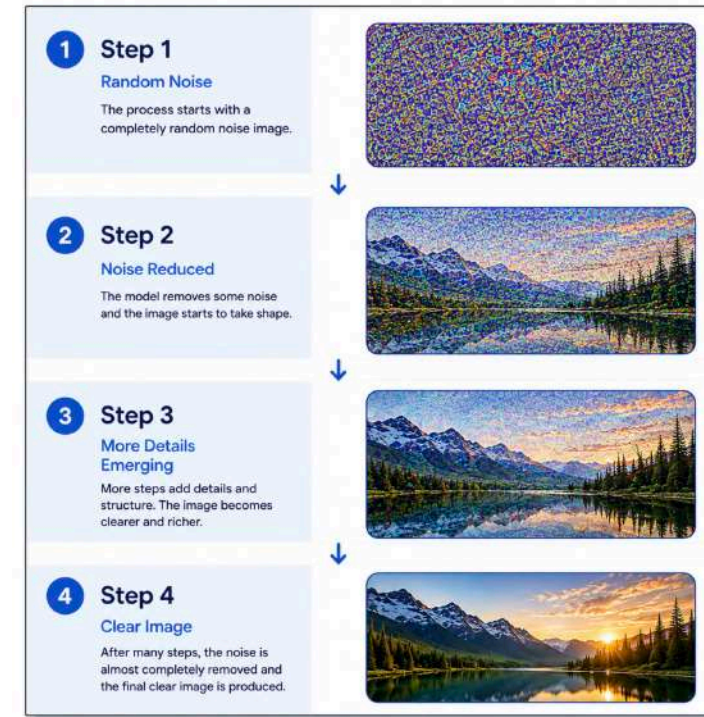
Diffusion models: Great at generating images, audio, and video

How they work:

- Start with **random noise**
- Iteratively **refine it** step-by-step
- Result: **Structured, realistic output**

Used for:

- Creating AI art and realistic pictures from text prompts
- Generating life-like human voices or music
- Producing short AI-generated videos





Foundation Models - Scenarios

Scenario	Solution
Models trained on large, diverse datasets across domains. Adaptable to a wide variety of tasks.	Foundation Models
Text foundation models trained on huge volumes of diverse text data.	Large Language Models
Models that generate images, audio, or video starting with random noise and refining step-by-step	Diffusion Models



in28minutes

Getting Started with AI Agents



Why Do We Need AI Agents?

Models are great at understanding, reasoning, and generating outputs

- **Real-world tasks** often require more than generating a response:
 - **Decide** what needs to happen next
 - **Interact** with external systems
 - **Take Actions** based on results
 - **Complete Tasks** involving multiple steps

Question: How do we go one level above models and focus on achieving a specific goal?

- **Answer:** AI Agents





What Are AI Agents?

AI Agent: A system that uses AI models to pursue a goal

- **Observe:** Understand user input and context
- **Reason:** Determine what needs to be done
- **Use Tools:** Access data, invoke API and interact with other applications

Agentic AI: Moves beyond one-off prompts

- Uses models to pursue goals (Can adapt across multiple steps)

Key Idea: Move beyond generating responses to getting things done





AI Agents - An Example

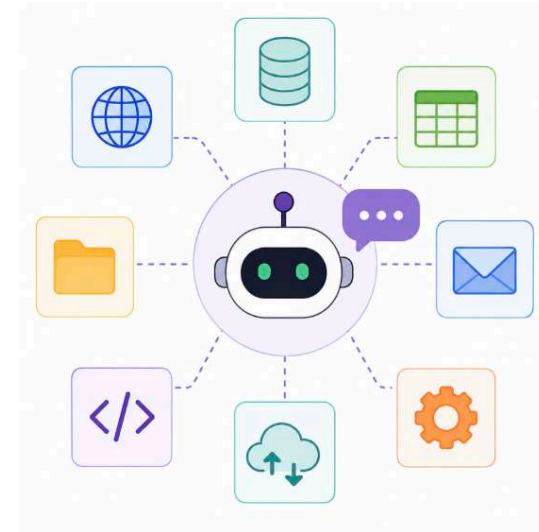
User Request: "Summarize report and send to team"

A model can:

- **Generate:** Summarize the report

An agent can:

- **Understand:** Identify the user's goal
- **Generate:** Create the summary using a model
- **Find Information:** Identify the team members
- **Use Tools:** Connect to email
 - **Take Action:** Send the summary
- **Result:** The task is completed





Use Cases for AI Agents [1/2]

Customer Agents: Assist customers and perform customer-facing tasks

- **Example:** Answer account questions and help resolve issues

Employee Agents: Help employees complete internal tasks

- **Example:** Answer HR questions and submit leave requests

Creative Agents: Assist with content creation workflows

- **Example:** Create campaign copy and visuals





Use Cases for AI Agents [2/2]

Data Agents: Analyze data and generate insights

- **Example:** Analyze sales data and summarize important trends

Coding Agents: Assist with software development tasks

- **Example:** Generate code, review, and fix issues

Security Agents: Assist with detecting and responding to threats

- **Example:** Investigate suspicious activity and initiate remediation





Agent Pattern: Conversational Agents

Goal: Interact with users through natural language

Flow:

- **1: Receive Request:** User requests an action
- **2: Understand:** Agent understands the request
- **3: Use Tools:** Fetch information or perform actions
- **4: Respond:** Return the result to the user

Examples:

- **Healthcare:** "Book an appointment with Dr. XYZ tomorrow"
- **Retail:** "Where is my order?"





Agent Pattern: Workflow Agents

Goal: Execute multi-step processes to achieve a business goal

- **1: Trigger:** User or system initiates a task
- **2: Determine Steps:** Determine what to do
- **3: Execute:** Use tools and external systems
- **4: Adapt:** Determine subsequent steps
- **5: Complete:** Deliver the final result

Examples:

- **HR On-boarding Agent:** Create account → assign equipment → schedule orientation
- **IT Operations Agent:** Diagnose issue → restart service → create incident → notify team





Getting Started with AI Agent Architecture [1/2]

Persona: Defines the agent's role, behavior, and communication style

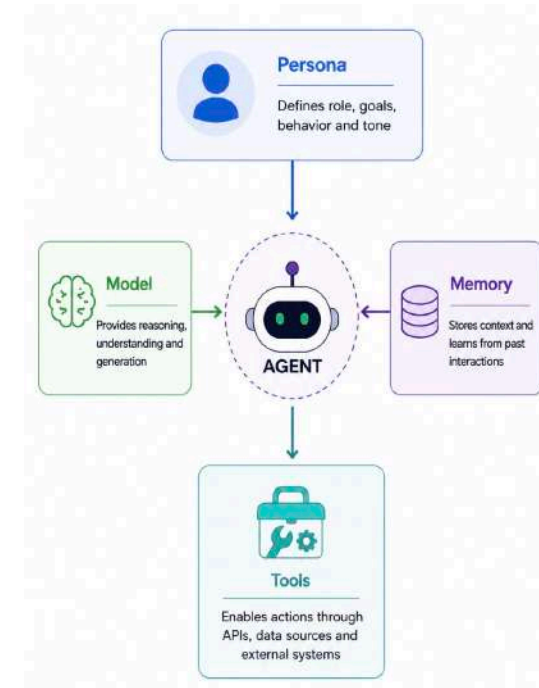
- **Defined Using:** Agent instructions or system prompts

Model (LLM): Provides the core intelligence for the agent

- **Understand:** Comprehend user requests and instructions
- **Reason:** Determine what needs to be done
- **Generate:** Create responses

Memory: Provides context across interactions

- **Short-Term:** Current conversation and task context
- **Long-Term:** Historical information and past interactions





Getting Started with AI Agent Architecture [2/2]

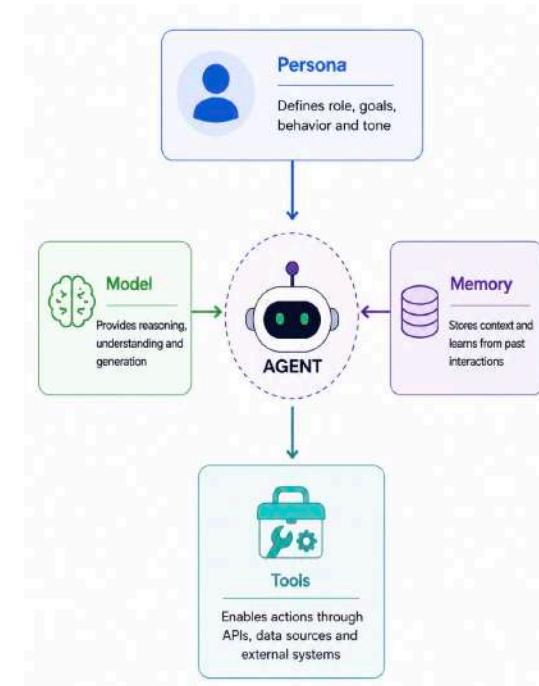
Tools: Extend what an agent can do

Knowledge Tools: Access external information

- **Examples:** Search engines, documents, databases, etc.

Action Tools: Perform tasks in external systems

- **APIs:** Connect to services such as translation or travel booking
- **Functions:** Execute application logic such as `calculate_xyz()`
- **Integrations:** Connect to calendars, email, payment systems, etc





in28minutes

Getting Started with ML in Google Cloud



1 - Pre-built AI API [1/2]

Concept: Pre-trained models ready for you!

Examples of AI API:

- **Vision:** "Is there a cat in this photo?"
- **Speech-to-Text:** "Transcribe this audio file"
- **Natural Language:** "Identify sentiment of a user review"

Best For: Standard tasks where you don't need customization





1 - Pre-built AI API [2/2]

Usecase: Derive insights from unstructured text

- Natural Language - <https://cloud.google.com/natural-language>

Usecase: Speech to Text

- Speech to Text - <https://cloud.google.com/speech-to-text>

Usecase: Convert text into speech

- Text to Speech - <https://cloud.google.com/text-to-speech>

Usecase: Extract insights from images, videos and documents

- Vision - <https://cloud.google.com/vision>



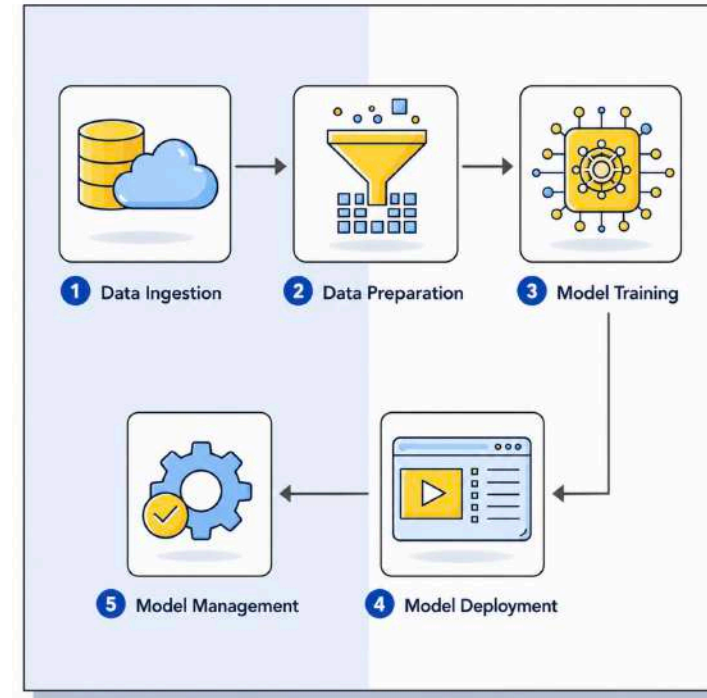


Agent Platform - The Unified AI Platform [1/2]

Agent Platform: One Platform for Machine Learning, Generative AI and Agentic AI

Train and Deploy Your Models: Automate your complete ML workflow (AutoML and Custom Training)

- **Infrastructure for Training and Inference:** Servers, GPUs, TPUs
- **Datasets:** Manage your training data
- **Model Monitoring:** Monitor your model; auto-trigger retraining if needed
- **Model Registry:** Maintain model versions



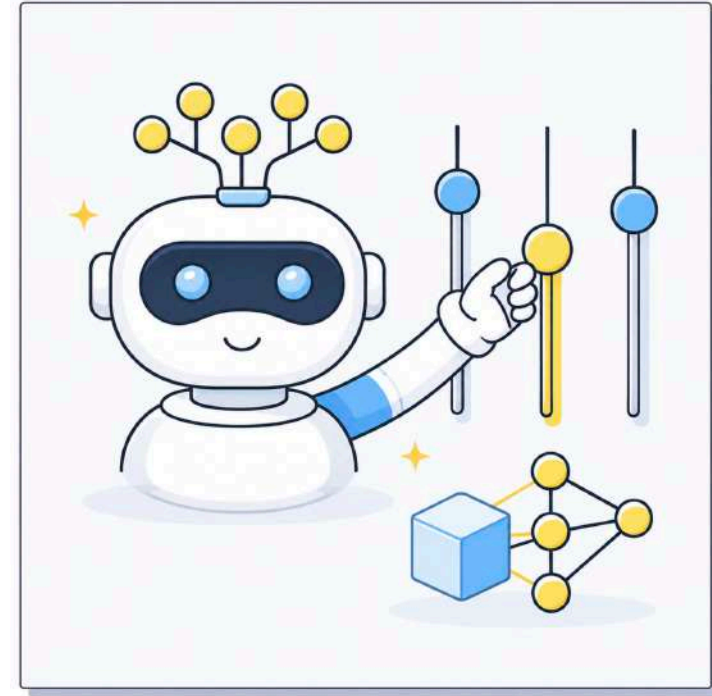


Agent Platform - The Unified AI Platform [2/2]

Model Garden: Models from Google, partners, and open-source

- *First Party Google Models:* Gemini, Imagen, Veo, ..
- *Open Models:* Gemma (Lightweight open models from Google), CodeGemma, PaliGemma, Llama, Mistral
- *Partner Models:* Claude (Anthropic), and more
- **Prompt Gallery:** Ready-to-use prompts
- **Fine-Tuning:** Fine-tune foundation models
- **Agent Platform Studio:** Rapid prototyping and testing of generative AI models

Build and Deploy Agents: Build enterprise-grade agents using generative AI models



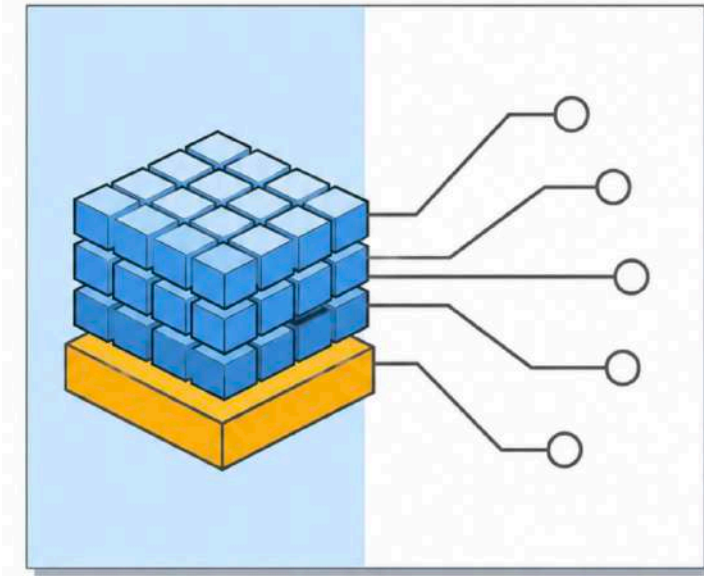


2 - Use Foundation Models via API

What: Access powerful Foundation Models via API

Agent Platform - Model Garden: A library of models (Gemini, Llama, Mistral, etc.) you can deploy

Best For: Chatbots, Content Creation, Coding help, etc.





3 - Fine-Tune Foundation Models

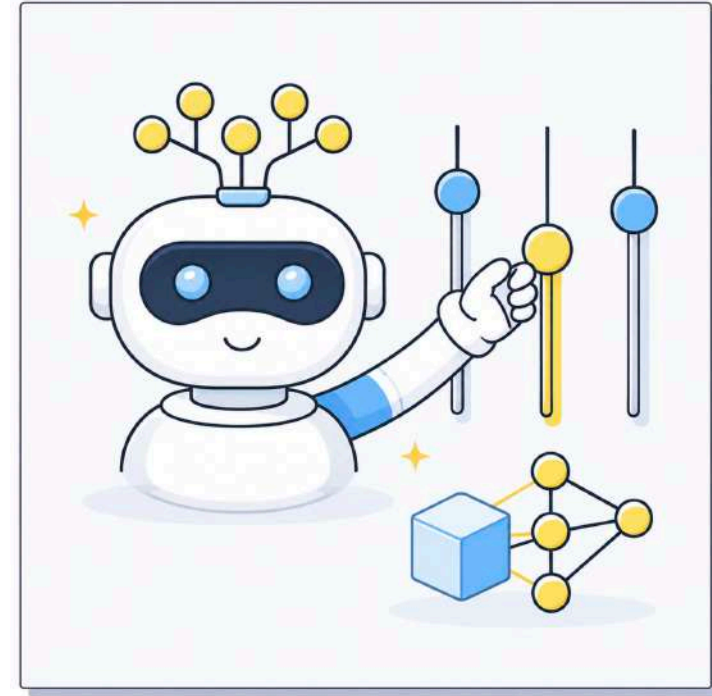
Scenario: Foundation Models do NOT know your business communication style

Solution: Agent Platform - Model Garden - Fine Tuning

- Take a foundation model and "teach" it your style
- Feed it your own data: Support emails or legal docs

Result: A version of Foundation Model that speaks "Your Language"

Best For: Domain-specific or Enterprise-specific customization





4 - AutoML - Train Without Code

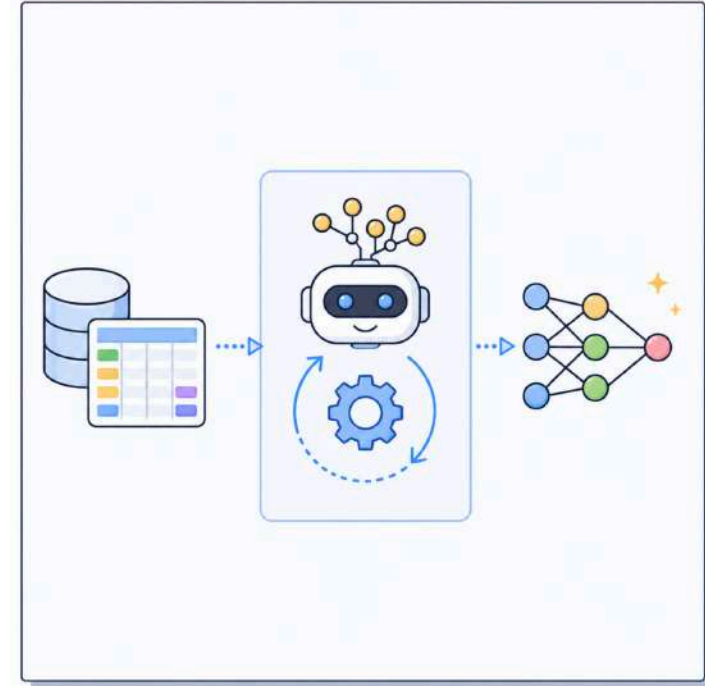
Scenario: The APIs aren't enough

- *Ex:* Vision API sees a "Dog", but you need to know if it's "Your Company's Mascot"

Solution: AutoML on Agent Platform

- **1. Upload Data:** Photos of your mascot
- **2. Click Train:** Auto ML automates the training
- **3. Deploy:** You get a custom API

Best For: Custom model without needing ML expertise





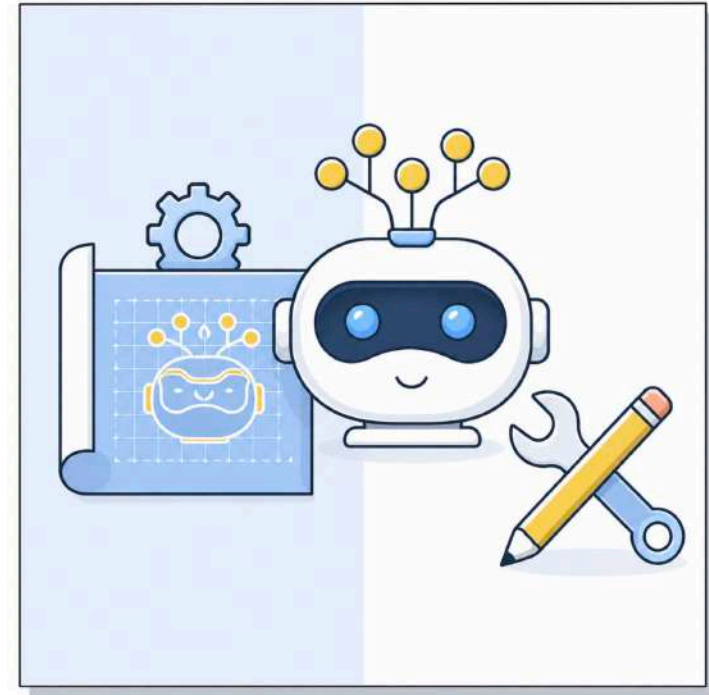
5 - Training ML Models Using ML Frameworks

Scenario: You have a complex ML problem to solve

- You have a team of Data Scientists
- You want to use specific ML Frameworks (**TensorFlow**, **PyTorch**, **scikit-learn**, etc)

Solution: Agent Platform - Custom Training

- Bring your own code
- Custom Training provides the infrastructure needed
- Deploy the model and take complete control of the entire ML cycle





Summary - ML Managed Services in Google Cloud

ML Approach	Description	Google Cloud
Use Pre-built APIs	Use ready-made APIs for text, speech, image, or video tasks	Cloud Natural Language API, Speech-to-Text API, Cloud Vision API, Video Intelligence API, etc.
Use Foundation Models via API	Use models for chat, code generation, multimodal tasks, etc	Gemini Enterprise Agent Platform - Model Garden
Fine-Tune Foundation Models	Take a foundation model and "teach" it your style	Gemini Enterprise Agent Platform - Model Garden
AutoML	Build models from zero using your data without writing code	Gemini Enterprise Agent Platform - AutoML
Training ML Models Using Frameworks	Build and train custom ML models using frameworks such as TensorFlow and PyTorch	Gemini Enterprise Agent Platform - Custom Training



AI and ML in Google Cloud – Scenarios

Scenario	Solution
A company wants to build a chatbot using a foundation model without any fine-tuning	Use Foundation Models from Model Garden
A healthcare company wants their AI assistant to respond in a medical tone with accurate terminology	Fine Tune Foundation Model
An e-commerce platform wants an image generator that matches their brand aesthetic	Fine Tune a Foundation Model (with brand-specific data)
A retail team wants to build a custom model to categorize images but doesn't have ML engineers	Agent Platform - AutoML
A telecom firm has a team of data scientists building a complex churn prediction model using TensorFlow	Build Complex Custom Models (Agent Platform Custom Training)
A logistics company wants to train a route optimization model using PyTorch	Build Complex Custom Models (Agent Platform Custom Training)



in28minutes

Getting Started with

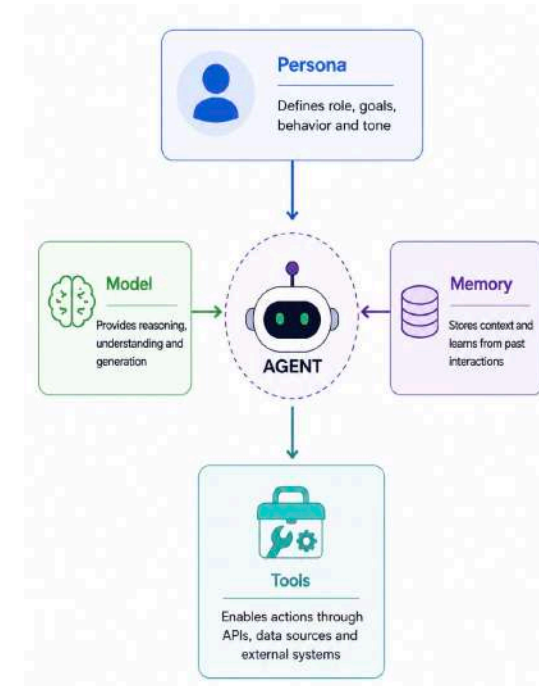
AI Agents - Google Cloud



Getting Started with AI Agents in Google Cloud

Gemini Enterprise Agent Platform: Google's unified platform to build, deploy, and optimize AI agents

- **Agent Studio:** Build agents with a low-code experience
- **Agent Development Kit (ADK):** Open-source framework to build and orchestrate sophisticated agents
- **Agent Runtime:** Managed runtime to deploy, scale, and operate agents in production
- **Models:** Gemini & 200+ supported models - Model Garden
- **Tools & Integrations:** Connect agents to search, RAG, APIs, enterprise systems, and other services
- **Protocols:** Supports MCP for tools and A2A for agent-to-agent collaboration





in28minutes

Getting Ready

For the Certification Exam



What to Expect in the Exam

Exam Format: 50-60 multiple choice/select questions

No Penalty: Wrong answers do not reduce your score

Instant Result: Result shown immediately after submitting

Email Confirmation: Sent within a few days





Certification Exam - My Recommendations

Read the Entire Question

- Identify the **key requirements and constraints**
- Read **all answer options** before choosing

Eliminate Wrong Answers

- Remove clearly incorrect options first

Mark and Review

- Mark difficult questions for review
- Revisit them before final submission

Manage Your Time

- Do not spend too much time on one question
- Move on and return later if needed





Get Ready For Your Exam

Great Effort: You made strong progress throughout the course

Strengthen Your Prep: Reinforce your understanding before taking the exam

- **Refresh Concepts:** Review the presentation end-to-end
- **Practice Tests:** Attempt at least one practice test to understand exam patterns
- **Close Your Gaps:** Revisit areas where you did not do well in practice tests and re-watch videos as needed





in28minutes

WOW!

Congratulations



Let's Clap for You!

Congratulations: Congratulations on your patience and persistence

You Have Put Your Best Foot Forward toward getting certified

Final Step: Review the key concepts and complete your exam preparation

Good Luck!





Do Not Forget!

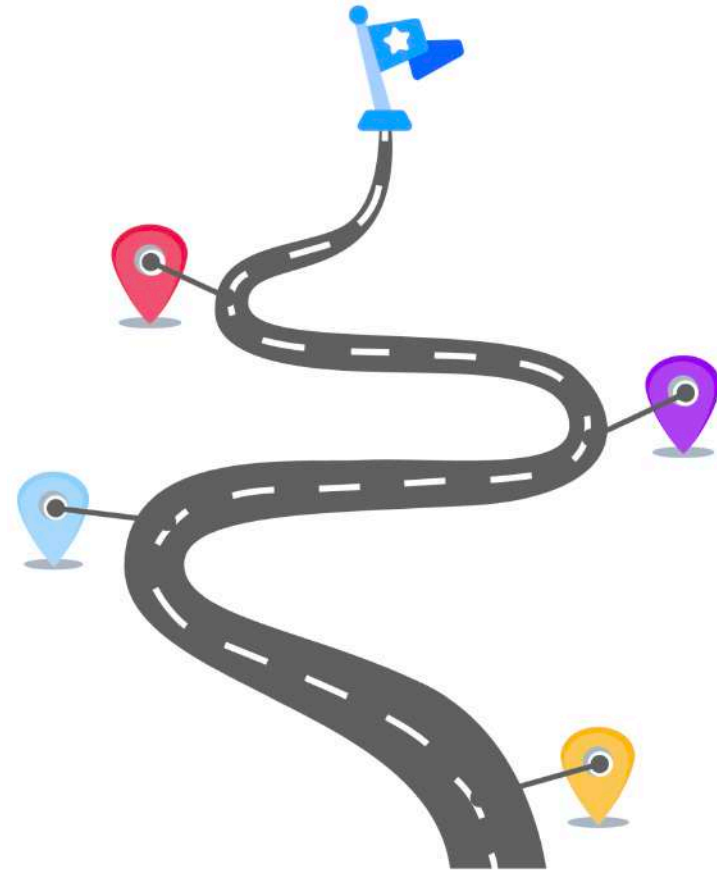
Recommend this Course to your friends

Leave a Review: Your feedback helps future learners

Your Success = Our Success

- Share your success story on LinkedIn (Tag: in28minutes and Ranga Karanam)
- Share your journey and lessons learned in Q&A to inspire others

Continue Your Learning Journey: Checkout our Roadmaps!





01  Generative AI Certifications

02  Cloud Certifications

03  Interview Guides

04  Programming

 **in28minutes**
From Cloud to Generative AI - Learn, Build, and Evolve with in28minutes

